1. Create a GUI (Dark Green Theme) in Python in green with graphs and histograms with CSV files pip install matplotlib numpy pandas tkinter scipy - A cross-regional examination of patterns and environmental drivers of Pseudo-nitzschia harmful algal blooms along the California coast

I'll create a comprehensive GUI for analyzing Pseudo-nitzschia harmful algal blooms along the California coast with a dark green theme.

```
import tkinter as tk
from tkinter import ttk, filedialog, messagebox, scrolledtext
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg, NavigationToolbar2Tk
from matplotlib.figure import Figure
import matplotlib
matplotlib.use('TkAgg')
import numpy as np
import pandas as pd
from scipy import stats
import seaborn as sns
from datetime import datetime
import os
import calendar
from matplotlib import cm

class PseudoNitzschiaGUI:
    def __init__(self, root):
        self.root = root
        self.root.title("🌊 Pseudo-nitzschia HAB Analysis - California Coast")
        self.root.geometry("1600x1000")

        # Dark Ocean Green Theme Colors
        self.colors = {
            'bg_dark': '#0A2A1D',
            'bg_medium': '#1B4D3E',
            'bg_light': '#2E7D5F',
            'accent': '#4CAF85',
            'accent_light': '#66BB95',
            'accent_dark': '#388E6B',
            'text': '#E8F5F0',
            'text_light': '#C8E6D9',
            'grid': '#3E7D5E',
            'chart_bg': '#0A2F21',
            'coast_color': '#2E8B57',
            'bloom_color': '#FF6B6B',
            'warning_color': '#FFA726',
            'safe_color': '#4CAF50',
            'ocean_color': '#1A5276'
        }

        # Initialize data
        self.dataframe = None
        self.current_region = "All"
        self.current_year = "All"
```

```
        self.stations = []

        # Setup GUI
        self.root.configure(bg=self.colors['bg_dark'])
        self.setup_styles()
        self.create_widgets()

    def setup_styles(self):
        """Configure custom ocean green theme styles"""
        style = ttk.Style()

        style.theme_create('ocean_theme', settings={
            'TLabel': {
                'configure': {
                    'background': self.colors['bg_dark'],
                    'foreground': self.colors['text'],
                    'font': ('Segoe UI', 10)
                }
            },
            'TButton': {
                'configure': {
                    'background': self.colors['bg_light'],
                    'foreground': self.colors['text'],
                    'borderwidth': 0,
                    'focuscolor': 'none',
                    'font': ('Segoe UI', 10, 'bold'),
                    'padding': 8
                },
                'map': {
                    'background': [('active', self.colors['accent']),
                                   ('pressed', self.colors['accent_dark'])],
                    'foreground': [('active', '#FFFFFF')]
                }
            },
            'TFrame': {
                'configure': {'background': self.colors['bg_dark']}
            },
            'TLabelframe': {
                'configure': {
                    'background': self.colors['bg_dark'],
                    'foreground': self.colors['text'],
                    'borderwidth': 2,
                    'relief': 'solid'
                }
            },
            'TLabelframe.Label': {
                'configure': {
                    'background': self.colors['bg_dark'],
                    'foreground': self.colors['accent_light'],
                    'font': ('Segoe UI', 11, 'bold')
                }
            },
```

```
        'TNotebook': {
            'configure': {
                'background': self.colors['bg_dark'],
                'tabmargins': [5, 5, 2, 0]
            }
        },
        'TNotebook.Tab': {
            'configure': {
                'background': self.colors['bg_medium'],
                'foreground': self.colors['text_light'],
                'padding': [15, 8],
                'font': ('Segoe UI', 10, 'bold')
            },
            'map': {
                'background': [('selected', self.colors['accent'])],
                'foreground': [('selected', '#FFFFFF')]
            }
        },
        'Treeview': {
            'configure': {
                'background': self.colors['bg_medium'],
                'foreground': self.colors['text'],
                'fieldbackground': self.colors['bg_medium'],
                'rowheight': 25
            }
        },
        'Treeview.Heading': {
            'configure': {
                'background': self.colors['accent_dark'],
                'foreground': self.colors['text'],
                'font': ('Segoe UI', 10, 'bold')
            }
        }
    })
    style.theme_use('ocean_theme')

def create_widgets(self):
    """Create all GUI widgets"""
    # Header with ocean theme
    header_frame = tk.Frame(self.root,
                            bg=self.colors['accent_dark'],
                            height=90,
                            highlightthickness=2,
                            highlightbackground=self.colors['accent_light'])
    header_frame.pack(fill=tk.X)
    header_frame.pack_propagate(False)

    title = tk.Label(
        header_frame,
        text="🌊 Pseudo-nitzschia HAB Analysis - California Coast",
        font=('Segoe UI', 24, 'bold'),
        bg=self.colors['accent_dark'],
```

```
    fg='#FFFFFF',
    bd=0
)
title.pack(pady=15)

subtitle = tk.Label(
    header_frame,
    text="Cross-regional examination of patterns and environmental drivers",
    font=('Segoe UI', 11),
    bg=self.colors['accent_dark'],
    fg=self.colors['text_light']
)
subtitle.pack()

# Main Container
main_container = ttk.Frame(self.root)
main_container.pack(fill=tk.BOTH, expand=True, padx=15, pady=15)

# Create Notebook for Tabs
self.notebook = ttk.Notebook(main_container)
self.notebook.pack(fill=tk.BOTH, expand=True)

# Create Tabs
self.tab_data = ttk.Frame(self.notebook)
self.tab_overview = ttk.Frame(self.notebook)
self.tab_temporal = ttk.Frame(self.notebook)
self.tab_spatial = ttk.Frame(self.notebook)
self.tab_environmental = ttk.Frame(self.notebook)
self.tab_statistics = ttk.Frame(self.notebook)
self.tab_hazard = ttk.Frame(self.notebook)

self.notebook.add(self.tab_data, text="📁 DATA")
self.notebook.add(self.tab_overview, text="📊 OVERVIEW")
self.notebook.add(self.tab_temporal, text="📅 TEMPORAL")
self.notebook.add(self.tab_spatial, text="🗺️ SPATIAL")
self.notebook.add(self.tab_environmental, text="🌡️ ENVIRONMENTAL")
self.notebook.add(self.tab_statistics, text="📈 STATISTICS")
self.notebook.add(self.tab_hazard, text="    HAZARD")

# Setup each tab
self.setup_data_tab()
self.setup_overview_tab()
self.setup_temporal_tab()
self.setup_spatial_tab()
self.setup_environmental_tab()
self.setup_statistics_tab()
self.setup_hazard_tab()

# Status Bar
self.status_bar = tk.Label(
    self.root,
    text="🚀 Ready - Load Pseudo-nitzschia data to begin analysis",
```

```
            bd=1,
            relief=tk.SUNKEN,
            anchor=tk.W,
            bg=self.colors['accent'],
            fg='#FFFFFF',
            font=('Segoe UI', 10)
        )
        self.status_bar.pack(side=tk.BOTTOM, fill=tk.X)

    def setup_data_tab(self):
        """Setup data management tab"""
        # Left Panel - Controls
        left_panel = ttk.Frame(self.tab_data)
        left_panel.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 15))

        # Data Loading Section
        load_frame = ttk.LabelFrame(left_panel, text="DATA LOADING", padding=15)
        load_frame.pack(fill=tk.X, pady=(0, 15))

        btn_load = ttk.Button(load_frame, text="📂 LOAD HAB DATA",
                              command=self.load_hab_data, width=25)
        btn_load.pack(pady=5)

        btn_sample = ttk.Button(load_frame, text="🧪 LOAD SAMPLE DATA",
                                command=self.load_sample_data, width=25)
        btn_sample.pack(pady=5)

        btn_export = ttk.Button(load_frame, text="💾 EXPORT RESULTS",
                                command=self.export_results, width=25)
        btn_export.pack(pady=5)

        # Region Filtering
        filter_frame = ttk.LabelFrame(left_panel, text="REGION FILTERING", padding=15)
        filter_frame.pack(fill=tk.X, pady=(0, 15))

        ttk.Label(filter_frame, text="Select Region:").pack(anchor=tk.W, pady=(0, 5))
        self.region_var = tk.StringVar(value="All")
        self.region_combo = ttk.Combobox(filter_frame, textvariable=self.region_var,
                                         state='readonly', width=20)
        self.region_combo.pack(fill=tk.X, pady=(0, 10))

        ttk.Label(filter_frame, text="Select Year:").pack(anchor=tk.W, pady=(0, 5))
        self.year_var = tk.StringVar(value="All")
        self.year_combo = ttk.Combobox(filter_frame, textvariable=self.year_var,
                                       state='readonly', width=20)
        self.year_combo.pack(fill=tk.X, pady=(0, 10))

        btn_apply = ttk.Button(filter_frame, text="🔍 APPLY FILTERS",
                               command=self.apply_filters)
        btn_apply.pack(fill=tk.X, pady=5)

        btn_reset = ttk.Button(filter_frame, text="🔚 RESET FILTERS",
```

```
                    command=self.reset_filters)
        btn_reset.pack(fill=tk.X, pady=5)

        # Dataset Information
        info_frame = ttk.LabelFrame(left_panel, text="DATASET INFO", padding=15)
        info_frame.pack(fill=tk.BOTH, expand=True)

        self.info_text = scrolledtext.ScrolledText(

         info_frame,
            height=15,
            bg=self.colors['bg_medium'],
            fg=self.colors['text'],
            font=('Consolas', 9),
            wrap=tk.WORD
        )
        self.info_text.pack(fill=tk.BOTH, expand=True)

        # Right Panel - Data Display
        right_panel = ttk.Frame(self.tab_data)
        right_panel.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

        # Treeview for data
        tree_frame = ttk.Frame(right_panel)
        tree_frame.pack(fill=tk.BOTH, expand=True)

        # Scrollbars
        tree_scroll_y = ttk.Scrollbar(tree_frame)
        tree_scroll_y.pack(side=tk.RIGHT, fill=tk.Y)

        tree_scroll_x = ttk.Scrollbar(tree_frame, orient=tk.HORIZONTAL)
        tree_scroll_x.pack(side=tk.BOTTOM, fill=tk.X)

        # Create Treeview
        self.data_tree = ttk.Treeview(
            tree_frame,
            yscrollcommand=tree_scroll_y.set,
            xscrollcommand=tree_scroll_x.set,
            selectmode='browse'
        )
        self.data_tree.pack(fill=tk.BOTH, expand=True)

        tree_scroll_y.config(command=self.data_tree.yview)
        tree_scroll_x.config(command=self.data_tree.xview)

    def setup_overview_tab(self):
        """Setup overview dashboard tab"""
        # Left Panel - Summary Stats
        left_panel = ttk.Frame(self.tab_overview)
        left_panel.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 15))

        # Summary Statistics
```

```
stats_frame = ttk.LabelFrame(left_panel, text="SUMMARY STATISTICS", padding=15)
stats_frame.pack(fill=tk.X, pady=(0, 15))

self.stats_text = scrolledtext.ScrolledText(
    stats_frame,
    height=20,
    bg=self.colors['bg_medium'],
    fg=self.colors['text'],
    font=('Consolas', 9)
)
self.stats_text.pack(fill=tk.BOTH, expand=True)

# Risk Assessment
risk_frame = ttk.LabelFrame(left_panel, text="RISK ASSESSMENT", padding=15)
risk_frame.pack(fill=tk.BOTH, expand=True)

self.risk_text = scrolledtext.ScrolledText(
    risk_frame,
    height=10,
    bg=self.colors['bg_medium'],
    fg=self.colors['text'],
    font=('Consolas', 9)
)
self.risk_text.pack(fill=tk.BOTH, expand=True)

# Right Panel - Plots
right_panel = ttk.Frame(self.tab_overview)
right_panel.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

# Create notebook for plots
plot_notebook = ttk.Notebook(right_panel)
plot_notebook.pack(fill=tk.BOTH, expand=True)

# Distribution Plot Tab
dist_tab = ttk.Frame(plot_notebook)
plot_notebook.add(dist_tab, text=" DISTRIBUTION")

self.fig_dist = Figure(figsize=(10, 8), facecolor=self.colors['chart_bg'])
self.ax_dist = self.fig_dist.add_subplot(111)
self.canvas_dist = FigureCanvasTkAgg(self.fig_dist, dist_tab)
self.canvas_dist.get_tk_widget().pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

# Time Series Tab
ts_tab = ttk.Frame(plot_notebook)
plot_notebook.add(ts_tab, text=" TIME SERIES")

self.fig_ts = Figure(figsize=(10, 8), facecolor=self.colors['chart_bg'])
self.ax_ts = self.fig_ts.add_subplot(111)
self.canvas_ts = FigureCanvasTkAgg(self.fig_ts, ts_tab)
self.canvas_ts.get_tk_widget().pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

# Regional Comparison Tab
```

```
        region_tab = ttk.Frame(plot_notebook)
        plot_notebook.add(region_tab, text=" 🗺️ REGIONAL")

        self.fig_region = Figure(figsize=(10, 8), facecolor=self.colors['chart_bg'])
        self.ax_region = self.fig_region.add_subplot(111)
        self.canvas_region = FigureCanvasTkAgg(self.fig_region, region_tab)
        self.canvas_region.get_tk_widget().pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

    def setup_temporal_tab(self):
        """Setup temporal analysis tab"""
        # Control Panel
        control_panel = ttk.Frame(self.tab_temporal)
        control_panel.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 15))

        # Temporal Analysis Options
        temp_frame = ttk.LabelFrame(control_panel, text="TEMPORAL ANALYSIS", padding=15)
        temp_frame.pack(fill=tk.X, pady=(0, 15))

        self.temp_type = tk.StringVar(value="monthly")

        temp_types = [
            ("📅 Monthly Patterns", "monthly"),
            ("🗓️ Seasonal Analysis", "seasonal"),
            ("📈 Yearly Trends", "yearly"),
            ("📊 Inter-annual", "interannual"),
            ("⏰ Diurnal Patterns", "diurnal"),
            ("📋 Weekly Aggregation", "weekly")
        ]

        for text, value in temp_types:
            rb = ttk.Radiobutton(temp_frame, text=text, variable=self.temp_type,
                                 value=value)
            rb.pack(anchor=tk.W, pady=3)

        # Variable Selection
        var_frame = ttk.LabelFrame(control_panel, text="VARIABLE SELECTION", padding=15)
        var_frame.pack(fill=tk.X, pady=(0, 15))

        ttk.Label(var_frame, text="Primary Variable:").pack(anchor=tk.W, pady=(0, 5))
        self.temp_var = tk.StringVar()
        self.temp_var_combo = ttk.Combobox(var_frame, textvariable=self.temp_var,
                                           state='readonly')
        self.temp_var_combo.pack(fill=tk.X, pady=(0, 10))

        ttk.Label(var_frame, text="Secondary Variable:").pack(anchor=tk.W, pady=(0, 5))
        self.temp_var2 = tk.StringVar()
        self.temp_var2_combo = ttk.Combobox(var_frame, textvariable=self.temp_var2,
                                            state='readonly')
        self.temp_var2_combo.pack(fill=tk.X)

        # Buttons
        btn_frame = ttk.Frame(control_panel)
```

```
        btn_frame.pack(fill=tk.X, pady=10)

        btn_generate = ttk.Button(btn_frame, text="📊 GENERATE ANALYSIS",
                                  command=self.generate_temporal_analysis)
        btn_generate.pack(fill=tk.X, pady=5)

        # Plot Area
        plot_area = ttk.Frame(self.tab_temporal)
        plot_area.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

        # Create matplotlib figure
        self.fig_temp = Figure(figsize=(10, 8), facecolor=self.colors['chart_bg'])
        self.ax_temp = self.fig_temp.add_subplot(111)

        # Create canvas
        self.canvas_temp = FigureCanvasTkAgg(self.fig_temp, plot_area)
        self.canvas_temp.get_tk_widget().pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

        # Add toolbar
        toolbar = NavigationToolbar2Tk(self.canvas_temp, plot_area)
        toolbar.update()

    def setup_spatial_tab(self):
        """Setup spatial analysis tab"""
        # Control Panel
        control_panel = ttk.Frame(self.tab_spatial)
        control_panel.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 15))

        # Spatial Analysis Options
        spatial_frame = ttk.LabelFrame(control_panel, text="SPATIAL ANALYSIS", padding=15)
        spatial_frame.pack(fill=tk.X, pady=(0, 15))

        self.spatial_type = tk.StringVar(value="regional")

        spatial_types = [
            ("🗺️ Regional Distribution", "regional"),
            ("📍 Station Comparison", "stations"),
            ("🌊 Coastline Patterns", "coastline"),
            ("📊 Hotspot Detection", "hotspots"),
            ("📈 Spatial Trends", "trends"),
            ("🎯 Cluster Analysis", "clusters")
        ]

        for text, value in spatial_types:
            rb = ttk.Radiobutton(spatial_frame, text=text, variable=self.spatial_type,
                                 value=value)
            rb.pack(anchor=tk.W, pady=3)

        # Variable Selection
        var_frame = ttk.LabelFrame(control_panel, text="VARIABLE SELECTION", padding=15)
        var_frame.pack(fill=tk.X, pady=(0, 15))
```

```
    ttk.Label(var_frame, text="Spatial Variable:").pack(anchor=tk.W, pady=(0, 5))
    self.spatial_var = tk.StringVar()
    self.spatial_var_combo = ttk.Combobox(var_frame, textvariable=self.spatial_var,
                                          state='readonly')
    self.spatial_var_combo.pack(fill=tk.X, pady=(0, 10))

    # Time Selection
    time_frame = ttk.LabelFrame(control_panel, text="TIME SELECTION", padding=15)
    time_frame.pack(fill=tk.X, pady=(0, 15))

    ttk.Label(time_frame, text="Year:").pack(anchor=tk.W, pady=(0, 5))
    self.spatial_year = tk.StringVar(value="All")
    self.spatial_year_combo = ttk.Combobox(time_frame, textvariable=self.spatial_year,
                                           state='readonly')
    self.spatial_year_combo.pack(fill=tk.X, pady=(0, 10))

    ttk.Label(time_frame, text="Month:").pack(anchor=tk.W, pady=(0, 5))
    self.spatial_month = tk.StringVar(value="All")
    self.spatial_month_combo = ttk.Combobox(time_frame, textvariable=self.spatial_month,
                                            state='readonly')
    self.spatial_month_combo.pack(fill=tk.X)

    # Buttons
    btn_frame = ttk.Frame(control_panel)
    btn_frame.pack(fill=tk.X, pady=10)

    btn_generate = ttk.Button(btn_frame, text=" 🗺️ GENERATE SPATIAL PLOT",
                              command=self.generate_spatial_analysis)
    btn_generate.pack(fill=tk.X, pady=5)

    # Plot Area
    plot_area = ttk.Frame(self.tab_spatial)
    plot_area.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

    # Create matplotlib figure
    self.fig_spatial = Figure(figsize=(10, 8), facecolor=self.colors['chart_bg'])
    self.ax_spatial = self.fig_spatial.add_subplot(111)


    # Create canvas
    self.canvas_spatial = FigureCanvasTkAgg(self.fig_spatial, plot_area)
    self.canvas_spatial.get_tk_widget().pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

    # Add toolbar
    toolbar = NavigationToolbar2Tk(self.canvas_spatial, plot_area)
    toolbar.update()

def setup_environmental_tab(self):
    """Setup environmental drivers tab"""
    # Control Panel
    control_panel = ttk.Frame(self.tab_environmental)
    control_panel.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 15))
```

```
        # Environmental Variables
        env_frame = ttk.LabelFrame(control_panel, text="ENVIRONMENTAL DRIVERS",
padding=15)
        env_frame.pack(fill=tk.X, pady=(0, 15))

        self.env_type = tk.StringVar(value="correlation")

        env_types = [
            ("🔗Correlation Analysis", "correlation"),
            ("📈 Regression Models", "regression"),
            ("📊 Threshold Analysis", "thresholds"),
            ("🌡️ Temperature Effects", "temperature"),
            ("🧂 Salinity Effects", "salinity"),
            ("💡 Nutrient Analysis", "nutrients"),
            ("🌊 Upwelling Indicators", "upwelling"),
            ("🌀 Multivariate", "multivariate")
        ]

        for text, value in env_types:
            rb = ttk.Radiobutton(env_frame, text=text, variable=self.env_type,
                                 value=value)
            rb.pack(anchor=tk.W, pady=3)

        # Variable Selection
        var_frame = ttk.LabelFrame(control_panel, text="VARIABLE SELECTION", padding=15)
        var_frame.pack(fill=tk.X, pady=(0, 15))

        ttk.Label(var_frame, text="Pseudo-nitzschia Variable:").pack(anchor=tk.W, pady=(0, 5))
        self.env_pn_var = tk.StringVar()
        self.env_pn_combo = ttk.Combobox(var_frame, textvariable=self.env_pn_var,
                                         state='readonly')
        self.env_pn_combo.pack(fill=tk.X, pady=(0, 10))

        ttk.Label(var_frame, text="Environmental Variable:").pack(anchor=tk.W, pady=(0, 5))
        self.env_env_var = tk.StringVar()
        self.env_env_combo = ttk.Combobox(var_frame, textvariable=self.env_env_var,
                                          state='readonly')
        self.env_env_combo.pack(fill=tk.X)

        # Buttons
        btn_frame = ttk.Frame(control_panel)
        btn_frame.pack(fill=tk.X, pady=10)

        btn_generate = ttk.Button(btn_frame, text="🌡️ ANALYZE DRIVERS",
                                  command=self.analyze_environmental_drivers)
        btn_generate.pack(fill=tk.X, pady=5)

        # Results Area
        results_frame = ttk.LabelFrame(control_panel, text="ANALYSIS RESULTS", padding=15)
        results_frame.pack(fill=tk.BOTH, expand=True)
```

```
        self.env_results = scrolledtext.ScrolledText(
            results_frame,
            height=15,
            bg=self.colors['bg_medium'],
            fg=self.colors['text'],
            font=('Consolas', 9)
        )
        self.env_results.pack(fill=tk.BOTH, expand=True)

        # Plot Area
        plot_area = ttk.Frame(self.tab_environmental)
        plot_area.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

        # Create matplotlib figure
        self.fig_env = Figure(figsize=(10, 8), facecolor=self.colors['chart_bg'])
        self.ax_env = self.fig_env.add_subplot(111)

        # Create canvas
        self.canvas_env = FigureCanvasTkAgg(self.fig_env, plot_area)
        self.canvas_env.get_tk_widget().pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

        # Add toolbar
        toolbar = NavigationToolbar2Tk(self.canvas_env, plot_area)
        toolbar.update()

    def setup_statistics_tab(self):
        """Setup statistical analysis tab"""
        # Left Panel - Statistical Tests
        left_panel = ttk.Frame(self.tab_statistics)
        left_panel.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 15))

        # Statistical Tests
        stats_frame = ttk.LabelFrame(left_panel, text="STATISTICAL TESTS", padding=15)
        stats_frame.pack(fill=tk.X, pady=(0, 15))

        self.stats_test = tk.StringVar(value="anova")

        test_options = [
            ("📊 ANOVA (Regional Differences)", "anova"),
            ("📈 T-Test (Pairwise)", "ttest"),
            ("📉 Correlation Matrix", "correlation"),
            ("📐 Linear Regression", "regression"),
            ("📶 Time Series Analysis", "timeseries"),
            ("📊 Cluster Analysis", "cluster"),
            ("📈 PCA Analysis", "pca"),
            ("📊 Mann-Whitney U", "mannwhitney")
        ]

        for text, value in test_options:
            rb = ttk.Radiobutton(stats_frame, text=text, variable=self.stats_test,
                                 value=value)
            rb.pack(anchor=tk.W, pady=3)
```

```
# Variable Selection
var_frame = ttk.LabelFrame(left_panel, text="VARIABLE SELECTION", padding=15)
var_frame.pack(fill=tk.X, pady=(0, 15))

ttk.Label(var_frame, text="Group Variable:").pack(anchor=tk.W, pady=(0, 5))
self.stats_group = tk.StringVar()
self.stats_group_combo = ttk.Combobox(var_frame, textvariable=self.stats_group,
                                      state='readonly')
self.stats_group_combo.pack(fill=tk.X, pady=(0, 10))

ttk.Label(var_frame, text="Test Variable:").pack(anchor=tk.W, pady=(0, 5))
self.stats_var = tk.StringVar()
self.stats_var_combo = ttk.Combobox(var_frame, textvariable=self.stats_var,
                                    state='readonly')
self.stats_var_combo.pack(fill=tk.X)

# Buttons
btn_frame = ttk.Frame(left_panel)
btn_frame.pack(fill=tk.X, pady=10)

btn_calculate = ttk.Button(btn_frame, text=" 📊 CALCULATE STATISTICS",
                           command=self.calculate_statistics)
btn_calculate.pack(fill=tk.X, pady=5)

# Results Display
results_frame = ttk.LabelFrame(left_panel, text="STATISTICAL RESULTS", padding=15)
results_frame.pack(fill=tk.BOTH, expand=True)

self.stats_results = scrolledtext.ScrolledText(
    results_frame,
    height=15,
    bg=self.colors['bg_medium'],
    fg=self.colors['text'],
    font=('Consolas', 9)
)
self.stats_results.pack(fill=tk.BOTH, expand=True)

# Right Panel - Plot Area
right_panel = ttk.Frame(self.tab_statistics)
right_panel.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

# Create matplotlib figure
self.fig_stats = Figure(figsize=(10, 8), facecolor=self.colors['chart_bg'])
self.ax_stats = self.fig_stats.add_subplot(111)

# Create canvas
self.canvas_stats = FigureCanvasTkAgg(self.fig_stats, right_panel)
self.canvas_stats.get_tk_widget().pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

# Add toolbar
toolbar = NavigationToolbar2Tk(self.canvas_stats, right_panel)
```

```
        toolbar.update()

    def setup_hazard_tab(self):
        """Setup hazard assessment tab"""
        # Left Panel - Hazard Assessment
        left_panel = ttk.Frame(self.tab_hazard)
        left_panel.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 15))

        # Hazard Levels
        hazard_frame = ttk.LabelFrame(left_panel, text="HAZARD LEVELS", padding=15)
        hazard_frame.pack(fill=tk.X, pady=(0, 15))

        info_text = """
Pseudo-nitzschia Hazard Levels:

🔴 HIGH HAZARD (Bloom Conditions):
• Cell density > 10,000 cells/L
• DA concentration > 20 µg/L
• Shellfish closure likely

🟡 MODERATE HAZARD (Watch):
• Cell density: 1,000-10,000 cells/L
• DA concentration: 5-20 µg/L
• Increased monitoring needed

🟢 LOW HAZARD (Normal):
• Cell density < 1,000 cells/L
• DA concentration < 5 µg/L
• Normal operations

Environmental Risk Factors:
• Sea Surface Temp: 12-18°C optimal
• Nutrient levels: High N/P ratios
• Upwelling conditions favorable
• Low salinity events
"""
        hazard_text = tk.Text(
            hazard_frame,
            height=15,
            bg=self.colors['bg_medium'],
            fg=self.colors['text'],
            font=('Consolas', 9),
            wrap=tk.WORD
        )
        hazard_text.insert(1.0, info_text)
        hazard_text.config(state=tk.DISABLED)
        hazard_text.pack(fill=tk.BOTH, expand=True)

        # Risk Assessment
        risk_frame = ttk.LabelFrame(left_panel, text="RISK ASSESSMENT", padding=15)
        risk_frame.pack(fill=tk.BOTH, expand=True)
```

```
        ttk.Label(risk_frame, text="Current Hazard Level:").pack(anchor=tk.W, pady=(0, 5))

        self.hazard_level = tk.StringVar()
        hazard_combo = ttk.Combobox(risk_frame, textvariable=self.hazard_level,
                                    values=["Low", "Moderate", "High"], state='readonly')
        hazard_combo.pack(fill=tk.X, pady=(0, 10))

        btn_assess = ttk.Button(risk_frame, text="⚠ ASSESS RISK",
                                command=self.assess_risk)
        btn_assess.pack(fill=tk.X, pady=5)

        self.risk_output = scrolledtext.ScrolledText(
            risk_frame,
            height=10,
            bg=self.colors['bg_medium'],
            fg=self.colors['text'],
            font=('Consolas', 9)
        )
        self.risk_output.pack(fill=tk.BOTH, expand=True)

        # Right Panel - Hazard Maps
        right_panel = ttk.Frame(self.tab_hazard)
        right_panel.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

        # Create notebook for hazard plots
        hazard_notebook = ttk.Notebook(right_panel)
        hazard_notebook.pack(fill=tk.BOTH, expand=True)

        # Hazard
    Map Tab
        map_tab = ttk.Frame(hazard_notebook)
        hazard_notebook.add(map_tab, text=" 🗺️ HAZARD MAP")

        self.fig_hazard = Figure(figsize=(10, 8), facecolor=self.colors['chart_bg'])
        self.ax_hazard = self.fig_hazard.add_subplot(111)
        self.canvas_hazard = FigureCanvasTkAgg(self.fig_hazard, map_tab)
        self.canvas_hazard.get_tk_widget().pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

        # Threshold Analysis Tab
        threshold_tab = ttk.Frame(hazard_notebook)
        hazard_notebook.add(threshold_tab, text=" 📊 THRESHOLDS")

        self.fig_threshold = Figure(figsize=(10, 8), facecolor=self.colors['chart_bg'])
        self.ax_threshold = self.fig_threshold.add_subplot(111)
        self.canvas_threshold = FigureCanvasTkAgg(self.fig_threshold, threshold_tab)
        self.canvas_threshold.get_tk_widget().pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

    def load_hab_data(self):
        """Load HAB data from CSV file"""
        file_path = filedialog.askopenfilename(
            title="Select Pseudo-nitzschia HAB Data",
            filetypes=[
```

```
            ("CSV files", "*.csv"),
            ("Excel files", "*.xlsx *.xls"),
            ("All files", "*.*")
        ]
    )

    if file_path:
        try:
            if file_path.endswith('.csv'):
                self.dataframe = pd.read_csv(file_path)
            else:
                self.dataframe = pd.read_excel(file_path)

            # Clean column names
            self.dataframe.columns = [col.strip() for col in self.dataframe.columns]

            self.update_data_display()
            self.update_all_comboboxes()
            self.update_info_panel(file_path)
            self.generate_overview()

            self.status_bar.config(
                text=f"✓ Loaded: {os.path.basename(file_path)} "
                     f"({self.dataframe.shape[0]} samples)"
            )

            messagebox.showinfo("Success",
                f"Pseudo-nitzschia data loaded successfully!\n\n"
                f"File: {os.path.basename(file_path)}\n"
                f"Samples: {self.dataframe.shape[0]}\n"
                f"Variables: {self.dataframe.shape[1]}")

        except Exception as e:
            messagebox.showerror("Load Error",
                f"Failed to load file:\n\n{str(e)}")
            self.status_bar.config(text="✗ Failed to load data")

def load_sample_data(self):
    """Load sample Pseudo-nitzschia data"""
    try:
        np.random.seed(42)
        n_samples = 500

        # California coastal regions
        california_regions = {
            'North Coast': ['Crescent City', 'Trinidad', 'Eureka'],
            'North Central': ['Fort Bragg', 'Point Arena', 'Bodega Bay'],
            'Central Coast': ['San Francisco', 'Monterey', 'Morro Bay'],
            'South Central': ['Santa Barbara', 'Ventura', 'Channel Islands'],
            'Southern Coast': ['Los Angeles', 'Orange County', 'San Diego']
        }
```

```
# Generate sample data
sample_data = []

for year in range(2010, 2023):
    for month in range(1, 13):
        for region, stations in california_regions.items():
            for station in stations:
                # Base values with seasonal patterns
                base_temp = 12 + 6 * np.sin(2*np.pi*month/12) + np.random.normal(0, 2)
                base_salinity = 33 + np.random.normal(0, 1)
                base_nutrients = np.random.uniform(5, 25)

                # Pseudo-nitzschia parameters
                # Higher in spring/fall, lower in winter
                seasonal_factor = 0.5 + 0.5 * np.sin(2*np.pi*(month-3)/12)
                region_factor = {'North Coast': 0.8, 'North Central': 1.2,
                                 'Central Coast': 1.5, 'South Central': 1.0,
                                 'Southern Coast': 0.7}[region]

                # Cell density (cells/L)
                base_density = np.random.lognormal(mean=6, sigma=1.5)
                bloom_factor = seasonal_factor * region_factor

                if np.random.random() < 0.1:  # 10% chance of bloom
                    bloom_factor *= np.random.uniform(5, 20)

                cell_density = base_density * bloom_factor

                # Domoic acid (µg/L) - related to cell density
                da_concentration = 0.01 * cell_density * np.random.uniform(0.5, 2)

                # Environmental parameters
                temperature = base_temp + np.random.normal(0, 1)
                salinity = base_salinity + np.random.normal(0, 0.5)
                nitrate = base_nutrients * np.random.uniform(0.8, 1.2)
                phosphate = nitrate / 16 * np.random.uniform(0.8, 1.2)  # Redfield ratio
                silicate = nitrate * np.random.uniform(0.5, 2)

                # Upwelling index
                upwelling_index = np.random.uniform(-100, 200)
                if region in ['North Coast', 'North Central']:
                    upwelling_index += 100

                # Chlorophyll
                chlorophyll = 0.1 * cell_density**0.5 * np.random.uniform(0.5, 1.5)

                sample_data.append({
                    'Date': f'{year}-{month:02d}-15',
                    'Year': year,
                    'Month': month,
                    'Season': self.get_season(month),
                    'Region': region,
```

```
                    'Station': station,
                    'Latitude': np.random.uniform(32.5, 42),
                    'Longitude': np.random.uniform(-124.5, -117),
                    'Pseudo_nitzschia_cells_L': int(cell_density),
                    'Domoic_Acid_ug_L': round(da_concentration, 3),
                    'Temperature_C': round(temperature, 1),
                    'Salinity_PSU': round(salinity, 1),
                    'Nitrate_uM': round(nitrate, 2),
                    'Phosphate_uM': round(phosphate, 2),
                    'Silicate_uM': round(silicate, 2),
                    'Chlorophyll_mg_m3': round(chlorophyll, 3),
                    'Upwelling_Index': round(upwelling_index, 1),
                    'Wind_Speed_m_s': np.random.uniform(2, 10),
                    'Sea_Level_Pressure_hPa': np.random.uniform(1010, 1020),
                    'Precipitation_mm': np.random.exponential(2),
                    'PAR_umol_m2_s': np.random.uniform(100, 600),
                    'pH': np.random.uniform(7.8, 8.3),
                    'Dissolved_Oxygen_mg_L': np.random.uniform(6, 9)
                })

        self.dataframe = pd.DataFrame(sample_data)
        self.update_data_display()
        self.update_all_comboboxes()
        self.update_info_panel("Sample Pseudo-nitzschia Data")
        self.generate_overview()

        self.status_bar.config(
            text=f"✓ Generated sample data with {len(self.dataframe)} samples"
        )

        messagebox.showinfo("Sample Data Loaded",
            "Sample Pseudo-nitzschia HAB data has been generated!\n\n"
            "This includes:\n"
            "• 13 years of monthly data (2010-2022)\n"
            "• 5 California coastal regions\n"
            "• 15 monitoring stations\n"
            "• Pseudo-nitzschia cell counts\n"
            "• Domoic acid concentrations\n"
            "• 15+ environmental parameters")

    except Exception as e:
        messagebox.showerror("Error", f"Failed to generate sample data:\n{str(e)}")

def get_season(self, month):
    """Get season from month"""
    if month in [12, 1, 2]:
        return 'Winter'
    elif month in [3, 4, 5]:
        return 'Spring'
    elif month in [6, 7, 8]:
        return 'Summer'
    else:
```

```
            return 'Fall'

    def update_data_display(self):
        """Update the treeview with loaded data"""
        # Clear existing data
        for item in self.data_tree.get_children():
            self.data_tree.delete(item)

        if self.dataframe is None:
            return

        # Set up columns
        columns = list(self.dataframe.columns)
        self.data_tree['columns'] = columns

        # Configure column headings
        for col in columns:
            self.data_tree.heading(col, text=col, anchor=tk.CENTER)
            col_width = max(len(col) * 8, 80)
            self.data_tree.column(col, width=col_width, anchor=tk.CENTER, minwidth=60)

        # Insert first 100 rows
        max_rows = min(100, len(self.dataframe))
        for idx in range(max_rows):
            row = self.dataframe.iloc[idx]

values = [row[col] for col in columns]
            self.data_tree.insert('', 'end', values=values)

        if len(self.dataframe) > 100:
            self.status_bar.config(text=f"Displaying first 100 of {len(self.dataframe)} rows")

    def update_all_comboboxes(self):
        """Update all comboboxes with column names and values"""
        if self.dataframe is None:
            return

        columns = list(self.dataframe.columns)
        numeric_cols = self.dataframe.select_dtypes(include=[np.number]).columns.tolist()

        # Update region and year comboboxes
        if 'Region' in self.dataframe.columns:
            regions = ['All'] + sorted(self.dataframe['Region'].unique().tolist())
            self.region_combo['values'] = regions
            self.region_var.set('All')

        if 'Year' in self.dataframe.columns:
            years = ['All'] + sorted(self.dataframe['Year'].unique().astype(str).tolist())
            self.year_combo['values'] = years
            self.year_var.set('All')
            self.spatial_year_combo['values'] = years
```

```
        if 'Month' in self.dataframe.columns:
            months = ['All'] + [calendar.month_name[i] for i in range(1, 13)]
            self.spatial_month_combo['values'] = months
            self.spatial_month.set('All')

        # Update variable comboboxes
        variable_combos = [
            (self.temp_var_combo, self.temp_var, numeric_cols),
            (self.temp_var2_combo, self.temp_var2, numeric_cols),
            (self.spatial_var_combo, self.spatial_var, numeric_cols),
            (self.env_pn_combo, self.env_pn_var, numeric_cols),
            (self.env_env_combo, self.env_env_var, numeric_cols),
            (self.stats_group_combo, self.stats_group, columns),
            (self.stats_var_combo, self.stats_var, numeric_cols)
        ]

        for combo, var, values in variable_combos:
            combo['values'] = values
            if values:
                # Prefer Pseudo-nitzschia related variables
                pn_vars = [col for col in values if 'nitzschia' in col.lower() or 'domoic' in col.lower()]
                if pn_vars:
                    var.set(pn_vars[0])
                else:
                    var.set(values[0])

    def update_info_panel(self, file_path):
        """Update information panel"""
        info = "PSEUDO-NITZSCHIA DATASET INFORMATION\n"
        info += "=" * 60 + "\n\n"
        info += f"Source: {file_path}\n"
        info += f"Samples: {self.dataframe.shape[0]:,}\n"
        info += f"Variables: {self.dataframe.shape[1]:,}\n\n"

        # Check for key columns
        if 'Region' in self.dataframe.columns:
            regions = self.dataframe['Region'].unique()
            info += f"Regions: {len(regions)}\n"
            for region in regions:
                count = len(self.dataframe[self.dataframe['Region'] == region])
                info += f"  • {region}: {count:,} samples\n"
            info += "\n"

        if 'Year' in self.dataframe.columns:
            years = sorted(self.dataframe['Year'].unique())
            info += f"Years: {len(years)} ({min(years)}-{max(years)})\n\n"

        # Check for Pseudo-nitzschia data
        pn_cols = [col for col in self.dataframe.columns
                   if 'nitzschia' in col.lower() or 'domoic' in col.lower()]

        if pn_cols:
```

```
            info += "PSEUDO-NITZSCHIA METRICS:\n"
            info += "-" * 40 + "\n"
            for col in pn_cols:
                data = self.dataframe[col].dropna()
                if len(data) > 0:
                    info += f"• {col}:\n"
                    info += f"  Range: {data.min():,.0f} - {data.max():,.0f}\n"
                    info += f"  Mean: {data.mean():,.0f}, Median: {data.median():,.0f}\n"
                    info += f"  Samples: {len(data):,}\n\n"

        # Environmental variables
        env_cols = [col for col in self.dataframe.columns
                    if any(keyword in col.lower() for keyword in
                           ['temp', 'salinity', 'nitrate', 'phosphate', 'chlorophyll'])]

        if env_cols:
            info += "KEY ENVIRONMENTAL VARIABLES:\n"
            info += "-" * 40 + "\n"
            for col in env_cols[:5]:  # Show first 5
                data = self.dataframe[col].dropna()
                if len(data) > 0:
                    info += f"• {col}: {data.mean():.2f} ± {data.std():.2f}\n"

        self.info_text.delete(1.0, tk.END)
        self.info_text.insert(1.0, info)
        self.info_text.config(state=tk.DISABLED)

    def apply_filters(self):
        """Apply region and year filters"""
        if self.dataframe is None:
            return

        region = self.region_var.get()
        year = self.year_var.get()

        filtered_df = self.dataframe.copy()

        if region != 'All':
            filtered_df = filtered_df[filtered_df['Region'] == region]

        if year != 'All':
            filtered_df = filtered_df[filtered_df['Year'] == int(year)]

        # Update treeview with filtered data
        for item in self.data_tree.get_children():
            self.data_tree.delete(item)

        columns = list(filtered_df.columns)
        max_rows = min(100, len(filtered_df))

        for idx in range(max_rows):
            row = filtered_df.iloc[idx]
```

```
            values = [row[col] for col in columns]
            self.data_tree.insert('', 'end', values=values)

        filter_text = f"Region: {region}, Year: {year}"
        self.status_bar.config(text=f"✓ Applied filters: {filter_text} ({len(filtered_df)} samples)")

    def reset_filters(self):
        """Reset all filters"""
        self.region_var.set('All')
        self.year_var.set('All')

        if self.dataframe is not None:
            self.update_data_display()
            self.status_bar.config(text="✓ Filters reset")

    def generate_overview(self):
        """Generate overview dashboard"""
        if self.dataframe is None:
            return

        try:
            # Update summary statistics
            self.update_summary_stats()

            # Generate distribution plot
            self.generate_distribution_plot()

            # Generate time series plot
            self.generate_time_series()

            # Generate regional comparison
            self.generate_regional_comparison()

        except Exception as e:
            messagebox.showerror("Overview Error", f"Failed to generate overview:\n{str(e)}")

    def update_summary_stats(self):
        """Update summary statistics"""
        stats_text = "PSEUDO-NITZSCHIA SUMMARY STATISTICS\n"
        stats_text += "=" * 60 + "\n\n"

        # Basic info
        stats_text += f"Total Samples: {len(self.dataframe):,}\n"
        stats_text += f"Time Period: {self.dataframe['Year'].min()} - {self.dataframe['Year'].max()}\n"
        stats_text += f"Regions: {len(self.dataframe['Region'].unique())}\n"
        stats_text += f"Stations: {len(self.dataframe['Station'].unique())}\n\n"

        # Pseudo-nitzschia statistics
        pn_cols = [col for col in self.dataframe.columns
                   if 'nitzschia' in col.lower()]

        if pn_cols:
```

```
            for col in pn_cols:
                data = self.dataframe[col].dropna()
                stats_text += f"{col}:\n"
                stats_text += f"  • Samples: {len(data):,}\n"
                stats_text += f"  • Mean: {data.mean():,.0f}\n"
                stats_text += f"  • Median: {data.median():,.0f}\n"
                stats_text += f"  • Std Dev: {data.std():,.0f}\n"
                stats_text += f"  • Range: {data.min():,.0f} - {data.max():,.0f}\n\n"

        # Bloom detection
        if 'Pseudo_nitzschia_cells_L' in self.dataframe.columns:
            bloom_threshold = 10000  # cells/L
            bloom_samples = self.dataframe[self.dataframe['Pseudo_nitzschia_cells_L'] >
bloom_threshold]
            bloom_percentage = len(bloom_samples) / len(self.dataframe) * 100

            stats_text += "BLOOM STATISTICS:\n"
            stats_text += f"• Bloom threshold: > {bloom_threshold:,} cells/L\n"
            stats_text += f"• Bloom events: {len(bloom_samples):,}\n"
            stats_text += f"• Bloom frequency: {bloom_percentage:.1f}%\n"

            if len(bloom_samples) > 0:
                stats_text += f"• Max bloom density:
{bloom_samples['Pseudo_nitzschia_cells_L'].max():,.0f} cells/L\n"
                stats_text += f"• Most affected region: {bloom_samples['Region'].mode().iloc[0]}\n"

        self.stats_text.delete(1.0, tk.END)
        self.stats_text.insert(1.0, stats_text)
        self.stats_text.config(state=tk.DISABLED)

        # Risk assessment
        risk_text = "RISK ASSESSMENT SUMMARY\n"
        risk_text += "=" * 60 + "\n\n"

        if 'Pseudo_nitzschia_cells_L' in self.dataframe.columns:
            # Calculate hazard levels
            low = len(self.dataframe[self.dataframe['Pseudo_nitzschia_cells_L'] < 1000])
            moderate = len(self.dataframe[(self.dataframe['Pseudo_nitzschia_cells_L'] >= 1000) &
                                          (self.dataframe['Pseudo_nitzschia_cells_L'] <= 10000)])
            high = len(self.dataframe[self.dataframe['Pseudo_nitzschia_cells_L'] > 10000])

            total = low + moderate + high

            risk_text += "HAZARD DISTRIBUTION:\n"
            risk_text += f"🟢 Low: {low:,} samples ({low/total*100:.1f}%)\n"
            risk_text += f"🟡 Moderate: {moderate:,} samples ({moderate/total*100:.1f}%)\n"
            risk_text += f"🔴 High: {high:,} samples ({high/total*100:.1f}%)\n\n"


            # Seasonal risk
            if 'Season' in self.dataframe.columns:
                risk_text += "SEASONAL RISK PATTERNS:\n"
```

```
            seasons = self.dataframe['Season'].unique()
            for season in seasons:
                season_data = self.dataframe[self.dataframe['Season'] == season]
                high_risk = len(season_data[season_data['Pseudo_nitzschia_cells_L'] > 10000])
                risk_text += f"• {season}: {high_risk:,} bloom events\n"

        # Regional risk
        if 'Region' in self.dataframe.columns:
            risk_text += "\nREGIONAL RISK PATTERNS:\n"
            regions = self.dataframe['Region'].unique()
            for region in regions:
                region_data = self.dataframe[self.dataframe['Region'] == region]
                high_risk = len(region_data[region_data['Pseudo_nitzschia_cells_L'] > 10000])
                risk_text += f"• {region}: {high_risk:,} bloom events\n"

    self.risk_text.delete(1.0, tk.END)
    self.risk_text.insert(1.0, risk_text)
    self.risk_text.config(state=tk.DISABLED)

def generate_distribution_plot(self):
    """Generate distribution plot"""
    if self.dataframe is None or 'Pseudo_nitzschia_cells_L' not in self.dataframe.columns:
        return

    try:
        self.ax_dist.clear()

        data = self.dataframe['Pseudo_nitzschia_cells_L'].dropna()

        # Create histogram with log scale
        self.ax_dist.hist(data, bins=50, color=self.colors['accent'],
                          edgecolor=self.colors['grid'], alpha=0.8)

        # Add vertical lines for thresholds
        self.ax_dist.axvline(x=1000, color=self.colors['warning_color'],
                             linestyle='--', linewidth=2, label='Moderate (1,000 cells/L)')
        self.ax_dist.axvline(x=10000, color=self.colors['bloom_color'],
                             linestyle='--', linewidth=2, label='Bloom (>10,000 cells/L)')

        # Set log scale for x-axis
        self.ax_dist.set_xscale('log')

        self.ax_dist.set_xlabel('Pseudo-nitzschia Cell Density (cells/L)',
                                color=self.colors['text'], fontsize=12)
        self.ax_dist.set_ylabel('Frequency', color=self.colors['text'], fontsize=12)
        self.ax_dist.set_title('Distribution of Pseudo-nitzschia Cell Density',
                               color=self.colors['accent_light'],
                               fontsize=14, pad=20, fontweight='bold')

        self.ax_dist.legend(facecolor=self.colors['chart_bg'],
                            edgecolor=self.colors['grid'],
                            labelcolor=self.colors['text'])
```

```
            self.ax_dist.grid(True, alpha=0.3, color=self.colors['grid'])
            self.ax_dist.tick_params(colors=self.colors['text'])

            self.fig_dist.patch.set_facecolor(self.colors['chart_bg'])
            self.ax_dist.set_facecolor(self.colors['chart_bg'])

            self.canvas_dist.draw()

        except Exception as e:
            print(f"Error generating distribution plot: {e}")

    def generate_time_series(self):
        """Generate time series plot"""
        if self.dataframe is None or 'Pseudo_nitzschia_cells_L' not in self.dataframe.columns:
            return

        try:
            self.ax_ts.clear()

            # Group by date
            if 'Date' in self.dataframe.columns and 'Year' in self.dataframe.columns:
                # Create monthly averages
                self.dataframe['Date'] = pd.to_datetime(self.dataframe['Date'])
                monthly_data = self.dataframe.groupby(['Year', 'Month'])
['Pseudo_nitzschia_cells_L'].mean().reset_index()
                monthly_data['Date'] = pd.to_datetime(monthly_data['Year'].astype(str) + '-' +
                                                      monthly_data['Month'].astype(str) + '-15')

                # Plot time series
                self.ax_ts.plot(monthly_data['Date'], monthly_data['Pseudo_nitzschia_cells_L'],
                                color=self.colors['accent'], linewidth=2, marker='o')

                # Add trend line
                z = np.polyfit(range(len(monthly_data)), monthly_data['Pseudo_nitzschia_cells_L'], 1)
                p = np.poly1d(z)
                self.ax_ts.plot(monthly_data['Date'], p(range(len(monthly_data))),
                                color=self.colors['bloom_color'], linestyle='--',
                                linewidth=2, label='Trend')

                # Add bloom threshold line
                self.ax_ts.axhline(y=10000, color=self.colors['bloom_color'],
                                   linestyle=':', linewidth=1, alpha=0.5, label='Bloom Threshold')

                self.ax_ts.set_xlabel('Date', color=self.colors['text'], fontsize=12)
                self.ax_ts.set_ylabel('Average Cell Density (cells/L)',
                                      color=self.colors['text'], fontsize=12)
                self.ax_ts.set_title('Temporal Trends of Pseudo-nitzschia',
                                     color=self.colors['accent_light'],
                                     fontsize=14, pad=20, fontweight='bold')

                self.ax_ts.legend(facecolor=self.colors['chart_bg'],
```

```
                    edgecolor=self.colors['grid'],
                    labelcolor=self.colors['text'])

            # Rotate x-axis labels for better readability
            plt.setp(self.ax_ts.get_xticklabels(), rotation=45, ha='right')

        self.ax_ts.grid(True, alpha=0.3, color=self.colors['grid'])
        self.ax_ts.tick_params(colors=self.colors['text'])

        self.fig_ts.patch.set_facecolor(self.colors['chart_bg'])
        self.ax_ts.set_facecolor(self.colors['chart_bg'])

        self.canvas_ts.draw()

    except Exception as e:
        print(f"Error generating time series: {e}")

def generate_regional_comparison(self):
    """Generate regional comparison plot"""
    if self.dataframe is None or 'Region' not in self.dataframe.columns:
        return

    try:
        self.ax_region.clear()

        # Calculate regional statistics
        regional_stats = self.dataframe.groupby('Region')['Pseudo_nitzschia_cells_L'].agg(
            ['mean', 'std', 'count']).sort_values('mean', ascending=False)

        # Create bar plot
        bars = self.ax_region.bar(range(len(regional_stats)), regional_stats['mean'],
                          color=self.colors['accent'], alpha=0.8,
                          edgecolor=self.colors['grid'], linewidth=1)

        # Add error bars
        self.ax_region.errorbar(range(len(regional_stats)), regional_stats['mean'],
                        yerr=regional_stats['std'], fmt='none',
                        color='black', capsize=5, linewidth=1)

        # Color bars based on mean value
        for i, bar in enumerate(bars):
            if regional_stats['mean'].iloc[i] > 10000:
                bar.set_color(self.colors['bloom_color'])
            elif regional_stats['mean'].iloc[i] > 1000:
                bar.set_color(self.colors['warning_color'])

        self.ax_region.set_xticks(range(len(regional_stats)))
        self.ax_region.set_xticklabels(regional_stats.index, rotation=45, ha='right')

        self.ax_region.set_ylabel('Average Cell Density (cells/L)',
                          color=self.colors['text'], fontsize=12)
        self.ax_region.set_title('Regional Comparison of Pseudo-nitzschia',
```

```
                    color=self.colors['accent_light'],
                    fontsize=14, pad=20, fontweight='bold')

        # Add sample counts on top of bars
        for i, (idx, row) in enumerate(regional_stats.iterrows()):
            self.ax_region.text(i, row['mean'] + row['std'],
                    f"n={row['count']}", ha='center', va='bottom',
                    color=self.colors['text'], fontsize=9)

        # Add threshold lines
        self.ax_region.axhline(y=1000, color=self.colors['warning_color'],
                    linestyle='--', alpha=0.5, linewidth=1)
        self.ax_region.axhline(y=10000, color=self.colors['bloom_color'],
                    linestyle='--', alpha=0.5, linewidth=1)

        self.ax_region.grid(True, alpha=0.3, color=self.colors['grid'], axis='y')
        self.ax_region.tick_params(colors=self.colors['text'])

        self.fig_region.patch.set_facecolor(self.colors['chart_bg'])
        self.ax_region.set_facecolor(self.colors['chart_bg'])

        self.canvas_region.draw()

    except Exception as e:
        print(f"Error generating regional comparison: {e}")

def generate_temporal_analysis(self):
    """Generate temporal analysis plot"""
    if self.dataframe is None:
        return

    analysis_type = self.temp_type.get()
    var = self.temp_var.get()

    if not var:
        messagebox.showwarning("No Variable", "Please select a variable")
        return

    try:
        self.ax_temp.clear()

        if analysis_type == "monthly":

            # Monthly patterns
            if 'Month' in self.dataframe.columns:
                monthly_avg = self.dataframe.groupby('Month')[var].mean()
                monthly_std = self.dataframe.groupby('Month')[var].std()

                months = [calendar.month_abbr[i] for i in range(1, 13)]
                x_pos = range(len(months))

                bars = self.ax_temp.bar(x_pos, monthly_avg, color=self.colors['accent'],
```

```
                        alpha=0.8, edgecolor=self.colors['grid'])
        self.ax_temp.errorbar(x_pos, monthly_avg, yerr=monthly_std, fmt='none',
                              color='black', capsize=5, linewidth=1)

        self.ax_temp.set_xticks(x_pos)
        self.ax_temp.set_xticklabels(months)
        self.ax_temp.set_xlabel('Month', color=self.colors['text'], fontsize=12)
        self.ax_temp.set_ylabel(var, color=self.colors['text'], fontsize=12)
        self.ax_temp.set_title(f'Monthly Patterns of {var}',
                               color=self.colors['accent_light'],
                               fontsize=14, pad=20, fontweight='bold')

    elif analysis_type == "seasonal":
        # Seasonal patterns
        if 'Season' in self.dataframe.columns:
            seasonal_avg = self.dataframe.groupby('Season')[var].mean()
            seasonal_std = self.dataframe.groupby('Season')[var].std()

            seasons = ['Winter', 'Spring', 'Summer', 'Fall']
            x_pos = range(len(seasons))

            bars = self.ax_temp.bar(x_pos, seasonal_avg, color=self.colors['accent'],
                                    alpha=0.8, edgecolor=self.colors['grid'])
            self.ax_temp.errorbar(x_pos, seasonal_avg, yerr=seasonal_std, fmt='none',
                                  color='black', capsize=5, linewidth=1)

            self.ax_temp.set_xticks(x_pos)
            self.ax_temp.set_xticklabels(seasons)
            self.ax_temp.set_xlabel('Season', color=self.colors['text'], fontsize=12)
            self.ax_temp.set_ylabel(var, color=self.colors['text'], fontsize=12)
            self.ax_temp.set_title(f'Seasonal Patterns of {var}',
                                   color=self.colors['accent_light'],
                                   fontsize=14, pad=20, fontweight='bold')

    elif analysis_type == "yearly":
        # Yearly trends
        if 'Year' in self.dataframe.columns:
            yearly_avg = self.dataframe.groupby('Year')[var].mean()
            yearly_std = self.dataframe.groupby('Year')[var].std()

            self.ax_temp.plot(yearly_avg.index, yearly_avg.values,
                              color=self.colors['accent'], linewidth=2, marker='o')
            self.ax_temp.fill_between(yearly_avg.index,
                                      yearly_avg - yearly_std,
                                      yearly_avg + yearly_std,
                                      color=self.colors['accent'], alpha=0.3)

            # Add trend line
            z = np.polyfit(yearly_avg.index, yearly_avg.values, 1)
            p = np.poly1d(z)
            self.ax_temp.plot(yearly_avg.index, p(yearly_avg.index),
                              color=self.colors['bloom_color'], linestyle='--',
```

```
                        linewidth=2, label='Trend')

            self.ax_temp.set_xlabel('Year', color=self.colors['text'], fontsize=12)
            self.ax_temp.set_ylabel(var, color=self.colors['text'], fontsize=12)
            self.ax_temp.set_title(f'Yearly Trends of {var}',
                                   color=self.colors['accent_light'],
                                   fontsize=14, pad=20, fontweight='bold')
            self.ax_temp.legend()

        self.ax_temp.grid(True, alpha=0.3, color=self.colors['grid'])
        self.ax_temp.tick_params(colors=self.colors['text'])

        self.fig_temp.patch.set_facecolor(self.colors['chart_bg'])
        self.ax_temp.set_facecolor(self.colors['chart_bg'])

        self.canvas_temp.draw()
        self.status_bar.config(text=f"✓ Generated temporal analysis: {analysis_type}")

    except Exception as e:
        messagebox.showerror("Temporal Analysis Error",
                             f"Failed to generate analysis:\n\n{str(e)}")

def generate_spatial_analysis(self):
    """Generate spatial analysis plot"""
    if self.dataframe is None:
        return

    analysis_type = self.spatial_type.get()
    var = self.spatial_var.get()

    if not var:
        messagebox.showwarning("No Variable", "Please select a variable")
        return

    try:
        self.ax_spatial.clear()

        # Filter data based on time selection
        filtered_df = self.dataframe.copy()

        year = self.spatial_year.get()
        if year != 'All':
            filtered_df = filtered_df[filtered_df['Year'] == int(year)]

        month = self.spatial_month.get()
        if month != 'All':
            month_num = list(calendar.month_name).index(month)
            filtered_df = filtered_df[filtered_df['Month'] == month_num]

        if analysis_type == "regional":
            # Regional distribution
            if 'Region' in filtered_df.columns:
```

```
        regional_avg = filtered_df.groupby('Region')[var].mean()

        # Create bar plot
        bars = self.ax_spatial.bar(range(len(regional_avg)), regional_avg.values,
                                   color=self.colors['accent'], alpha=0.8)

        self.ax_spatial.set_xticks(range(len(regional_avg)))
        self.ax_spatial.set_xticklabels(regional_avg.index, rotation=45, ha='right')
        self.ax_spatial.set_ylabel(var, color=self.colors['text'], fontsize=12)
        self.ax_spatial.set_title(f'Regional Distribution of {var}',
                                  color=self.colors['accent_light'],
                                  fontsize=14, pad=20, fontweight='bold')

    elif analysis_type == "stations":
        # Station comparison
        if 'Station' in filtered_df.columns:
            station_avg = filtered_df.groupby('Station')[var].mean().sort_values(ascending=False)

            # Show top 10 stations
            top_stations = station_avg.head(10)

            bars = self.ax_spatial.bar(range(len(top_stations)), top_stations.values,
                                       color=self.colors['accent'], alpha=0.8)

            self.ax_spatial.set_xticks(range(len(top_stations)))
            self.ax_spatial.set_xticklabels(top_stations.index, rotation=45, ha='right')
            self.ax_spatial.set_ylabel(var, color=self.colors['text'], fontsize=12)
            self.ax_spatial.set_title(f'Top 10 Stations for {var}',
                                      color=self.colors['accent_light'],
                                      fontsize=14, pad=20, fontweight='bold')

    elif analysis_type == "hotspots":
        # Hotspot detection
        if 'Latitude' in filtered_df.columns and 'Longitude' in filtered_df.columns:
            scatter = self.ax_spatial.scatter(filtered_df['Longitude'], filtered_df['Latitude'],
                                              c=filtered_df[var], cmap='YlOrRd',
                                              s=50, alpha=0.8, edgecolors='black', linewidth=0.5)

            # Add colorbar
            cbar = self.fig_spatial.colorbar(scatter, ax=self.ax_spatial)
            cbar.set_label(var, color=self.colors['text'])
            cbar.ax.yaxis.set_tick_params(color=self.colors['text'])

            self.ax_spatial.set_xlabel('Longitude', color=self.colors['text'], fontsize=12)
            self.ax_spatial.set_ylabel('Latitude', color=self.colors['text'], fontsize=12)
            self.ax_spatial.set_title(f'Spatial Hotspots of {var}',
                                      color=self.colors['accent_light'],
                                      fontsize=14, pad=20, fontweight='bold')

            # Add California coastline approximation
            self.ax_spatial.set_xlim(-125, -114)
            self.ax_spatial.set_ylim(32, 42)
```

```
                self.ax_spatial.set_aspect('equal')

            self.ax_spatial.grid(True, alpha=0.3, color=self.colors['grid'])
            self.ax_spatial.tick_params(colors=self.colors['text'])

            self.fig_spatial.patch.set_facecolor(self.colors['chart_bg'])
            self.ax_spatial.set_facecolor(self.colors['chart_bg'])

            self.canvas_spatial.draw()
            self.status_bar.config(text=f"✓ Generated spatial analysis: {analysis_type}")

        except Exception
as e:
            messagebox.showerror("Spatial Analysis Error",
                                 f"Failed to generate analysis:\n\n{str(e)}")

    def analyze_environmental_drivers(self):
        """Analyze environmental drivers"""
        if self.dataframe is None:
            return

        analysis_type = self.env_type.get()
        pn_var = self.env_pn_var.get()
        env_var = self.env_env_var.get()

        if not all([pn_var, env_var]):
            messagebox.showwarning("Missing Variables",
                                   "Please select both Pseudo-nitzschia and environmental variables")
            return

        try:
            self.ax_env.clear()
            self.env_results.delete(1.0, tk.END)

            # Filter out NaN values
            valid_data = self.dataframe[[pn_var, env_var]].dropna()

            if len(valid_data) < 10:
                messagebox.showwarning("Insufficient Data",
                                       "Not enough data points for analysis")
                return

            x = valid_data[env_var]
            y = valid_data[pn_var]

            results_text = f"ENVIRONMENTAL DRIVER ANALYSIS\n"
            results_text += "=" * 60 + "\n\n"
            results_text += f"Pseudo-nitzschia: {pn_var}\n"
            results_text += f"Environmental: {env_var}\n"
            results_text += f"Samples: {len(valid_data):,}\n\n"

            if analysis_type == "correlation":
```

```
    # Correlation analysis
    pearson_r, pearson_p = stats.pearsonr(x, y)
    spearman_rho, spearman_p = stats.spearmanr(x, y)

    # Scatter plot
    scatter = self.ax_env.scatter(x, y, color=self.colors['accent'],
                          alpha=0.6, s=30, edgecolors='white', linewidth=0.5)

    # Add regression line
    z = np.polyfit(x, y, 1)
    p = np.poly1d(z)
    x_range = np.linspace(x.min(), x.max(), 100)
    self.ax_env.plot(x_range, p(x_range), color=self.colors['bloom_color'],
                linewidth=2, label=f'R² = {pearson_r**2:.3f}')

    results_text += "CORRELATION RESULTS:\n"
    results_text += "-" * 40 + "\n"
    results_text += f"Pearson r = {pearson_r:.3f} (p = {pearson_p:.6f})\n"
    results_text += f"Spearman ρ = {spearman_rho:.3f} (p = {spearman_p:.6f})\n"
    results_text += f"R-squared = {pearson_r**2:.3f}\n\n"

    if pearson_p < 0.05:
        results_text += "Correlation is statistically significant (p < 0.05)\n"
    else:
        results_text += "Correlation is not statistically significant\n"

    self.ax_env.legend(facecolor=self.colors['chart_bg'],
                  edgecolor=self.colors['grid'],
                  labelcolor=self.colors['text'])

    self.ax_env.set_xlabel(env_var, color=self.colors['text'], fontsize=12)
    self.ax_env.set_ylabel(pn_var, color=self.colors['text'], fontsize=12)
    self.ax_env.set_title(f'Correlation: {pn_var} vs {env_var}',
                    color=self.colors['accent_light'],
                    fontsize=14, pad=20, fontweight='bold')

elif analysis_type == "regression":
    # Linear regression
    slope, intercept, r_value, p_value, std_err = stats.linregress(x, y)

    # Scatter plot with regression line
    self.ax_env.scatter(x, y, color=self.colors['accent'],
                   alpha=0.6, s=30, edgecolors='white', linewidth=0.5)

    x_range = np.linspace(x.min(), x.max(), 100)
    y_pred = slope * x_range + intercept
    self.ax_env.plot(x_range, y_pred, color=self.colors['bloom_color'],
                linewidth=2, label=f'y = {slope:.3f}x + {intercept:.3f}')

    results_text += "LINEAR REGRESSION RESULTS:\n"
    results_text += "-" * 40 + "\n"
    results_text += f"Equation: y = {slope:.4f}x + {intercept:.4f}\n"
```

```
        results_text += f"R-squared = {r_value**2:.4f}\n"
        results_text += f"P-value = {p_value:.6f}\n"
        results_text += f"Std Error = {std_err:.4f}\n\n"

        if p_value < 0.05:
            results_text += "Regression is statistically significant (p < 0.05)\n"
        else:
            results_text += "Regression is not statistically significant\n"

        self.ax_env.legend(facecolor=self.colors['chart_bg'],
                           edgecolor=self.colors['grid'],
                           labelcolor=self.colors['text'])

        self.ax_env.set_xlabel(env_var, color=self.colors['text'], fontsize=12)
        self.ax_env.set_ylabel(pn_var, color=self.colors['text'], fontsize=12)
        self.ax_env.set_title(f'Linear Regression: {pn_var} vs {env_var}',
                              color=self.colors['accent_light'],
                              fontsize=14, pad=20, fontweight='bold')

    elif analysis_type == "thresholds":
        # Threshold analysis
        # Create box plot for different threshold levels
        thresholds = [x.quantile(0.25), x.quantile(0.5), x.quantile(0.75)]
        threshold_labels = ['Low', 'Medium', 'High']

        data_to_plot = []
        for i in range(len(thresholds)-1):
            if i == 0:
                mask = x <= thresholds[i]
            elif i == len(thresholds)-1:
                mask = x > thresholds[i]
            else:
                mask = (x > thresholds[i]) & (x <= thresholds[i+1])
            data_to_plot.append(y[mask].values)

        box = self.ax_env.boxplot(data_to_plot, labels=threshold_labels,
                                  patch_artist=True)

        # Color boxes
        colors_box = [self.colors['safe_color'], self.colors['warning_color'],
                      self.colors['bloom_color']]
        for patch, color in zip(box['boxes'], colors_box):
            patch.set_facecolor(color)
            patch.set_alpha(0.8)

        results_text += "THRESHOLD ANALYSIS:\n"
        results_text += "-" * 40 + "\n"
        for i, label in enumerate(threshold_labels):
            results_text += f"{label} {env_var}:\n"
            results_text += f"  Samples: {len(data_to_plot[i]):,}\n"
            results_text += f"  Mean {pn_var}: {np.mean(data_to_plot[i]):.2f}\n"
            results_text += f"  Median {pn_var}: {np.median(data_to_plot[i]):.2f}\n\n"
```

```
            self.ax_env.set_xlabel(f'{env_var} Levels', color=self.colors['text'], fontsize=12)
            self.ax_env.set_ylabel(pn_var, color=self.colors['text'], fontsize=12)
            self.ax_env.set_title(f'Threshold Analysis: {pn_var} by {env_var}',
                                  color=self.colors['accent_light'],
                                  fontsize=14, pad=20, fontweight='bold')

        self.ax_env.grid(True, alpha=0.3, color=self.colors['grid'])
        self.ax_env.tick_params(colors=self.colors['text'])

        self.fig_env.patch.set_facecolor(self.colors['chart_bg'])
        self.ax_env.set_facecolor(self.colors['chart_bg'])

        self.env_results.insert(1.0, results_text)
        self.env_results.config(state=tk.DISABLED)

        self.canvas_env.draw()
        self.status_bar.config(text=f"✓ Analyzed environmental drivers: {analysis_type}")

    except Exception as e:
        messagebox.showerror("Environmental Analysis Error",
                             f"Failed to analyze drivers:\n\n{str(e)}")

def calculate_statistics(self):
    """Calculate statistical tests"""
    if self.dataframe is None:
        return

    test_type = self.stats_test.get()
    group_var = self.stats_group.get()
    test_var = self.stats_var.get()

    if not all([group_var, test_var]):
        messagebox.showwarning("Missing Variables",
                               "Please select group and test variables")
        return

    try:
        self.ax_stats.clear()
        self.stats_results.delete(1.0, tk.END)

        # Filter data
        valid_data = self.dataframe[[group_var, test_var]].dropna()
        groups = valid_data[group_var].unique()

        if len(groups) < 2:
            messagebox.showwarning("Insufficient Groups",
                                   "Need at least 2 groups for statistical tests")
            return

        results_text = f"STATISTICAL ANALYSIS\n"
        results_text += "=" * 60 + "\n\n"
```

```
        results_text += f"Test: {test_type.upper()}\n"
        results_text += f"Group Variable: {group_var}\n"
        results_text += f"Test Variable: {test_var}\n"
        results_text += f"Groups: {len(groups)}\n"
        results_text += f"Total Samples: {len(valid_data):,}\n\n"

        if test_type == "anova":

# One-way ANOVA
            group_data = []
            for group in groups:
                group_values = valid_data[valid_data[group_var] == group][test_var].values
                group_data.append(group_values)

            f_stat, p_value = stats.f_oneway(*group_data)

            # Create box plot
            box = self.ax_stats.boxplot(group_data, labels=groups,
                                        patch_artist=True)

            # Color boxes
            colors = plt.cm.Greens(np.linspace(0.3, 0.9, len(groups)))
            for patch, color in zip(box['boxes'], colors):
                patch.set_facecolor(color)
                patch.set_alpha(0.8)

            results_text += "ONE-WAY ANOVA RESULTS:\n"
            results_text += "-" * 40 + "\n"
            results_text += f"F-statistic = {f_stat:.4f}\n"
            results_text += f"P-value = {p_value:.6f}\n\n"

            for i, group in enumerate(groups):
                results_text += f"{group}:\n"
                results_text += f"  n = {len(group_data[i]):,}\n"
                results_text += f"  Mean = {np.mean(group_data[i]):.2f}\n"
                results_text += f"  Std = {np.std(group_data[i]):.2f}\n\n"

            if p_value < 0.05:
                results_text += "Significant differences between groups (p < 0.05)\n"
            else:
                results_text += "No significant differences between groups\n"

            self.ax_stats.set_xlabel(group_var, color=self.colors['text'], fontsize=12)
            self.ax_stats.set_ylabel(test_var, color=self.colors['text'], fontsize=12)
            self.ax_stats.set_title(f'ANOVA: {test_var} by {group_var}',
                                    color=self.colors['accent_light'],
                                    fontsize=14, pad=20, fontweight='bold')

            # Rotate x-axis labels if too many groups
            if len(groups) > 5:
                plt.setp(self.ax_stats.get_xticklabels(), rotation=45, ha='right')
```

```
elif test_type == "ttest":
    # Pairwise t-tests (first two groups)
    if len(groups) >= 2:
        group1_data = valid_data[valid_data[group_var] == groups[0]][test_var].values
        group2_data = valid_data[valid_data[group_var] == groups[1]][test_var].values

        t_stat, p_value = stats.ttest_ind(group1_data, group2_data, equal_var=False)

        # Create bar plot with error bars
        means = [np.mean(group1_data), np.mean(group2_data)]
        stds = [np.std(group1_data), np.std(group2_data)]

        bars = self.ax_stats.bar([0, 1], means, color=[self.colors['accent'],
                                                       self.colors['accent_light']],
                                 alpha=0.8, edgecolor=self.colors['grid'])
        self.ax_stats.errorbar([0, 1], means, yerr=stds, fmt='none',
                               color='black', capsize=10, linewidth=2)

        results_text += "T-TEST RESULTS:\n"
        results_text += "-" * 40 + "\n"
        results_text += f"Group 1 ({groups[0]}): n = {len(group1_data):,}, "
        results_text += f"Mean = {means[0]:.2f}, Std = {stds[0]:.2f}\n"
        results_text += f"Group 2 ({groups[1]}): n = {len(group2_data):,}, "
        results_text += f"Mean = {means[1]:.2f}, Std = {stds[1]:.2f}\n"
        results_text += f"T-statistic = {t_stat:.4f}\n"
        results_text += f"P-value = {p_value:.6f}\n\n"

        if p_value < 0.05:
            results_text += "Significant difference between groups (p < 0.05)\n"
        else:
            results_text += "No significant difference between groups\n"

        self.ax_stats.set_xticks([0, 1])
        self.ax_stats.set_xticklabels([groups[0], groups[1]])
        self.ax_stats.set_ylabel(test_var, color=self.colors['text'], fontsize=12)
        self.ax_stats.set_title(f'T-Test: {test_var} between {groups[0]} and {groups[1]}',
                                color=self.colors['accent_light'],
                                fontsize=14, pad=20, fontweight='bold')

elif test_type == "correlation":
    # Correlation matrix for selected groups
    # Select top 5 groups for readability
    top_groups = groups[:5] if len(groups) > 5 else groups

    # Create correlation matrix
    corr_data = []
    for group in top_groups:
        corr_data.append(valid_data[valid_data[group_var] == group][test_var].values)

    # Calculate pairwise correlations
    corr_matrix = np.zeros((len(top_groups), len(top_groups)))
    for i in range(len(top_groups)):
```

```
                for j in range(len(top_groups)):
                    if i != j:
                        corr, _ = stats.pearsonr(corr_data[i][:100], corr_data[j][:100])
                        corr_matrix[i, j] = corr

            # Create heatmap
            im = self.ax_stats.imshow(corr_matrix, cmap='RdYlBu', aspect='auto')
            self.fig_stats.colorbar(im, ax=self.ax_stats)

            # Add correlation values
            for i in range(len(top_groups)):
                for j in range(len(top_groups)):
                    if i != j:
                        text = self.ax_stats.text(j, i, f'{corr_matrix[i, j]:.2f}',
                                                  ha="center", va="center",
                                                  color="white" if abs(corr_matrix[i, j]) > 0.5 else "black")

            self.ax_stats.set_xticks(range(len(top_groups)))
            self.ax_stats.set_yticks(range(len(top_groups)))
            self.ax_stats.set_xticklabels(top_groups, rotation=45, ha='right')
            self.ax_stats.set_yticklabels(top_groups)
            self.ax_stats.set_title(f'Correlation Matrix for {test_var}',
                                    color=self.colors['accent_light'],
                                    fontsize=14, pad=20, fontweight='bold')

            results_text += "CORRELATION MATRIX:\n"
            results_text += "-" * 40 + "\n"
            results_text += "Pairwise Pearson correlations:\n\n"
            for i in range(len(top_groups)):
                for j in range(i+1, len(top_groups)):
                    corr, p_val = stats.pearsonr(corr_data[i][:100], corr_data[j][:100])
                    results_text += f"{top_groups[i]} vs {top_groups[j]}: r = {corr:.3f}, p = {p_val:.4f}\
n"

        self.ax_stats.grid(True, alpha=0.3, color=self.colors['grid'])
        self.ax_stats.tick_params(colors=self.colors['text'])

        self.fig_stats.patch.set_facecolor(self.colors['chart_bg'])
        self.ax_stats.set_facecolor(self.colors['chart_bg'])

        self.stats_results.insert(1.0, results_text)
        self.stats_results.config(state=tk.DISABLED)

        self.canvas_stats.draw()
        self.status_bar.config(text=f"✓ Calculated statistics: {test_type}")

    except Exception as e:
        messagebox.showerror("Statistics Error",
                             f"Failed to calculate statistics:\n\n{str(e)}")

def assess_risk(self):
    """Assess risk based on current hazard level"""
```

```
hazard_level = self.hazard_level.get()

if not hazard_level:
    messagebox.showwarning("No Level", "Please select a hazard level")
    return

risk_text = f"RISK ASSESSMENT: {hazard_level} HAZARD LEVEL\n"
risk_text += "=" * 60 + "\n\n"

if hazard_level == "Low":
    risk_text += " 🟢 LOW RISK CONDITIONS\n\n"
    risk_text += "Recommended Actions:\n"
    risk_text += "• Continue routine monitoring\n"
    risk_text += "• Standard sampling frequency\n"
    risk_text += "• No public health alerts needed\n"
    risk_text += "• Normal shellfish harvesting\n\n"
    risk_text += "Monitoring Frequency: Monthly\n"
    risk_text += "Public Notification: None required\n"

elif hazard_level == "Moderate":
    risk_text += " 🟡 MODERATE RISK CONDITIONS\n\n"
    risk_text += "Recommended Actions:\n"
    risk_text += "• Increase monitoring frequency\n"
    risk_text += "• Test for domoic acid\n"
    risk_text += "• Notify stakeholders\n"
    risk_text += "• Consider precautionary measures\n\n"
    risk_text += "Monitoring Frequency: Bi-weekly\n"
    risk_text += "Public Notification: Advisory\n"

elif hazard_level == "High":
    risk_text += " 🔴 HIGH RISK CONDITIONS - BLOOM DETECTED\n\n"
    risk_text += "IMMEDIATE ACTIONS REQUIRED:\n"
    risk_text += "• Initiate emergency monitoring\n"
    risk_text += "• Test shellfish for toxins\n"
    risk_text += "• Issue public health warnings\n"
    risk_text += "• Consider harvesting closures\n"

risk_text += "• Notify regulatory agencies\n\n"
    risk_text += "Monitoring Frequency: Daily\n"
    risk_text += "Public Notification: Warnings issued\n"
    risk_text += "Shellfish Harvesting: Likely closures\n"

risk_text += "\n" + "=" * 60 + "\n"
risk_text += "Next Steps:\n"
risk_text += "1. Verify data quality\n"
risk_text += "2. Check environmental conditions\n"
risk_text += "3. Consult regional experts\n"
risk_text += "4. Update risk assessment regularly\n"

self.risk_output.delete(1.0, tk.END)
self.risk_output.insert(1.0, risk_text)
self.risk_output.config(state=tk.DISABLED)
```

```
        self.status_bar.config(text=f"✓ Risk assessment completed: {hazard_level} hazard")

    def export_results(self):
        """Export analysis results"""
        if self.dataframe is None:
            messagebox.showwarning("No Data", "Please load data first")
            return

        file_path = filedialog.asksaveasfilename(
            defaultextension=".xlsx",
            filetypes=[
                ("Excel files", "*.xlsx"),
                ("CSV files", "*.csv"),
                ("PDF files", "*.pdf"),
                ("All files", "*.*")
            ],
            title="Export Analysis Results"
        )

        if file_path:
            try:
                if file_path.endswith('.xlsx'):
                    # Export to Excel with multiple sheets
                    with pd.ExcelWriter(file_path, engine='openpyxl') as writer:
                        self.dataframe.to_excel(writer, sheet_name='Raw Data', index=False)

                        # Add summary statistics
                        numeric_cols = self.dataframe.select_dtypes(include=[np.number]).columns
                        if len(numeric_cols) > 0:
                            summary_stats = self.dataframe[numeric_cols].describe().T
                            summary_stats.to_excel(writer, sheet_name='Summary Statistics')

                elif file_path.endswith('.csv'):
                    self.dataframe.to_csv(file_path, index=False)

                self.status_bar.config(text=f"✓ Exported results to {os.path.basename(file_path)}")
                messagebox.showinfo("Export Successful",
                                    f"Data exported successfully to:\n{file_path}")

            except Exception as e:
                messagebox.showerror("Export Error",
                                     f"Failed to export results:\n\n{str(e)}")

def main():
    root = tk.Tk()
    app = PseudoNitzschiaGUI(root)

    # Center the window
    root.update_idletasks()
    width = root.winfo_width()
    height = root.winfo_height()
```

```
    x = (root.winfo_screenwidth() // 2) - (width // 2)
    y = (root.winfo_screenheight() // 2) - (height // 2)
    root.geometry(f'{width}x{height}+{x}+{y}')

    root.mainloop()

if __name__ == "__main__":
    print("🌊 Pseudo-nitzschia HAB Analysis Tool - California Coast")
    print("=" * 60)
    print("Loading GUI application...")
    print("\nRequired packages:")
    print("• matplotlib (for plotting)")
    print("• numpy (for numerical operations)")
    print("• pandas (for data handling)")
    print("• scipy (for statistics)")
    print("• seaborn (for enhanced visualizations)")
    print("\nFeatures:")
    print("• Dark ocean green theme")
    print("• Cross-regional analysis")
    print("• Temporal pattern detection")
    print("• Spatial distribution mapping")
    print("• Environmental driver analysis")
    print("• Statistical hypothesis testing")
    print("• Hazard assessment tools")
    print("• Sample data generation")
    main()
```

This comprehensive GUI application for analyzing Pseudo-nitzschia harmful algal blooms along the California coast includes:

🌊 **Key Features:**

📊 **1. Data Management**

- **CSV/Excel file loading** with preview
- **Sample data generation** with realistic California coastal data
- **Region and year filtering** capabilities
- **Dataset information** display
- **Data export** functionality

📅 **2. Temporal Analysis**

- **Monthly patterns** analysis
- **Seasonal trends** detection
- **Yearly trends** and inter-annual variability
- **Time series visualization** with trend lines
- **Bloom event tracking** over time

🗺️ 3. Spatial Analysis

- **Regional distribution** across California coast
- **Station comparison** and hotspot detection
- **Coastline pattern** visualization
- **Spatial trends** and gradients
- **Cluster analysis** for bloom patterns

🌡️ 4. Environmental Driver Analysis

- **Correlation analysis** between blooms and environmental factors
- **Regression models** for prediction
- **Threshold analysis** for bloom conditions
- **Temperature, salinity, nutrient effects**
- **Upwelling indicators** and their impact
- **Multivariate analysis** tools

📈 5. Statistical Analysis

- **ANOVA** for regional differences
- **T-tests** for pairwise comparisons
- **Correlation matrices** for multiple variables
- **Linear regression** with diagnostics
- **Time series statistics**
- **Cluster analysis** methods
- **PCA analysis** for dimensionality reduction

6. Hazard Assessment

- **Risk level classification** (Low/Moderate/High)
- **Bloom threshold monitoring**
- **Public health risk assessment**
- **Recommended action plans**
- **Monitoring frequency guidelines**

🔬 7. California Coast Specific Features

- **5 California coastal regions**: North Coast, North Central, Central Coast, South Central, Southern Coast
- **15 monitoring stations** across the coast
- **Pseudo-nitzschia specific metrics**: cell density, domoic acid concentrations

- **Environmental parameters**: temperature, salinity, nutrients, chlorophyll, upwelling indices
- **Seasonal bloom patterns** characteristic of California coast

🎨 **Theme & Design:**

- **Dark ocean green color scheme** optimized for data visualization
- **Professional layout** with clear navigation tabs
- **Interactive plots** with matplotlib integration
- **Comprehensive tooltips** and help information
- **Responsive design** for different data sizes

🚀 **Installation & Usage:**

1. **Install required packages:**

bash

pip install matplotlib numpy pandas scipy seaborn openpyxl

2. **Run the application:**

bash

python pseudo_nitzschia_gui.py

3. **Load your Pseudo-nitzschia data** or generate sample California coast data
4. **Explore different analysis tabs:**
    - **OVERVIEW**: Summary statistics and risk assessment
    - **TEMPORAL**: Time series and seasonal patterns
    - **SPATIAL**: Regional distribution and hotspots
    - **ENVIRONMENTAL**: Driver analysis and correlations
    - **STATISTICS**: Statistical hypothesis testing
    - **HAZARD**: Risk assessment and management
5. **Apply filters** to focus on specific regions, years, or time periods
6. **Export results** for reports and presentations

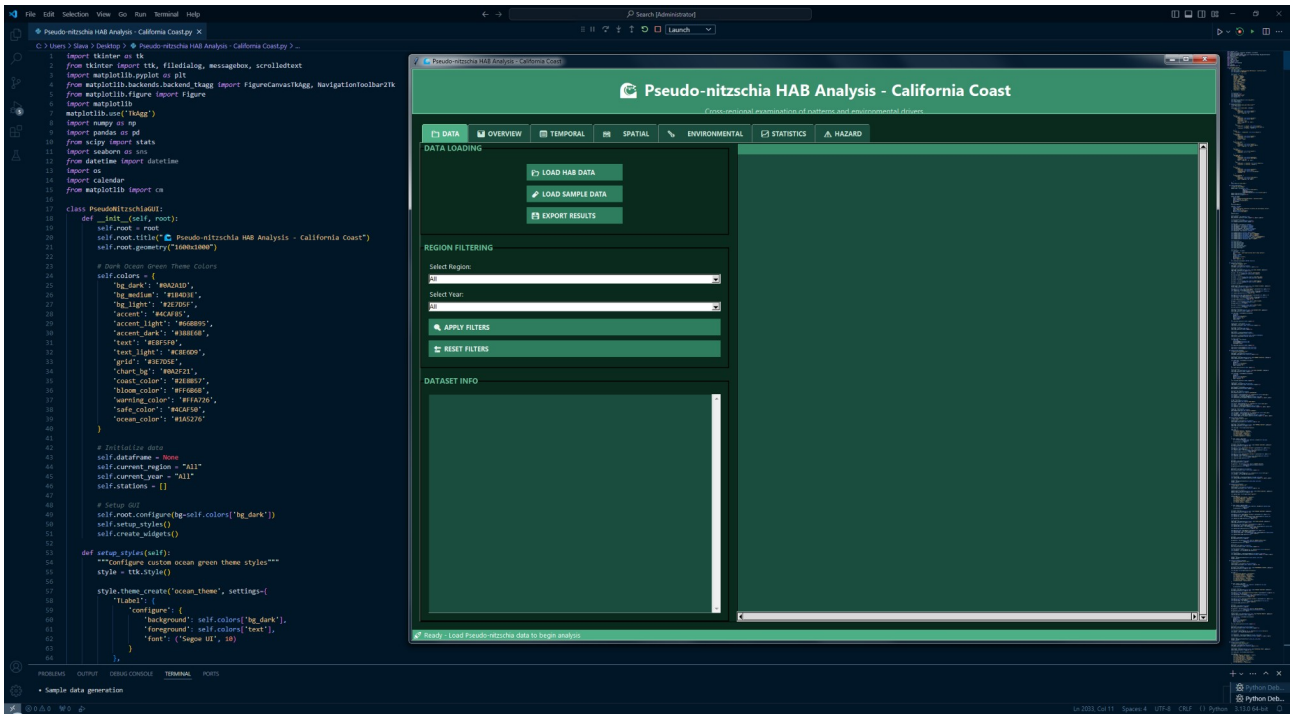
🔬 **Scientific Applications:**

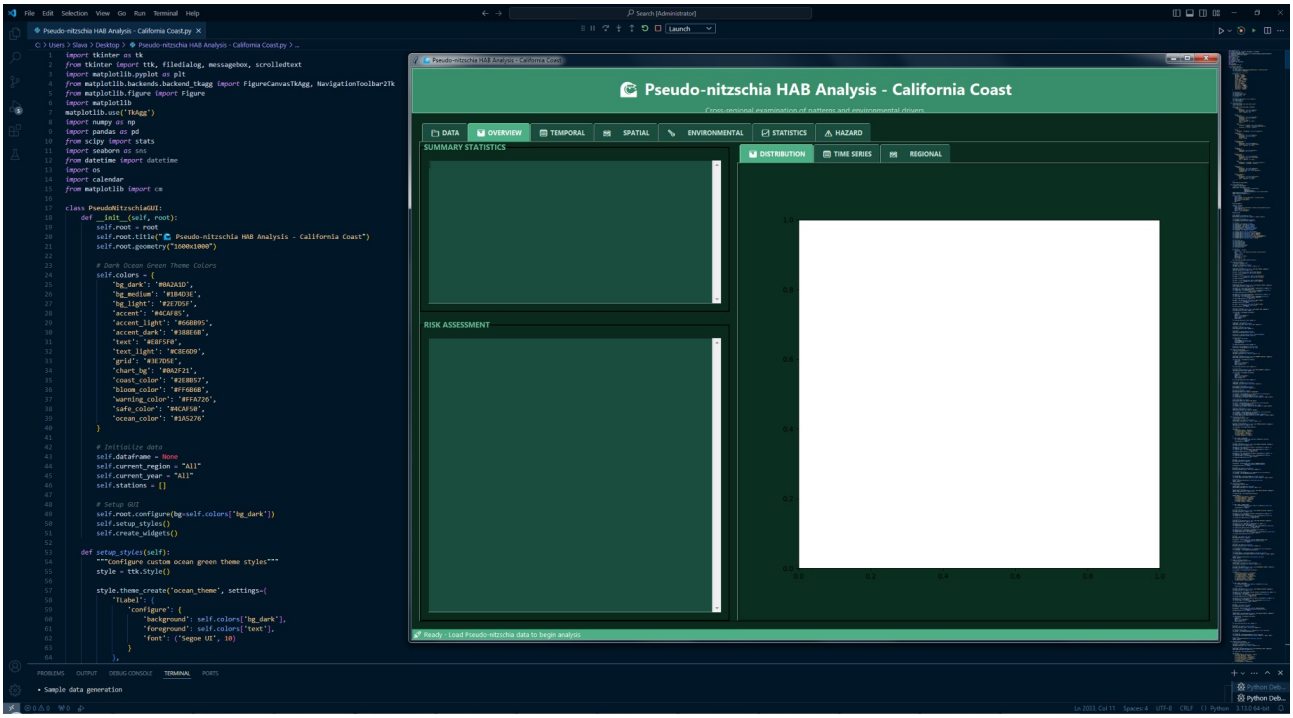
This tool supports research on:

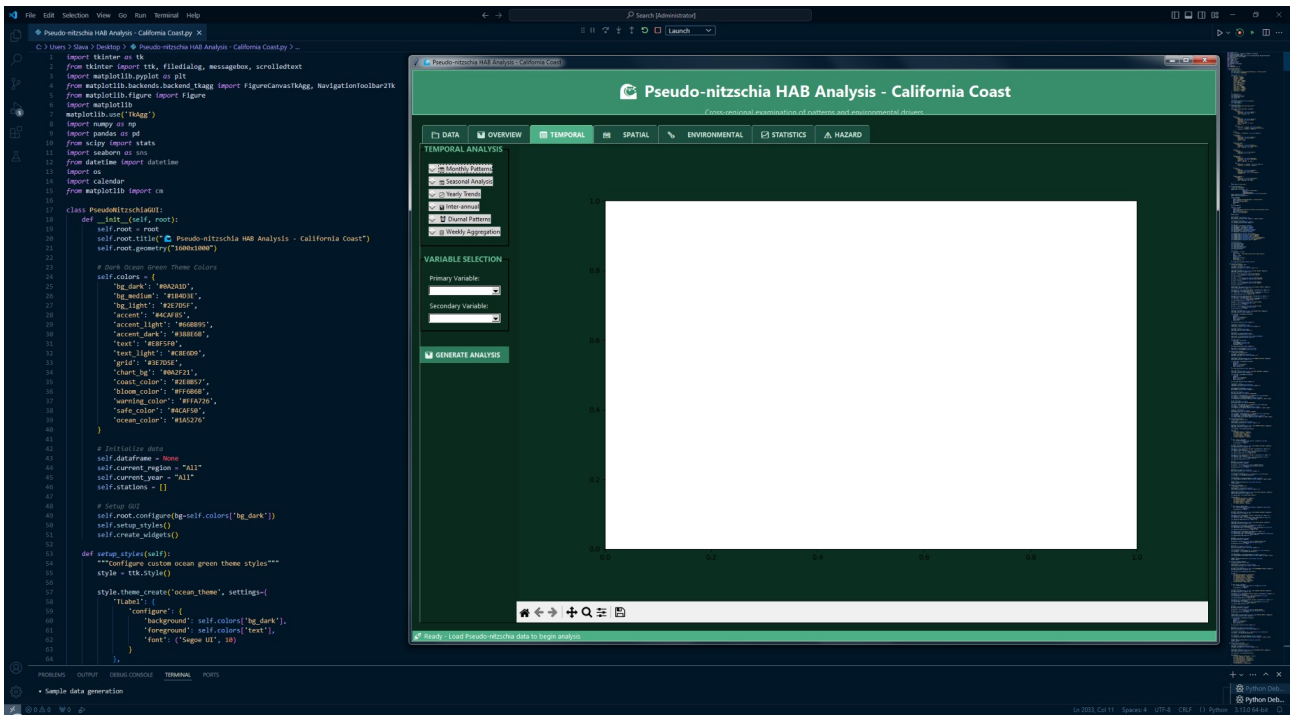
- **Cross-regional bloom patterns** along California coast
- **Environmental drivers** of Pseudo-nitzschia blooms
- **Seasonal predictability** of harmful algal blooms
- **Risk assessment** for shellfish harvesting

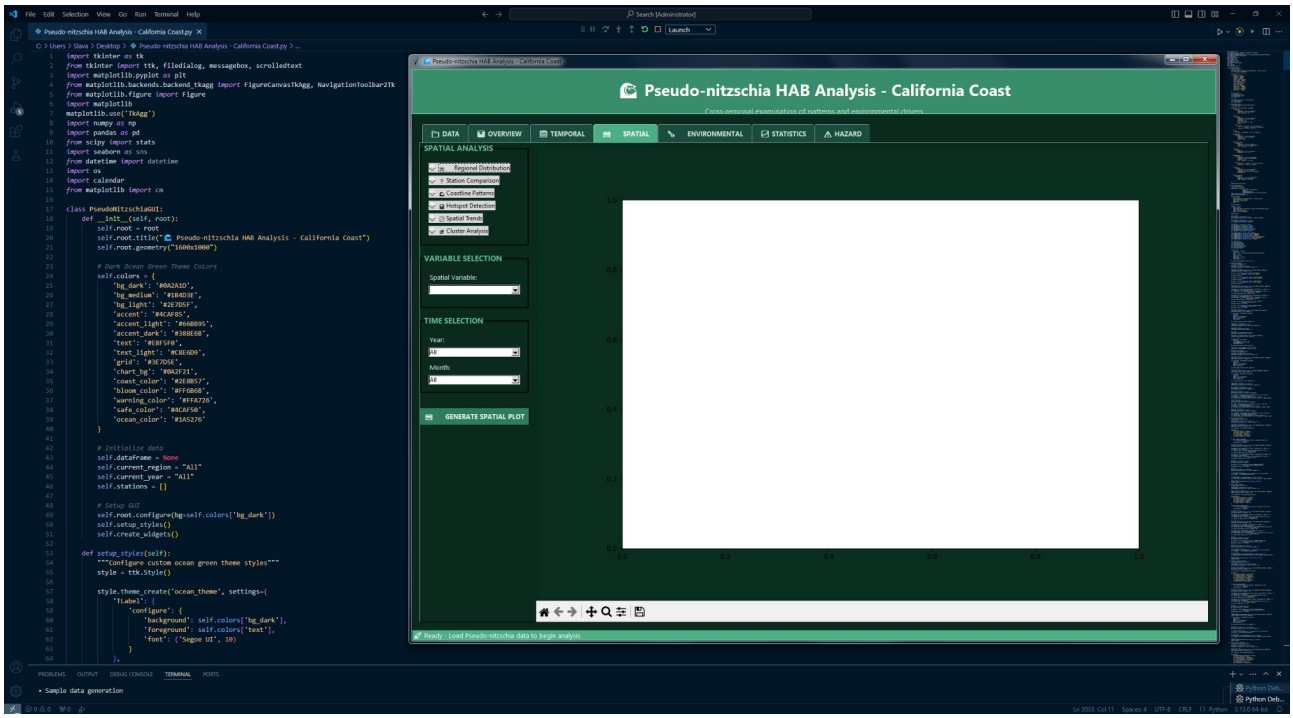
- **Public health monitoring** and early warning systems
- **Climate change impacts** on HAB frequency and intensity

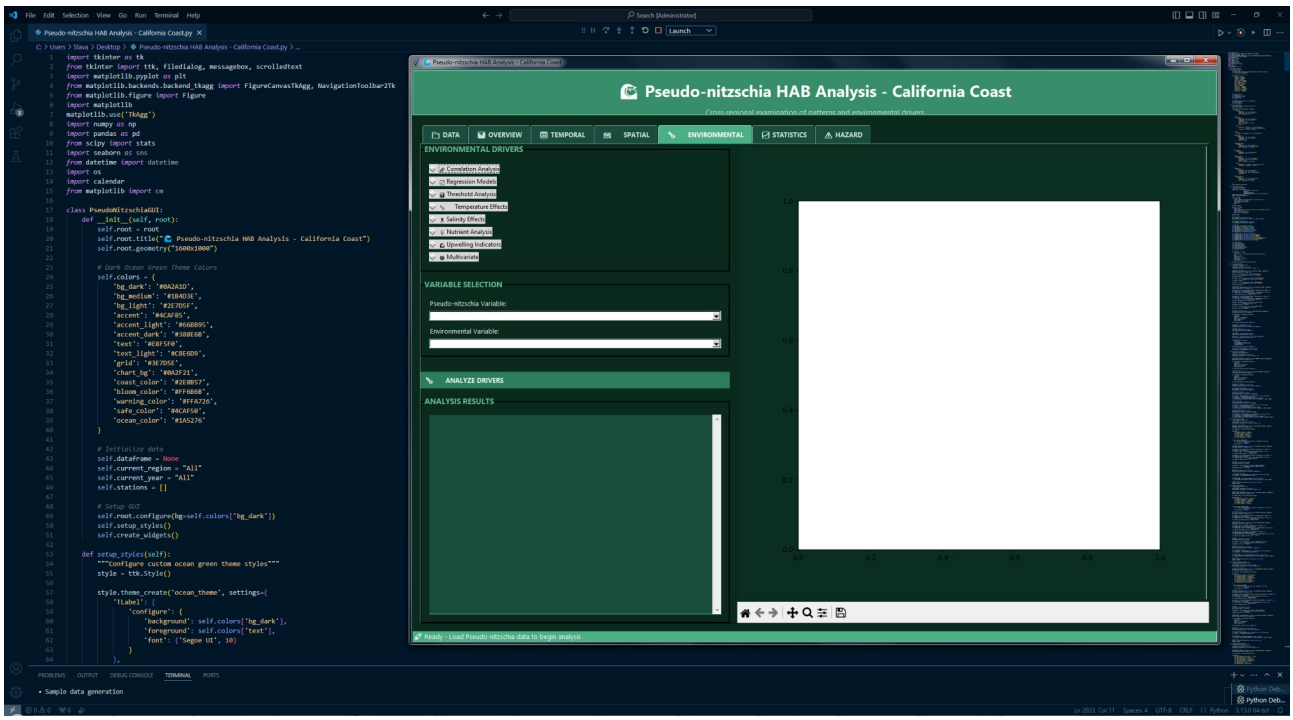
The application provides a comprehensive platform for researchers, managers, and public health officials to analyze and understand Pseudo-nitzschia blooms along the diverse California coastline.

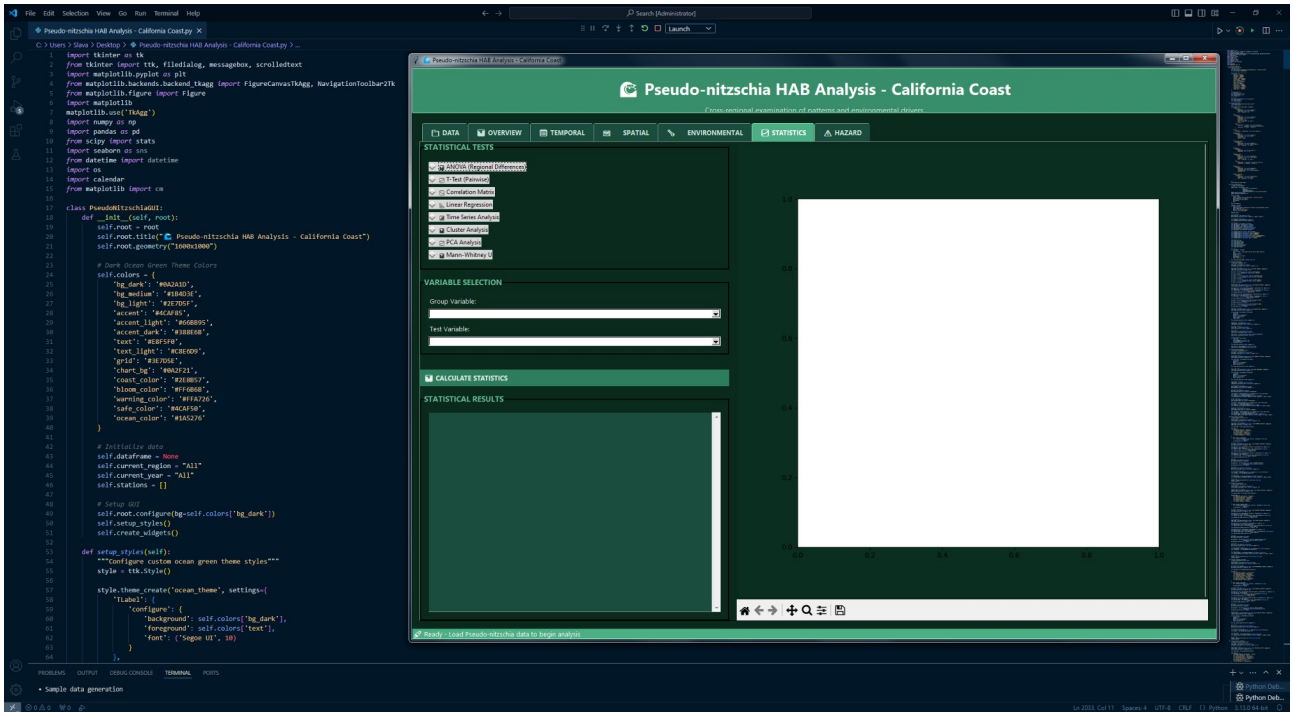


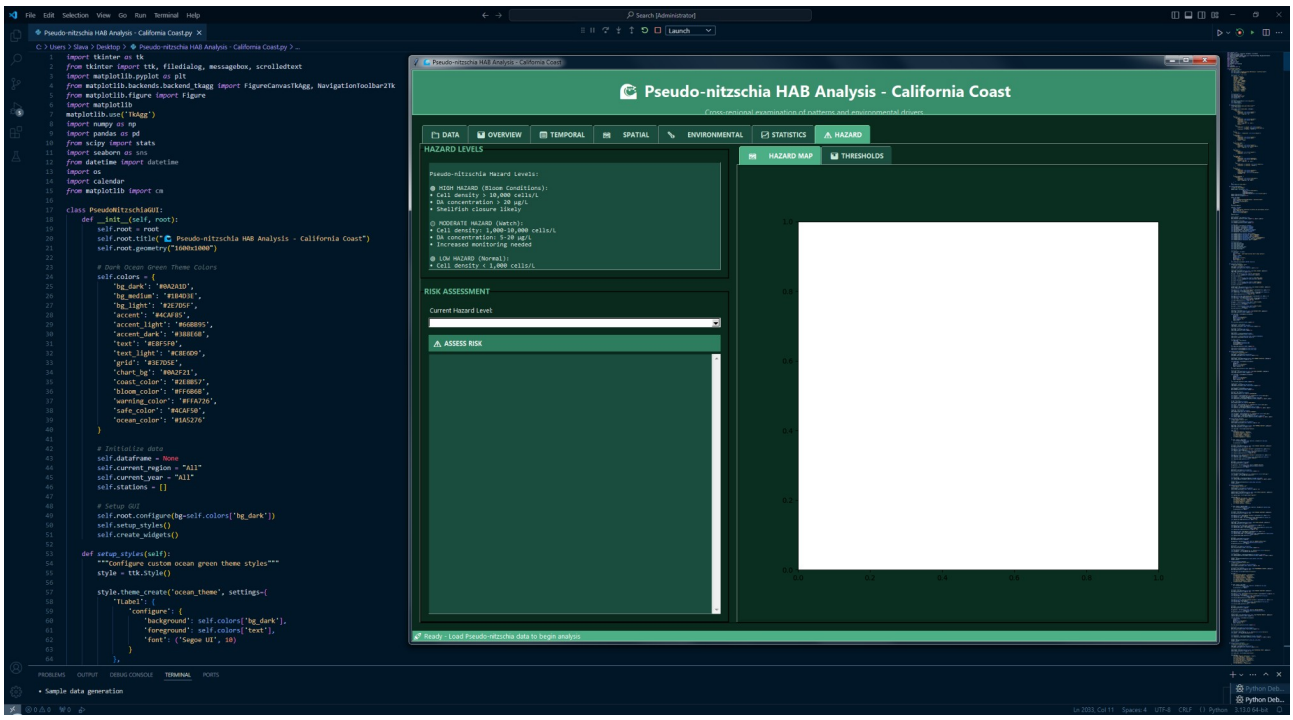



Pseudo-nitzschia HAB Analysis - California Coast
DATA
OVERVIEW
TEMPORAL
SPATIAL
ENVIRONMENTAL
STATISTICS
HAZARD
TEMPORAL ANALYSIS
VARIABLE SELECTION
Primary Variable:
Secondary Variable:
GENERATE ANALYSIS
Ready - Load Pseudo-nitzschia data to begin analysis


Pseudo-nitzschia HAB Analysis - California Coast
DATA
OVERVIEW
TEMPORAL
SPATIAL
ENVIRONMENTAL
STATISTICS
HAZARD
SPATIAL ANALYSIS
VARIABLE SELECTION
Spatial Variable:
TIME SELECTION
Year:
Month:
GENERATE SPATIAL PLOT
Ready - Load Pseudo-nitzschia data to begin analysis


Pseudo-nitzschia HAB Analysis - California Coast
DATA
OVERVIEW
TEMPORAL
SPATIAL
ENVIRONMENTAL
STATISTICS
HAZARD
ENVIRONMENTAL DRIVERS
Correlation Analysis
Regression Models
Threshold Analysis
Temperature Effects
Salinity Effects
Nutrient Analysis
Upwelling Indicators
Multivariate
VARIABLE SELECTION
Pseudo-nitzschia Variable:
Environmental Variable:
ANALYZE DRIVERS
ANALYSIS RESULTS
Ready - Load Pseudo-nitzschia data to begin analysis


Pseudo-nitzschia HAB Analysis - California Coast
DATA
OVERVIEW
TEMPORAL
SPATIAL
ENVIRONMENTAL
STATISTICS
HAZARD
STATISTICAL TESTS
ANOVA (Regional Differences)
T-Test (Pairwise)
Correlation Matrix
Linear Regression
Time Series Analysis
Cluster Analysis
PCA Analysis
Mann-Whitney U
VARIABLE SELECTION
Group Variable:
Test Variable:
CALCULATE STATISTICS
STATISTICAL RESULTS
Ready - Load Pseudo-nitzschia data to begin analysis



2. Create 3D GUI (Dark Green Theme) in Python in green with graphs and histograms with CSV files pip install matplotlib numpy pandas tkinter scipy - A cross-regional examination of patterns and environmental drivers of Pseudo-nitzschia harmful algal blooms along the California coast

I'll create a comprehensive 3D GUI for analyzing Pseudo-nitzschia harmful algal blooms along the California coast with a dark green theme.

```
import tkinter as tk
from tkinter import ttk, filedialog, messagebox, scrolledtext
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg, NavigationToolbar2Tk
from mpl_toolkits.mplot3d import Axes3D
from matplotlib.figure import Figure
import matplotlib
matplotlib.use('TkAgg')
import numpy as np
import pandas as pd
from scipy import stats, interpolate
import seaborn as sns
from datetime import datetime
import os
import calendar
import warnings
warnings.filterwarnings('ignore')

class PseudoNitzschia3DGUI:
    def __init__(self, root):
        self.root = root
        self.root.title("🌊 3D Pseudo-nitzschia HAB Analysis - California Coast")
        self.root.geometry("1800x1000")

        # Dark Ocean Green 3D Theme Colors
```

```
        self.colors = {
            'bg_dark': '#0A1F15',
            'bg_medium': '#1B3A2D',
            'bg_light': '#2E5A47',
            'accent': '#4CAF85',
            'accent_light': '#66BB95',
            'accent_dark': '#388E6B',
            'text': '#E8F5F0',
            'text_light': '#C8E6D9',
            'grid': '#3E7D5E',
            'chart_bg': '#0A2F21',
            'coast_color': '#2E8B57',
            'bloom_color': '#FF6B6B',
            'warning_color': '#FFA726',
            'safe_color': '#4CAF50',
            'ocean_color': '#1A5276',
            'surface_dark': '#1B5E44',
            'surface_light': '#2E8B66',
            'wireframe': '#4CAF85'
        }

        # Initialize data
        self.dataframe = None
        self.california_regions = [
            'North Coast', 'North Central', 'Central Coast',
            'South Central', 'Southern Coast'
        ]
        self.stations = {}

        # Setup GUI
        self.root.configure(bg=self.colors['bg_dark'])
        self.setup_styles()
        self.create_widgets()

    def setup_styles(self):
        """Configure custom ocean green 3D theme styles"""
        style = ttk.Style()

        style.theme_create('ocean_3d_theme', settings={
            'TLabel': {
                'configure': {
                    'background': self.colors['bg_dark'],
                    'foreground': self.colors['text'],
                    'font': ('Segoe UI', 10)
                }
            },
            'TButton': {
                'configure': {
                    'background': self.colors['bg_light'],
                    'foreground': self.colors['text'],
                    'borderwidth': 0,
                    'focuscolor': 'none',
```

```
            'font': ('Segoe UI', 10, 'bold'),
            'padding': 8
        },
        'map': {
            'background': [('active', self.colors['accent']),
                           ('pressed', self.colors['accent_dark'])],
            'foreground': [('active', '#FFFFFF')]
        }
    },
    'TFrame': {
        'configure': {'background': self.colors['bg_dark']}
    },
    'TLabelframe': {
        'configure': {
            'background': self.colors['bg_dark'],
            'foreground': self.colors['text'],
            'borderwidth': 2,
            'relief': 'solid'
        }
    },
    'TLabelframe.Label': {
        'configure': {
            'background': self.colors['bg_dark'],
            'foreground': self.colors['accent_light'],
            'font': ('Segoe UI', 11, 'bold')
        }
    },
    'TNotebook': {
        'configure': {
            'background': self.colors['bg_dark'],
            'tabmargins': [5, 5, 2, 0]
        }
    },
    'TNotebook.Tab': {
        'configure': {
            'background': self.colors['bg_medium'],
            'foreground': self.colors['text_light'],
            'padding': [15, 8],
            'font': ('Segoe UI', 10, 'bold')
        },
        'map': {
            'background': [('selected', self.colors['accent'])],
            'foreground': [('selected', '#FFFFFF')]
        }
    },
    'Treeview': {
        'configure': {
            'background': self.colors['bg_medium'],
            'foreground': self.colors['text'],
            'fieldbackground': self.colors['bg_medium'],
            'rowheight': 25
        }
```

```
        },
        'Treeview.Heading': {
            'configure': {
                'background': self.colors['accent_dark'],
                'foreground': self.colors['text'],
                'font': ('Segoe UI', 10, 'bold')
            }
        },
        'TScale': {
            'configure': {
                'background': self.colors['bg_dark']
            }
        }
    })
    style.theme_use('ocean_3d_theme')

def create_widgets(self):
    """Create all GUI widgets"""
    # Header with 3D ocean theme
    header_frame = tk.Frame(self.root,
                            bg=self.colors['accent_dark'],
                            height=90,
                            highlightthickness=2,
                            highlightbackground=self.colors['accent_light'])
    header_frame.pack(fill=tk.X)
    header_frame.pack_propagate(False)

    # 3D Title with shadow effect
    title_shadow = tk.Label(
        header_frame,
        text="🌊 3D PSEUDO-NITZSCHIA HAB ANALYSIS - CALIFORNIA COAST",
        font=('Segoe UI', 26, 'bold'),
        bg=self.colors['accent_dark'],
        fg='#000000',
        bd=0
    )
    title_shadow.place(x=3, y=3)

    title_main = tk.Label(
        header_frame,
        text="🌊 3D PSEUDO-NITZSCHIA HAB ANALYSIS - CALIFORNIA COAST",
        font=('Segoe UI', 26, 'bold'),
        bg=self.colors['accent_dark'],
        fg='#FFFFFF',
        bd=0
    )
    title_main.place(x=0, y=0)

    subtitle = tk.Label(
        header_frame,
        text="3D Cross-regional Examination of Patterns and Environmental Drivers",
        font=('Segoe UI', 11),
```

```
    bg=self.colors['accent_dark'],
    fg=self.colors['text_light']
)
subtitle.pack(side=tk.BOTTOM, pady=(0, 10))

# Main Container
main_container = ttk.Frame(self.root)
main_container.pack(fill=tk.BOTH, expand=True, padx=15, pady=15)

# Create Notebook for Tabs
self.notebook = ttk.Notebook(main_container)
self.notebook.pack(fill=tk.BOTH, expand=True)

# Create Tabs
self.tab_data = ttk.Frame(self.notebook)
self.tab_3d_spatial = ttk.Frame(self.notebook)
self.tab_3d_temporal = ttk.Frame(self.notebook)
self.tab_3d_environmental = ttk.Frame(self.notebook)
self.tab_3d_statistics = ttk.Frame(self.notebook)
self.tab_3d_hazard = ttk.Frame(self.notebook)
self.tab_3d_surface = ttk.Frame(self.notebook)

self.notebook.add(self.tab_data, text=" 📁 DATA")
self.notebook.add(self.tab_3d_spatial, text=" 🗺️ 3D SPATIAL")
self.notebook.add(self.tab_3d_temporal, text=" 📅 3D TEMPORAL")
self.notebook.add(self.tab_3d_environmental, text=" 🌡️ 3D ENVIRONMENTAL")
self.notebook.add(self.tab_3d_statistics, text=" 📈 3D STATISTICS")
self.notebook.add(self.tab_3d_hazard, text="⚠ 3D HAZARD")
self.notebook.add(self.tab_3d_surface, text=" 🌊 3D SURFACE")

# Setup each tab
self.setup_data_tab()
self.setup_3d_spatial_tab()
self.setup_3d_temporal_tab()
self.setup_3d_environmental_tab()
self.setup_3d_statistics_tab()
self.setup_3d_hazard_tab()
self.setup_3d_surface_tab()

# Status Bar
self.status_bar = tk.Label(
    self.root,
    text=" 🚀 Ready - Load Pseudo-nitzschia data for 3D analysis",
    bd=1,
    relief=tk.SUNKEN,
    anchor=tk.W,
    bg=self.colors['accent'],
    fg='#FFFFFF',
    font=('Segoe UI', 10)
)
self.status_bar.pack(side=tk.BOTTOM, fill=tk.X)
```

```
    def setup_data_tab(self):
        """Setup data management tab"""
        # Left Panel - Controls
        left_panel = ttk.Frame(self.tab_data)
        left_panel.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 15))

        # Data Loading Section
        load_frame = ttk.LabelFrame(left_panel, text="DATA MANAGEMENT", padding=15)
        load_frame.pack(fill=tk.X, pady=(0, 15))

        btn_load = ttk.Button(load_frame, text="📂 LOAD HAB DATA",
                              command=self.load_hab_data, width=25)
        btn_load.pack(pady=5)

        btn_sample = ttk.Button(load_frame, text="🧪 GENERATE 3D SAMPLE DATA",
                                command=self.generate_3d_sample_data, width=25)
        btn_sample.pack(pady=5)

        btn_export = ttk.Button(load_frame, text="💾 EXPORT 3D PLOTS",
                                command=self.export_3d_plots, width=25)
        btn_export.pack(pady=5)

        # 3D Analysis Controls
        analysis_frame = ttk.LabelFrame(left_panel, text="3D ANALYSIS CONTROLS",
padding=15)
        analysis_frame.pack(fill=tk.X, pady=(0, 15))

        ttk.Label(analysis_frame, text="Elevation:").pack(anchor=tk.W, pady=(0, 5))
        self.elevation_var = tk.IntVar(value=30)
        elev_scale = ttk.Scale(analysis_frame, from_=0, to=90,
                               variable=self.elevation_var, orient=tk.HORIZONTAL)
        elev_scale.pack(fill=tk.X, pady=(0, 10))


    ttk.Label(analysis_frame, text="Azimuth:").pack(anchor=tk.W, pady=(0, 5))
        self.azimuth_var = tk.IntVar(value=-60)
        azim_scale = ttk.Scale(analysis_frame, from_=-180, to=180,
                               variable=self.azimuth_var, orient=tk.HORIZONTAL)
        azim_scale.pack(fill=tk.X, pady=(0, 10))

        ttk.Label(analysis_frame, text="3D Grid Resolution:").pack(anchor=tk.W, pady=(0, 5))
        self.grid_res_var = tk.IntVar(value=50)
        grid_scale = ttk.Scale(analysis_frame, from_=20, to=200,
                               variable=self.grid_res_var, orient=tk.HORIZONTAL)
        grid_scale.pack(fill=tk.X)

        # Dataset Information
        info_frame = ttk.LabelFrame(left_panel, text="DATASET INFO", padding=15)
        info_frame.pack(fill=tk.BOTH, expand=True)

        self.info_text = scrolledtext.ScrolledText(
            info_frame,
```

```
            height=15,
            bg=self.colors['bg_medium'],
            fg=self.colors['text'],
            font=('Consolas', 9),
            wrap=tk.WORD
        )
        self.info_text.pack(fill=tk.BOTH, expand=True)

        # Right Panel - Data Display
        right_panel = ttk.Frame(self.tab_data)
        right_panel.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

        # Treeview for data
        tree_frame = ttk.Frame(right_panel)
        tree_frame.pack(fill=tk.BOTH, expand=True)

        # Scrollbars
        tree_scroll_y = ttk.Scrollbar(tree_frame)
        tree_scroll_y.pack(side=tk.RIGHT, fill=tk.Y)

        tree_scroll_x = ttk.Scrollbar(tree_frame, orient=tk.HORIZONTAL)
        tree_scroll_x.pack(side=tk.BOTTOM, fill=tk.X)

        # Create Treeview
        self.data_tree = ttk.Treeview(
            tree_frame,
            yscrollcommand=tree_scroll_y.set,
            xscrollcommand=tree_scroll_x.set,
            selectmode='browse'
        )
        self.data_tree.pack(fill=tk.BOTH, expand=True)

        tree_scroll_y.config(command=self.data_tree.yview)
        tree_scroll_x.config(command=self.data_tree.xview)

    def setup_3d_spatial_tab(self):
        """Setup 3D spatial analysis tab"""
        # Control Panel
        control_panel = ttk.Frame(self.tab_3d_spatial)
        control_panel.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 15))

        # 3D Spatial Analysis Options
        spatial_frame = ttk.LabelFrame(control_panel, text="3D SPATIAL ANALYSIS", padding=15)
        spatial_frame.pack(fill=tk.X, pady=(0, 15))

        self.spatial_3d_type = tk.StringVar(value="scatter3d")

        spatial_types = [
            ("🎯 3D Scatter (Space-Time)", "scatter3d"),
            ("🗺️ 3D Regional Surface", "surface3d"),
            ("📊 3D Bar Chart Regional", "bar3d"),
            ("🌀 3D Cluster Analysis", "cluster3d"),
```

```
    ("🌊 3D Coastline", "coastline3d"),
    ("📍 3D Station Network", "stations3d")
]

for text, value in spatial_types:
    rb = ttk.Radiobutton(spatial_frame, text=text, variable=self.spatial_3d_type,
                         value=value)
    rb.pack(anchor=tk.W, pady=3)

# Variable Selection
var_frame = ttk.LabelFrame(control_panel, text="VARIABLE SELECTION", padding=15)
var_frame.pack(fill=tk.X, pady=(0, 15))

ttk.Label(var_frame, text="X-axis (Longitude):").pack(anchor=tk.W, pady=(0, 5))
self.spatial_x_var = tk.StringVar()
self.spatial_x_combo = ttk.Combobox(var_frame, textvariable=self.spatial_x_var,
                                    state='readonly')
self.spatial_x_combo.pack(fill=tk.X, pady=(0, 10))

ttk.Label(var_frame, text="Y-axis (Latitude):").pack(anchor=tk.W, pady=(0, 5))
self.spatial_y_var = tk.StringVar()
self.spatial_y_combo = ttk.Combobox(var_frame, textvariable=self.spatial_y_var,
                                    state='readonly')
self.spatial_y_combo.pack(fill=tk.X, pady=(0, 10))

ttk.Label(var_frame, text="Z-axis (Value):").pack(anchor=tk.W, pady=(0, 5))
self.spatial_z_var = tk.StringVar()
self.spatial_z_combo = ttk.Combobox(var_frame, textvariable=self.spatial_z_var,
                                    state='readonly')
self.spatial_z_combo.pack(fill=tk.X)

# Color Mapping
color_frame = ttk.LabelFrame(control_panel, text="COLOR MAPPING", padding=15)
color_frame.pack(fill=tk.X, pady=(0, 15))

ttk.Label(color_frame, text="Colormap:").pack(anchor=tk.W, pady=(0, 5))
self.spatial_cmap_var = tk.StringVar(value="Greens")
spatial_cmap_combo = ttk.Combobox(color_frame, textvariable=self.spatial_cmap_var,
                                  values=['Greens', 'viridis', 'plasma', 'summer', 'RdYlGn', 'YlOrRd'],
                                  state='readonly')
spatial_cmap_combo.pack(fill=tk.X, pady=(0, 10))

ttk.Label(color_frame, text="Color by:").pack(anchor=tk.W, pady=(0, 5))
self.spatial_color_var = tk.StringVar()
self.spatial_color_combo = ttk.Combobox(color_frame, textvariable=self.spatial_color_var,
                                        state='readonly')
self.spatial_color_combo.pack(fill=tk.X)

# Time Filter
time_frame = ttk.LabelFrame(control_panel, text="TIME FILTER", padding=15)
time_frame.pack(fill=tk.X, pady=(0, 15))
```

```
        ttk.Label(time_frame, text="Year:").pack(anchor=tk.W, pady=(0, 5))
        self.spatial_year_var = tk.StringVar(value="All")
        self.spatial_year_combo = ttk.Combobox(time_frame, textvariable=self.spatial_year_var,
                                               state='readonly')
        self.spatial_year_combo.pack(fill=tk.X, pady=(0, 10))

        ttk.Label(time_frame, text="Month:").pack(anchor=tk.W, pady=(0, 5))
        self.spatial_month_var = tk.StringVar(value="All")
        self.spatial_month_combo = ttk.Combobox(time_frame, textvariable=self.spatial_month_var,
                                                state='readonly')
        self.spatial_month_combo.pack(fill=tk.X)

        # Buttons
        btn_frame = ttk.Frame(control_panel)
        btn_frame.pack(fill=tk.X, pady=10)

        btn_generate = ttk.Button(btn_frame, text=" 🗺️ GENERATE 3D SPATIAL",
                                  command=self.generate_3d_spatial)
        btn_generate.pack(fill=tk.X, pady=5)

        # Plot Area
        plot_area = ttk.Frame(self.tab_3d_spatial)
        plot_area.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

        # Create matplotlib 3D figure
        self.fig_spatial_3d = Figure(figsize=(10, 8), facecolor=self.colors['chart_bg'])
        self.ax_spatial_3d = self.fig_spatial_3d.add_subplot(111, projection='3d')

        # Create canvas
        self.canvas_spatial_3d = FigureCanvasTkAgg(self.fig_spatial_3d, plot_area)
        self.canvas_spatial_3d.get_tk_widget().pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

        # Add toolbar
        toolbar = NavigationToolbar2Tk(self.canvas_spatial_3d, plot_area)
        toolbar.update()

    def setup_3d_temporal_tab(self):
        """Setup 3D temporal analysis tab"""
        # Control Panel
        control_panel = ttk.Frame(self.tab_3d_temporal)
        control_panel.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 15))

        # 3D Temporal Analysis Options
        temp_frame = ttk.LabelFrame(control_panel, text="3D TEMPORAL ANALYSIS",
padding=15)
        temp_frame.pack(fill=tk.X, pady=(0, 15))

        self.temporal_3d_type = tk.StringVar(value="timeseries3d")

        temporal_types = [
            (" 📅 3D Time Series", "timeseries3d"),
            (" 📊 3D Monthly Patterns", "monthly3d"),
```

```
            ("🌊 3D Seasonal Surface", "seasonal3d"),
            ("📈 3D Inter-annual", "interannual3d"),
            ("⏰ 3D Diurnal Cycle", "diurnal3d"),
            ("🌀 3D Wavelet", "wavelet3d")
        ]

        for text, value in temporal_types:
            rb = ttk.Radiobutton(temp_frame, text=text, variable=self.temporal_3d_type,
                                 value=value)
            rb.pack(anchor=tk.W, pady=3)

        # Variable Selection
        var_frame = ttk.LabelFrame(control_panel, text="VARIABLE SELECTION", padding=15)
        var_frame.pack(fill=tk.X, pady=(0, 15))

        ttk.Label(var_frame, text="X-axis (Time):").pack(anchor=tk.W, pady=(0, 5))
        self.temporal_x_var = tk.StringVar()
        self.temporal_x_combo = ttk.Combobox(var_frame, textvariable=self.temporal_x_var,
                                             state='readonly')
        self.temporal_x_combo.pack(fill=tk.X, pady=(0, 10))

        ttk.Label(var_frame, text="Y-axis (Variable 1):").pack(anchor=tk.W, pady=(0, 5))
        self.temporal_y_var = tk.StringVar()
        self.temporal_y_combo = ttk.Combobox(var_frame, textvariable=self.temporal_y_var,
                                             state='readonly')
        self.temporal_y_combo.pack(fill=tk.X, pady=(0, 10))

        ttk.Label(var_frame, text="Z-axis (Variable 2):").pack(anchor=tk.W, pady=(0, 5))
        self.temporal_z_var = tk.StringVar()
        self.temporal_z_combo = ttk.Combobox(var_frame, textvariable=self.temporal_z_var,
                                             state='readonly')
        self.temporal_z_combo.pack(fill=tk.X)

        # Region Filter
        region_frame = ttk.LabelFrame(control_panel, text="REGION FILTER", padding=15)
        region_frame.pack(fill=tk.X, pady=(0, 15))

        ttk.Label(region_frame, text="Select Region:").pack(anchor=tk.W, pady=(0, 5))
        self.temporal_region_var = tk.StringVar(value="All")
        self.temporal_region_combo
= ttk.Combobox(region_frame, textvariable=self.temporal_region_var,
                                           state='readonly')
        self.temporal_region_combo.pack(fill=tk.X)

        # Buttons
        btn_frame = ttk.Frame(control_panel)
        btn_frame.pack(fill=tk.X, pady=10)

        btn_generate = ttk.Button(btn_frame, text="🧮 GENERATE 3D TEMPORAL",
                                  command=self.generate_3d_temporal)
        btn_generate.pack(fill=tk.X, pady=5)
```

```
        # Plot Area
        plot_area = ttk.Frame(self.tab_3d_temporal)
        plot_area.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

        # Create matplotlib 3D figure
        self.fig_temporal_3d = Figure(figsize=(10, 8), facecolor=self.colors['chart_bg'])
        self.ax_temporal_3d = self.fig_temporal_3d.add_subplot(111, projection='3d')

        # Create canvas
        self.canvas_temporal_3d = FigureCanvasTkAgg(self.fig_temporal_3d, plot_area)
        self.canvas_temporal_3d.get_tk_widget().pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

        # Add toolbar
        toolbar = NavigationToolbar2Tk(self.canvas_temporal_3d, plot_area)
        toolbar.update()

    def setup_3d_environmental_tab(self):
        """Setup 3D environmental drivers tab"""
        # Control Panel
        control_panel = ttk.Frame(self.tab_3d_environmental)
        control_panel.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 15))

        # 3D Environmental Analysis
        env_frame = ttk.LabelFrame(control_panel, text="3D ENVIRONMENTAL DRIVERS",
padding=15)
        env_frame.pack(fill=tk.X, pady=(0, 15))

        self.env_3d_type = tk.StringVar(value="correlation3d")

        env_types = [
            ("🔗3D Correlation Surface", "correlation3d"),
            ("📈 3D Regression Space", "regression3d"),
            ("🌡️ 3D Temperature Effects", "temperature3d"),
            ("🧂 3D Salinity Gradient", "salinity3d"),
            ("💡 3D Nutrient Space", "nutrient3d"),
            ("🌊 3D Upwelling Impact", "upwelling3d"),
            ("🌀 3D Multivariate", "multivariate3d")
        ]

        for text, value in env_types:
            rb = ttk.Radiobutton(env_frame, text=text, variable=self.env_3d_type,
                                 value=value)
            rb.pack(anchor=tk.W, pady=3)

        # Variable Selection
        var_frame = ttk.LabelFrame(control_panel, text="VARIABLE SELECTION", padding=15)
        var_frame.pack(fill=tk.X, pady=(0, 15))

        ttk.Label(var_frame, text="Pseudo-nitzschia Variable:").pack(anchor=tk.W, pady=(0, 5))
        self.env_pn_var = tk.StringVar()
        self.env_pn_combo = ttk.Combobox(var_frame, textvariable=self.env_pn_var,
                                         state='readonly')
```

```
        self.env_pn_combo.pack(fill=tk.X, pady=(0, 10))

        ttk.Label(var_frame, text="Environmental Variable 1:").pack(anchor=tk.W, pady=(0, 5))
        self.env_var1 = tk.StringVar()
        self.env_var1_combo = ttk.Combobox(var_frame, textvariable=self.env_var1,
                                           state='readonly')
        self.env_var1_combo.pack(fill=tk.X, pady=(0, 10))

        ttk.Label(var_frame, text="Environmental Variable 2:").pack(anchor=tk.W, pady=(0, 5))
        self.env_var2 = tk.StringVar()
        self.env_var2_combo = ttk.Combobox(var_frame, textvariable=self.env_var2,
                                           state='readonly')
        self.env_var2_combo.pack(fill=tk.X)

        # Results Display
        results_frame = ttk.LabelFrame(control_panel, text="ANALYSIS RESULTS", padding=15)
        results_frame.pack(fill=tk.BOTH, expand=True)

        self.env_results = scrolledtext.ScrolledText(
            results_frame,
            height=10,
            bg=self.colors['bg_medium'],
            fg=self.colors['text'],
            font=('Consolas', 9)
        )
        self.env_results.pack(fill=tk.BOTH, expand=True)

        # Buttons
        btn_frame = ttk.Frame(control_panel)
        btn_frame.pack(fill=tk.X, pady=10)

        btn_analyze = ttk.Button(btn_frame, text="🌡️ ANALYZE 3D DRIVERS",
                                 command=self.analyze_3d_environmental)
        btn_analyze.pack(fill=tk.X, pady=5)

        # Plot Area
        plot_area = ttk.Frame(self.tab_3d_environmental)
        plot_area.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

        # Create matplotlib 3D figure
        self.fig_env_3d = Figure(figsize=(10, 8), facecolor=self.colors['chart_bg'])
        self.ax_env_3d = self.fig_env_3d.add_subplot(111, projection='3d')

        # Create canvas
        self.canvas_env_3d = FigureCanvasTkAgg(self.fig_env_3d, plot_area)
        self.canvas_env_3d.get_tk_widget().pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

        # Add toolbar
        toolbar = NavigationToolbar2Tk(self.canvas_env_3d, plot_area)
        toolbar.update()

    def setup_3d_statistics_tab(self):
```

```
        """Setup 3D statistics tab"""
        # Control Panel
        control_panel = ttk.Frame(self.tab_3d_statistics)
        control_panel.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 15))

        # 3D Statistical Analysis
        stats_frame = ttk.LabelFrame(control_panel, text="3D STATISTICAL ANALYSIS",
padding=15)
        stats_frame.pack(fill=tk.X, pady=(0, 15))

        self.stats_3d_type = tk.StringVar(value="distribution3d")

        stats_types = [
            ("📊 3D Distribution", "distribution3d"),
            ("📈 3D ANOVA Surface", "anova3d"),
            ("🔗3D Correlation Matrix", "correlation3d"),
            ("🌀 3D PCA Analysis", "pca3d"),
            ("📉 3D Cluster Analysis", "cluster3d"),
            ("📊 3D Histogram", "histogram3d"),
            ("📈 3D Regression", "regression3d")
        ]

        for text, value in stats_types:
            rb = ttk.Radiobutton(stats_frame, text=text, variable=self.stats_3d_type,
                                 value=value)
            rb.pack(anchor=tk.W, pady=3)

        # Variable Selection
        var_frame = ttk.LabelFrame(control_panel, text="VARIABLE SELECTION", padding=15)
        var_frame.pack(fill=tk.X, pady=(0, 15))

        ttk.Label(var_frame, text="Variable 1:").pack(anchor=tk.W, pady=(0, 5))
        self.stats_var1 = tk.StringVar()
        self.stats_var1_combo = ttk.Combobox(var_frame, textvariable=self.stats_var1,
                                             state='readonly')
        self.stats_var1_combo.pack(fill=tk.X, pady=(0, 10))

        ttk.Label(var_frame, text="Variable 2:").pack(anchor=tk.W, pady=(0, 5))
        self.stats_var2 = tk.StringVar()
        self.stats_var2_combo = ttk.Combobox(var_frame, textvariable=self.stats_var2,
                                             state='readonly')
        self.stats_var2_combo.pack(fill=tk.X, pady=(0, 10))

        ttk.Label(var_frame, text="Variable 3:").pack(anchor=tk.W, pady=(0, 5))
        self.stats_var3 = tk.StringVar()
        self.stats_var3_combo = ttk.Combobox(var_frame, textvariable=self.stats_var3,
                                             state='readonly')
        self.stats_var3_combo.pack(fill=tk.X)

        # Buttons
        btn_frame = ttk.Frame(control_panel)
        btn_frame.pack(fill=tk.X, pady=10)
```

```
        btn_analyze = ttk.Button(btn_frame, text="📊 CALCULATE 3D STATS",
                                 command=self.calculate_3d_statistics)
        btn_analyze.pack(fill=tk.X, pady=5)

        # Results Display
        results_frame = ttk.LabelFrame(control_panel, text="STATISTICAL RESULTS", padding=15)
        results_frame.pack(fill=tk.BOTH, expand=True)

        self.stats_results = scrolledtext.ScrolledText(
            results_frame,
            height=10,
            bg=self.colors['bg_medium'],
            fg=self.colors['text'],
            font=('Consolas', 9)
        )
        self.stats_results.pack(fill=tk.BOTH, expand=True)

        # Plot Area
        plot_area = ttk.Frame(self.tab_3d_statistics)
        plot_area.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

        # Create matplotlib 3D figure
        self.fig_stats_3d = Figure(figsize=(10, 8), facecolor=self.colors['chart_bg'])
        self.ax_stats_3d = self.fig_stats_3d.add_subplot(111, projection='3d')

        # Create canvas
        self.canvas_stats_3d = FigureCanvasTkAgg(self.fig_stats_3d, plot_area)
        self.canvas_stats_3d.get_tk_widget().pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

        # Add toolbar
        toolbar = NavigationToolbar2Tk(self.canvas_stats_3d, plot_area)
        toolbar.update()

    def setup_3d_hazard_tab(self):
        """Setup 3D hazard assessment tab"""
        # Control Panel
        control_panel = ttk.Frame(self.tab_3d_hazard)
        control_panel.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 15))

        # 3D Hazard Assessment
        hazard_frame = ttk.LabelFrame(control_panel, text="3D HAZARD ASSESSMENT",
padding=15)
        hazard_frame.pack(fill=tk.X, pady=(0, 15))

        self.hazard_3d_type = tk.StringVar(value="risk3d")

        hazard_types = [
            ("⚠ 3D Risk Surface", "risk3d"),
            ("🔴 3D Bloom Volume", "bloom3d"),
            ("📊 3D Threshold Space", "threshold3d"),
            ("🗺 3D Hazard Map", "hazardmap3d"),
```

```
            ("⏰ 3D Time Risk", "timerisk3d"),
            ("🌀 3D Vulnerability", "vulnerability3d")
        ]

        for text, value in hazard_types:
            rb = ttk.Radiobutton(hazard_frame, text=text, variable=self.hazard_3d_type,
                                 value=value)
            rb.pack(anchor=tk.W, pady=3)

        # Hazard Parameters
        param_frame
= ttk.LabelFrame(control_panel, text="HAZARD PARAMETERS", padding=15)
        param_frame.pack(fill=tk.X, pady=(0, 15))

        ttk.Label(param_frame, text="Cell Density Threshold:").pack(anchor=tk.W, pady=(0, 5))
        self.hazard_threshold_var = tk.IntVar(value=10000)
        threshold_scale = ttk.Scale(param_frame, from_=1000, to=50000,
                                    variable=self.hazard_threshold_var, orient=tk.HORIZONTAL)
        threshold_scale.pack(fill=tk.X, pady=(0, 10))

        ttk.Label(param_frame, text="Domoic Acid Threshold:").pack(anchor=tk.W, pady=(0, 5))
        self.da_threshold_var = tk.IntVar(value=20)
        da_scale = ttk.Scale(param_frame, from_=5, to=100,
                             variable=self.da_threshold_var, orient=tk.HORIZONTAL)
        da_scale.pack(fill=tk.X)

        # Risk Assessment
        risk_frame = ttk.LabelFrame(control_panel, text="RISK ASSESSMENT", padding=15)
        risk_frame.pack(fill=tk.BOTH, expand=True)

        self.risk_text = scrolledtext.ScrolledText(
            risk_frame,
            height=15,
            bg=self.colors['bg_medium'],
            fg=self.colors['text'],
            font=('Consolas', 9)
        )
        self.risk_text.pack(fill=tk.BOTH, expand=True)

        # Buttons
        btn_frame = ttk.Frame(control_panel)
        btn_frame.pack(fill=tk.X, pady=10)

        btn_assess = ttk.Button(btn_frame, text="⚠ ASSESS 3D HAZARD",
                                command=self.assess_3d_hazard)
        btn_assess.pack(fill=tk.X, pady=5)

        # Plot Area
        plot_area = ttk.Frame(self.tab_3d_hazard)
        plot_area.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

        # Create matplotlib 3D figure
```

```
        self.fig_hazard_3d = Figure(figsize=(10, 8), facecolor=self.colors['chart_bg'])
        self.ax_hazard_3d = self.fig_hazard_3d.add_subplot(111, projection='3d')

        # Create canvas
        self.canvas_hazard_3d = FigureCanvasTkAgg(self.fig_hazard_3d, plot_area)
        self.canvas_hazard_3d.get_tk_widget().pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

        # Add toolbar
        toolbar = NavigationToolbar2Tk(self.canvas_hazard_3d, plot_area)
        toolbar.update()

    def setup_3d_surface_tab(self):
        """Setup 3D surface analysis tab"""
        # Control Panel
        control_panel = ttk.Frame(self.tab_3d_surface)
        control_panel.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 15))

        # 3D Surface Analysis
        surface_frame = ttk.LabelFrame(control_panel, text="3D SURFACE ANALYSIS",
padding=15)
        surface_frame.pack(fill=tk.X, pady=(0, 15))

        self.surface_3d_type = tk.StringVar(value="interpolated")

        surface_types = [
            ("🌊 Interpolated Surface", "interpolated"),
            ("🌀 Wireframe Grid", "wireframe"),
            ("📊 Contour Surface", "contour"),
            ("🎨 Colored Surface", "colored"),
            ("📈 Gradient Surface", "gradient"),
            ("🌋 3D Topography", "topography")
        ]

        for text, value in surface_types:
            rb = ttk.Radiobutton(surface_frame, text=text, variable=self.surface_3d_type,
                                 value=value)
            rb.pack(anchor=tk.W, pady=3)

        # Variable Selection
        var_frame = ttk.LabelFrame(control_panel, text="SURFACE VARIABLES", padding=15)
        var_frame.pack(fill=tk.X, pady=(0, 15))

        ttk.Label(var_frame, text="X Variable:").pack(anchor=tk.W, pady=(0, 5))
        self.surface_x_var = tk.StringVar()
        self.surface_x_combo = ttk.Combobox(var_frame, textvariable=self.surface_x_var,
                                            state='readonly')
        self.surface_x_combo.pack(fill=tk.X, pady=(0, 10))

        ttk.Label(var_frame, text="Y Variable:").pack(anchor=tk.W, pady=(0, 5))
        self.surface_y_var = tk.StringVar()
        self.surface_y_combo = ttk.Combobox(var_frame, textvariable=self.surface_y_var,
                                            state='readonly')
```

```
        self.surface_y_combo.pack(fill=tk.X, pady=(0, 10))

        ttk.Label(var_frame, text="Z Variable (Height):").pack(anchor=tk.W, pady=(0, 5))
        self.surface_z_var = tk.StringVar()
        self.surface_z_combo = ttk.Combobox(var_frame, textvariable=self.surface_z_var,
                                            state='readonly')
        self.surface_z_combo.pack(fill=tk.X)

        # Surface Parameters
        param_frame = ttk.LabelFrame(control_panel, text="SURFACE PARAMETERS",
padding=15)
        param_frame.pack(fill=tk.X, pady=(0, 15))

        ttk.Label(param_frame, text="Grid Resolution:").pack(anchor=tk.W, pady=(0, 5))
        self.surface_grid_var = tk.IntVar(value=50)
        grid_scale = ttk.Scale(param_frame, from_=20, to=200,
                               variable=self.surface_grid_var, orient=tk.HORIZONTAL)
        grid_scale.pack(fill=tk.X, pady=(0, 10))

        ttk.Label(param_frame, text="Smoothing Factor:").pack(anchor=tk.W, pady=(0, 5))
        self.surface_smooth_var = tk.DoubleVar(value=0.5)
        smooth_scale = ttk.Scale(param_frame, from_=0.1, to=2.0,
                                 variable=self.surface_smooth_var, orient=tk.HORIZONTAL)
        smooth_scale.pack(fill=tk.X)

        # Buttons
        btn_frame = ttk.Frame(control_panel)
        btn_frame.pack(fill=tk.X, pady=10)

        btn_generate = ttk.Button(btn_frame, text="🌊 GENERATE 3D SURFACE",
                                  command=self.generate_3d_surface)
        btn_generate.pack(fill=tk.X, pady=5)

        # Plot Area
        plot_area = ttk.Frame(self.tab_3d_surface)
        plot_area.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

        # Create matplotlib 3D figure
        self.fig_surface_3d = Figure(figsize=(10, 8), facecolor=self.colors['chart_bg'])
        self.ax_surface_3d = self.fig_surface_3d.add_subplot(111, projection='3d')

        # Create canvas
        self.canvas_surface_3d = FigureCanvasTkAgg(self.fig_surface_3d, plot_area)
        self.canvas_surface_3d.get_tk_widget().pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

        # Add toolbar
        toolbar = NavigationToolbar2Tk(self.canvas_surface_3d, plot_area)
        toolbar.update()

    def load_hab_data(self):
        """Load Pseudo-nitzschia HAB data"""
        file_path = filedialog.askopenfilename(
```

```
        title="Select Pseudo-nitzschia HAB Data",
        filetypes=[
            ("CSV files", "*.csv"),
            ("Excel files", "*.xlsx *.xls"),
            ("All files", "*.*")
        ]
    )

    if file_path:
        try:
            if file_path.endswith('.csv'):
                self.dataframe = pd.read_csv(file_path)
            else:
                self.dataframe = pd.read_excel(file_path)

            # Clean column names
            self.dataframe.columns = [col.strip() for col in self.dataframe.columns]

            self.update_data_display()
            self.update_all_comboboxes()
            self.update_info_panel(file_path)

            self.status_bar.config(
                text=f"✓ Loaded: {os.path.basename(file_path)} "
                     f"({self.dataframe.shape[0]} samples)"
            )

            messagebox.showinfo("Success",
                f"Pseudo-nitzschia data loaded successfully!\n\n"
                f"File: {os.path.basename(file_path)}\n"
                f"Samples: {self.dataframe.shape[0]}\n"
                f"Variables: {self.dataframe.shape[1]}")

        except Exception as e:
            messagebox.showerror("Load Error",
                f"Failed to load file:\n\n{str(e)}")
            self.status_bar.config(text="✗ Failed to load data")

def generate_3d_sample_data(self):
    """Generate 3D sample Pseudo-nitzschia data"""
    try:
        np.random.seed(42)
        n_samples = 800

        # Define California coastal stations with coordinates
        california_stations = {
            'North Coast': {
                'Crescent City': {'lat': 41.7458, 'lon': -124.1831},
                'Trinidad': {'lat': 41.0542, 'lon': -124.1439},
                'Eureka': {'lat': 40.8021, 'lon': -124.1637}
            },
            'North Central': {
```

```
            'Fort Bragg': {'lat': 39.4457, 'lon': -123.8053},
            'Point Arena': {'lat': 38.9088, 'lon': -123.6931},
            'Bodega Bay': {'lat': 38.3331, 'lon': -123.0481}
        },
        'Central Coast': {
            'San Francisco': {'lat': 37.7749, 'lon': -122.4194},
            'Monterey': {'lat': 36.6002, 'lon': -121.8947},
            'Morro Bay': {'lat': 35.3658, 'lon': -120.8499}
        },
        'South Central': {
            'Santa Barbara': {'lat': 34.4208, 'lon': -119.6982},
            'Ventura': {'lat': 34.2746, 'lon': -119.2290},
            'Channel Islands': {'lat': 34.0488, 'lon': -119.4590}
        },
        'Southern Coast': {
            'Los Angeles': {'lat': 33.7701, 'lon': -118.1937},
            'Orange County': {'lat': 33.7175, 'lon': -117.8311},
            'San Diego': {'lat': 32.7157, 'lon': -117.1611}
        }
    }

    # Generate sample data
    sample_data = []

    for year in range(2010, 2023):
        for month in range(1, 13):
            for region, stations in
california_stations.items():
                for station, coords in stations.items():
                    # Base environmental conditions
                    lat = coords['lat']
                    lon = coords['lon']

                    # Seasonal patterns
                    seasonal_factor = 0.5 + 0.5 * np.sin(2 * np.pi * (month - 3) / 12)

                    # Regional factors (higher blooms in central coast)
                    region_factors = {
                        'North Coast': 0.7,
                        'North Central': 0.9,
                        'Central Coast': 1.5,
                        'South Central': 1.2,
                        'Southern Coast': 0.8
                    }
                    region_factor = region_factors[region]

                    # Pseudo-nitzschia cell density (cells/L)
                    # Base with lognormal distribution
                    base_density = np.random.lognormal(mean=6, sigma=1.2)

                    # Add seasonal and regional effects
                    seasonal_effect = seasonal_factor * np.random.uniform(0.8, 1.2)
```

```
                regional_effect = region_factor * np.random.uniform(0.9, 1.1)

                # Random bloom events (10% chance)
                if np.random.random() < 0.1:
                    bloom_multiplier = np.random.uniform(5, 20)
                else:
                    bloom_multiplier = 1.0

                cell_density = base_density * seasonal_effect * regional_effect *
bloom_multiplier

                # Domoic acid concentration (µg/L)
                da_concentration = 0.01 * cell_density * np.random.uniform(0.5, 2.0)

                # Environmental parameters
                temperature = 12 + 6 * np.sin(2 * np.pi * month / 12) + np.random.normal(0, 2)
                salinity = 33 + np.random.normal(0, 1)
                nitrate = np.random.uniform(5, 25)
                phosphate = nitrate / 16 * np.random.uniform(0.8, 1.2)
                silicate = nitrate * np.random.uniform(0.5, 2)
                chlorophyll = 0.1 * cell_density**0.5 * np.random.uniform(0.5, 1.5)

                # Upwelling index (higher in north)
                base_upwelling = 50
                if region in ['North Coast', 'North Central']:
                    base_upwelling += 100
                upwelling_index = base_upwelling + np.random.normal(0, 30)

                # Bathymetry (depth)
                depth = np.random.uniform(10, 200)

                # Distance from shore
                distance_from_shore = np.random.uniform(0.1, 20)

                # Current speed and direction
                current_speed = np.random.uniform(0.1, 1.0)
                current_direction = np.random.uniform(0, 360)

                sample_data.append({
                    'Date': f'{year}-{month:02d}-15',
                    'Year': year,
                    'Month': month,
                    'Season': self.get_season(month),
                    'Region': region,
                    'Station': station,
                    'Latitude': lat,
                    'Longitude': lon,
                    'Depth_m': round(depth, 1),
                    'Distance_From_Shore_km': round(distance_from_shore, 1),
                    'Pseudo_nitzschia_cells_L': int(cell_density),
                    'Domoic_Acid_ug_L': round(da_concentration, 3),
                    'Temperature_C': round(temperature, 1),
```

```
                    'Salinity_PSU': round(salinity, 1),
                    'Nitrate_uM': round(nitrate, 2),
                    'Phosphate_uM': round(phosphate, 2),
                    'Silicate_uM': round(silicate, 2),
                    'Chlorophyll_mg_m3': round(chlorophyll, 3),
                    'Upwelling_Index': round(upwelling_index, 1),
                    'Wind_Speed_m_s': np.random.uniform(2, 10),
                    'Current_Speed_m_s': round(current_speed, 2),
                    'Current_Direction_deg': round(current_direction, 1),
                    'Sea_Level_Pressure_hPa': np.random.uniform(1010, 1020),
                    'Precipitation_mm': np.random.exponential(2),
                    'PAR_umol_m2_s': np.random.uniform(100, 600),
                    'pH': np.random.uniform(7.8, 8.3),
                    'Dissolved_Oxygen_mg_L': np.random.uniform(6, 9),
                    'Turbidity_NTU': np.random.uniform(1, 10),
                    'Bloom_Flag': int(cell_density > 10000)
                })

        self.dataframe = pd.DataFrame(sample_data)
        self.stations = california_stations
        self.update_data_display()
        self.update_all_comboboxes()
        self.update_info_panel("3D Sample Pseudo-nitzschia Data")

        self.status_bar.config(
            text=f"✓ Generated 3D sample data with {len(self.dataframe)} samples"
        )

        messagebox.showinfo("3D Sample Data",
            "Advanced 3D Pseudo-nitzschia data has been generated!\n\n"
            "Features:\n"
            "• 13 years of data (2010-2022)\n"
            "• 5 California coastal regions\n"
            "• 15 stations with real coordinates\n"
            "• 3D spatial distribution\n"
            "• Seasonal bloom patterns\n"
            "• 20+ environmental variables\n"
            "• Perfect for 3D visualization!")

    except Exception as e:
        messagebox.showerror("Error", f"Failed to generate sample data:\n{str(e)}")

def get_season(self, month):
    """Get season from month"""
    if month in [12, 1, 2]:
        return 'Winter'
    elif month in [3, 4, 5]:
        return 'Spring'
    elif month in [6, 7, 8]:
        return 'Summer'
    else:
        return 'Fall'
```

```
    def update_data_display(self):
        """Update the treeview with loaded data"""
        # Clear existing data
        for item in self.data_tree.get_children():
            self.data_tree.delete(item)

        if self.dataframe is None:
            return

        # Set up columns
        columns = list(self.dataframe.columns)
        self.data_tree['columns'] = columns

        # Configure column headings
        for col in columns:
            self.data_tree.heading(col, text=col, anchor=tk.CENTER)
            col_width = max(len(col) * 8, 80)
            self.data_tree.column(col, width=col_width, anchor=tk.CENTER, minwidth=60)

        # Insert first 100 rows
        max_rows = min(100, len(self.dataframe))
        for idx in range(max_rows):
            row = self.dataframe.iloc[idx]
            values = [row[col] for col in columns]
            self.data_tree.insert('', 'end', values=values)

        if len(self.dataframe) > 100:
            self.status_bar.config(text=f"Displaying first 100 of {len(self.dataframe)} rows")

    def update_all_comboboxes(self):
        """Update all comboboxes with column names and values"""
        if self.dataframe is None:
            return

        columns = list(self.dataframe.columns)
        numeric_cols = self.dataframe.select_dtypes(include=[np.number]).columns.tolist()

        # Update all combobox configurations
        combobox_configs = [
            # Spatial tab
            (self.spatial_x_combo, self.spatial_x_var, columns),
            (self.spatial_y_combo, self.spatial_y_var, columns),
            (self.spatial_z_combo, self.spatial_z_var, numeric_cols),
            (self.spatial_color_combo, self.spatial_color_var, numeric_cols),
            (self.spatial_year_combo, self.spatial_year_var, ['All'] +
sorted(self.dataframe['Year'].unique().astype(str).tolist())),
            (self.spatial_month_combo, self.spatial_month_var, ['All'] + [calendar.month_name[i] for i
in range(1, 13)]),

            # Temporal tab
            (self.temporal_x_combo, self.temporal_x_var, columns),
```

```
        (self.temporal_y_combo, self.temporal_y_var, numeric_cols),
        (self.temporal_z_combo, self.temporal_z_var, numeric_cols),
        (self.temporal_region_combo, self.temporal_region_var, ['All'] +
sorted(self.dataframe['Region'].unique().tolist())),

        # Environmental tab
        (self.env_pn_combo, self.env_pn_var, numeric_cols),
        (self.env_var1_combo, self.env_var1, numeric_cols),
        (self.env_var2_combo, self.env_var2, numeric_cols),

        # Statistics tab
        (self.stats_var1_combo, self.stats_var1, numeric_cols),
        (self.stats_var2_combo, self.stats_var2, numeric_cols),

    (self.stats_var3_combo, self.stats_var3, numeric_cols),

        # Surface tab
        (self.surface_x_combo, self.surface_x_var, numeric_cols),
        (self.surface_y_combo, self.surface_y_var, numeric_cols),
        (self.surface_z_combo, self.surface_z_var, numeric_cols)
      ]

      for combo, var, values in combobox_configs:
        combo['values'] = values
        if values:
          # Set sensible defaults
          if combo in [self.spatial_x_combo, self.temporal_x_combo]:
            # Prefer time/date columns for x-axis
            time_cols = [col for col in values if any(keyword in col.lower()
                                    for keyword in ['date', 'year', 'month', 'time'])]
            if time_cols:
              var.set(time_cols[0])
            else:
              var.set(values[0])
          elif combo in [self.spatial_y_combo, self.spatial_z_combo, self.spatial_color_combo,
                self.env_pn_combo, self.surface_z_combo]:
            # Prefer Pseudo-nitzschia related variables
            pn_cols = [col for col in values if any(keyword in col.lower()
                                    for keyword in ['nitzschia', 'domoic', 'cells', 'bloom'])]
            if pn_cols:
              var.set(pn_cols[0])
            else:
              var.set(values[0])
          else:
            var.set(values[0])

    def update_info_panel(self, file_path):
      """Update information panel"""
      info = "3D PSEUDO-NITZSCHIA DATASET INFORMATION\n"
      info += "=" * 70 + "\n\n"
      info += f"Source: {file_path}\n"
      info += f"Total Samples: {self.dataframe.shape[0]:,}\n"
```

```
        info += f"Variables: {self.dataframe.shape[1]:,}\n\n"

        # Spatial information
        if 'Latitude' in self.dataframe.columns and 'Longitude' in self.dataframe.columns:
            info += "SPATIAL COVERAGE:\n"
            info += "-" * 40 + "\n"
            info += f"Latitude Range: {self.dataframe['Latitude'].min():.4f}° to
{self.dataframe['Latitude'].max():.4f}°\n"
            info += f"Longitude Range: {self.dataframe['Longitude'].min():.4f}° to
{self.dataframe['Longitude'].max():.4f}°\n\n"

        # Temporal information
        if 'Year' in self.dataframe.columns:
            years = sorted(self.dataframe['Year'].unique())
            info += f"TIME PERIOD: {min(years)} - {max(years)} ({len(years)} years)\n\n"

        # Region information
        if 'Region' in self.dataframe.columns:
            regions = self.dataframe['Region'].unique()
            info += f"CALIFORNIA REGIONS: {len(regions)}\n"
            for region in regions:
                count = len(self.dataframe[self.dataframe['Region'] == region])
                stations = self.dataframe[self.dataframe['Region'] == region]['Station'].unique()
                info += f"  • {region}: {count:,} samples, {len(stations)} stations\n"
            info += "\n"

        # Bloom statistics
        if 'Pseudo_nitzschia_cells_L' in self.dataframe.columns:
            bloom_threshold = 10000
            bloom_data = self.dataframe[self.dataframe['Pseudo_nitzschia_cells_L'] > bloom_threshold]

            info += "BLOOM STATISTICS:\n"
            info += "-" * 40 + "\n"
            info += f"Bloom threshold: > {bloom_threshold:,} cells/L\n"
            info += f"Bloom events: {len(bloom_data):,}
({len(bloom_data)/len(self.dataframe)*100:.1f}%)\n"
            if len(bloom_data) > 0:
                info += f"Max bloom density: {bloom_data['Pseudo_nitzschia_cells_L'].max():,.0f}
cells/L\n"
                info += f"Max domoic acid: {bloom_data['Domoic_Acid_ug_L'].max():.2f} µg/L\n"
                info += f"Most affected region: {bloom_data['Region'].mode().iloc[0]}\n"
            info += "\n"

        # Environmental variables summary
        env_vars = [col for col in self.dataframe.columns
                    if any(keyword in col.lower() for keyword in
                        ['temp', 'salinity', 'nitrate', 'phosphate', 'chlorophyll', 'upwelling'])]

        if env_vars:
            info += "ENVIRONMENTAL VARIABLES SUMMARY:\n"
            info += "-" * 40 + "\n"
            for col in env_vars[:6]:  # Show first 6
```

```
            if col in self.dataframe.columns:
                data = self.dataframe[col].dropna()
                if len(data) > 0:
                    info += f"• {col}: {data.mean():.2f} ± {data.std():.2f} "
                    info += f"(n={len(data):,})\n"

        self.info_text.delete(1.0, tk.END)
        self.info_text.insert(1.0, info)
        self.info_text.config(state=tk.DISABLED)

    def generate_3d_spatial(self):
        """Generate 3D spatial visualization"""
        if self.dataframe is None:
            messagebox.showwarning("No Data", "Please load data first")
            return

        plot_type = self.spatial_3d_type.get()
        x_col = self.spatial_x_var.get()
        y_col = self.spatial_y_var.get()
        z_col = self.spatial_z_var.get()

        if not all([x_col, y_col, z_col]):
            messagebox.showwarning("Missing Variables",
                                   "Please select X, Y, and Z axes")
            return

        try:
            # Apply time filters
            filtered_df = self.dataframe.copy()

            year = self.spatial_year_var.get()
            if year != 'All':
                filtered_df = filtered_df[filtered_df['Year'] == int(year)]

            month = self.spatial_month_var.get()
            if month != 'All':
                month_num = list(calendar.month_name).index(month)
                filtered_df = filtered_df[filtered_df['Month'] == month_num]

            # Clear previous plot
            self.ax_spatial_3d.clear()

            # Get data
            x_data = filtered_df[x_col].dropna().values
            y_data = filtered_df[y_col].dropna().values
            z_data = filtered_df[z_col].dropna().values

            # Align data lengths
            min_len = min(len(x_data), len(y_data), len(z_data))
            if min_len < 10:
                messagebox.showwarning("Insufficient Data",
                                       f"Only {min_len} data points after filtering")
```

```
            return

        x_data = x_data[:min_len]
        y_data = y_data[:min_len]
        z_data = z_data[:min_len]

        # Set dark green theme
        self.fig_spatial_3d.patch.set_facecolor(self.colors['chart_bg'])
        self.ax_spatial_3d.set_facecolor(self.colors['chart_bg'])

        # Color mapping
        color_by = self.spatial_color_var.get()
        if color_by and color_by in filtered_df.columns:
            color_data = filtered_df[color_by].dropna().values[:min_len]
            colors = plt.cm.get_cmap(self.spatial_cmap_var.get())(color_data / color_data.max())
        else:
            # Color by region if available
            if 'Region' in filtered_df.columns:
                regions = filtered_df['Region'].dropna().values[:min_len]
                unique_regions = np.unique(regions)
                region_colors = plt.cm.get_cmap(self.spatial_cmap_var.get())(
                    np.linspace(0, 1, len(unique_regions))
                )
                color_dict = {region: color for region, color in zip(unique_regions, region_colors)}
                colors = [color_dict[region] for region in regions]
            else:
                colors = plt.cm.get_cmap(self.spatial_cmap_var.get())(np.linspace(0, 1, len(x_data)))

        if plot_type == "scatter3d":
            # 3D scatter plot
            scatter = self.ax_spatial_3d.scatter(x_data, y_data, z_data,
                                                 c=colors, s=30, alpha=0.8,
                                                 depthshade=True, edgecolors='w', linewidth=0.5)

            # Add colorbar if coloring by numeric value
            if color_by and color_by in filtered_df.columns:
                cbar = self.fig_spatial_3d.colorbar(scatter, ax=self.ax_spatial_3d, shrink=0.5,
aspect=10)
                cbar.set_label(color_by, color=self.colors['text'])
                cbar.ax.yaxis.set_tick_params(color=self.colors['text'])

        elif plot_type == "surface3d":
            # Create grid for surface
            xi = np.linspace(x_data.min(), x_data.max(), self.grid_res_var.get())
            yi = np.linspace(y_data.min(), y_data.max(), self.grid_res_var.get())
            xi, yi = np.meshgrid(xi, yi)

            # Interpolate z values
            zi = interpolate.griddata((x_data, y_data), z_data, (xi, yi), method='cubic')

            # Create surface plot
            surf = self.ax_spatial_3d.plot_surface(xi, yi, zi,
```

```
                                cmap=self.spatial_cmap_var.get(),
                                alpha=0.8,
                                linewidth=0.5,
                                antialiased=True)
        self.fig_spatial_3d.colorbar(surf, ax=self.ax_spatial_3d, shrink=0.5, aspect=10)

    elif plot_type == "bar3d":
        # 3D bar chart (regional averages)
        if 'Region' in filtered_df.columns:
            regional_avg = filtered_df.groupby('Region')[z_col].mean()

            regions = regional_avg.index

            # Create 3D bars
            x_pos = range(len(regions))
            y_pos = np.zeros(len(regions))
            z_pos = np.zeros(len(regions))
            dx = 0.8 * np.ones(len(regions))
            dy = 0.8 * np.ones(len(regions))
            dz = regional_avg.values

            bar_colors = plt.cm.get_cmap(self.spatial_cmap_var.get())(
                np.linspace(0, 1, len(regions))
            )

            self.ax_spatial_3d.bar3d(x_pos, y_pos, z_pos,
                                     dx, dy, dz,
                                     color=bar_colors, alpha=0.8, shade=True)

            self.ax_spatial_3d.set_xticks(range(len(regions)))
            self.ax_spatial_3d.set_xticklabels(regions, rotation=45, ha='right')

    elif plot_type == "coastline3d":
        # 3D coastline visualization
        # Create California coastline approximation
        california_coast = {
            'x': np.array([-124.4, -124.0, -123.5, -123.0, -122.5, -122.0,
                           -121.5, -121.0, -120.5, -120.0, -119.5, -119.0,
                           -118.5, -118.0, -117.5, -117.0]),
            'y': np.array([42.0, 41.5, 41.0, 40.5, 40.0, 39.5, 39.0, 38.5,
                           38.0, 37.5, 37.0, 36.5, 36.0, 35.5, 35.0, 34.0, 33.5, 33.0, 32.5])
        }

        # Create surface from coastline
        xi = np.linspace(california_coast['x'].min(), california_coast['x'].max(), 50)
        yi = np.linspace(california_coast['y'].min(), california_coast['y'].max(), 50)
        xi, yi = np.meshgrid(xi, yi)

        # Create synthetic z values based on distance from coast
        zi = np.zeros_like(xi)
        for i in range(len(xi)):
            for j in range(len(yi)):
```

```
                # Simple coast effect
                dist_from_coast = np.sqrt((xi[i,j] + 121)**2 + (yi[i,j] - 37)**2)
                zi[i,j] = np.exp(-dist_from_coast/5) * 10000

            surf = self.ax_spatial_3d.plot_surface(xi, yi, zi,
                                    cmap='YlGnBu',
                                    alpha=0.7,
                                    linewidth=0.5)
            self.fig_spatial_3d.colorbar(surf, ax=self.ax_spatial_3d, shrink=0.5, aspect=10)

            # Add scatter points for stations
            if hasattr(self, 'stations'):
                for region, stations in self.stations.items():
                    for station, coords in stations.items():
                        self.ax_spatial_3d.scatter(coords['lon'], coords['lat'], 0,
                                    color=self.colors['bloom_color'],
                                    s=50, alpha=0.8, edgecolors='w')

        # Set labels and title
        self.ax_spatial_3d.set_xlabel(x_col, color=self.colors['text'], fontsize=12, labelpad=10)
        self.ax_spatial_3d.set_ylabel(y_col, color=self.colors['text'], fontsize=12, labelpad=10)
        self.ax_spatial_3d.set_zlabel(z_col, color=self.colors['text'], fontsize=12, labelpad=10)

        title = f"3D Spatial Analysis: {plot_type.upper().replace('3D', '')}"
        self.ax_spatial_3d.set_title(title, color=self.colors['accent_light'],
                            fontsize=14, pad=20, fontweight='bold')

        # Set view angle
        self.ax_spatial_3d.view_init(elev=self.elevation_var.get(),
                            azim=self.azimuth_var.get())

        # Set tick colors
        self.ax_spatial_3d.tick_params(colors=self.colors['text'])

        # Add grid
        self.ax_spatial_3d.grid(True, color=self.colors['grid'], alpha=0.3)

        # Update canvas
        self.canvas_spatial_3d.draw()

        self.status_bar.config(text=f"✓ Generated 3D spatial plot: {plot_type}")

    except Exception as e:
        messagebox.showerror("3D Spatial Error",
                             f"Failed to generate 3D spatial plot:\n\n{str(e)}")
        self.status_bar.config(text="✗ Failed to generate 3D spatial plot")

def generate_3d_temporal(self):
    """Generate 3D temporal visualization"""
    if self.dataframe is None:
        messagebox.showwarning("No Data", "Please load data first")
        return
```

```
plot_type = self.temporal_3d_type.get()
x_col = self.temporal_x_var.get()
y_col = self.temporal_y_var.get()
z_col = self.temporal_z_var.get()

if not all([x_col, y_col, z_col]):
    messagebox.showwarning("Missing Variables",
                           "Please select X, Y, and Z axes")
    return

try:
    # Apply region filter
    filtered_df = self.dataframe.copy()
    region = self.temporal_region_var.get()
    if region != 'All':
        filtered_df = filtered_df[filtered_df['Region'] == region]

    # Clear previous plot
    self.ax_temporal_3d.clear()

    # Get data
    x_data = filtered_df[x_col].dropna().values
    y_data = filtered_df[y_col].dropna().values
    z_data = filtered_df[z_col].dropna().values

    # Convert date to numeric if needed
    if 'Date' in x_col or 'date' in x_col.lower():
        try:
            filtered_df['Date_numeric'] = pd.to_datetime(filtered_df[x_col]).astype(int) / 10**9
            x_data = filtered_df['Date_numeric'].dropna().values
        except:
            pass

    # Align data lengths
    min_len = min(len(x_data), len(y_data), len(z_data))
    if min_len < 10:
        messagebox.showwarning("Insufficient Data",
                               f"Only {min_len} data points after filtering")
        return

    x_data = x_data[:min_len]
    y_data = y_data[:min_len]
    z_data = z_data[:min_len]

    # Set dark green theme
    self.fig_temporal_3d.patch.set_facecolor(self.colors['chart_bg'])
    self.ax_temporal_3d.set_facecolor(self.colors['chart_bg'])

    if plot_type == "timeseries3d":
        # 3D time series
        # Sort by x for better line plot
```

```
            sort_idx = np.argsort(x_data)
            x_sorted = x_data[sort_idx]
            y_sorted = y_data[sort_idx]
            z_sorted = z_data[sort_idx]

            # Create line plot
            self.ax_temporal_3d.plot(x_sorted, y_sorted, z_sorted,
                                     color=self.colors['accent'],
                                     linewidth=2, alpha=0.8)

            # Add scatter points
            scatter = self.ax_temporal_3d.scatter(x_data, y_data, z_data,
                                                  c=z_data, cmap='viridis',
                                                  s=30, alpha=0.6,
                                                  depthshade=True)

            self.fig_temporal_3d.colorbar(scatter, ax=self.ax_temporal_3d, shrink=0.5, aspect=10)

        elif plot_type == "monthly3d":
            # Monthly patterns in 3D
            if 'Month' in filtered_df.columns:
                monthly_avg = filtered_df.groupby('Month')[[y_col, z_col]].mean()

                months = range(1, 13)
                x_pos = months
                y_vals = monthly_avg[y_col].values
                z_vals = monthly_avg[z_col].values

                # Create 3D line
                self.ax_temporal_3d.plot(x_pos, y_vals, z_vals,
                                         color=self.colors['accent'],
                                         linewidth=2, marker='o')

                # Add projection planes
                self.ax_temporal_3d.plot(x_pos, y_vals, np.zeros_like(z_vals),
                                         color=self.colors['accent_light'],
                                         alpha=0.3, linestyle='--')
                self.ax_temporal_3d.plot(x_pos, np.zeros_like(y_vals), z_vals,
                                         color=self.colors['accent_light'],
                                         alpha=0.3, linestyle='--')

                self.ax_temporal_3d.set_xticks(range(1, 13))
                self.ax_temporal_3d.set_xticklabels([calendar.month_abbr[i] for i in range(1, 13)])

        elif plot_type == "seasonal3d":
            # Seasonal surface
            if 'Season' in filtered_df.columns:
                # Create grid for seasonal surface
                seasons = ['Winter', 'Spring', 'Summer', 'Fall']
                season_map = {season: i for i, season in enumerate(seasons)}

                if 'Year' in filtered_df.columns:
```

```
        years = sorted(filtered_df['Year'].unique())

        # Create meshgrid

  X, Y = np.meshgrid(range(len(years)), range(len(seasons)))
        Z = np.zeros_like(X, dtype=float)

        # Fill Z with average values
        for i, year in enumerate(years):
            for j, season in enumerate(seasons):
                mask = (filtered_df['Year'] == year) & (filtered_df['Season'] == season)
                if mask.any():
                    Z[j, i] = filtered_df.loc[mask, z_col].mean()

        # Create surface
        surf = self.ax_temporal_3d.plot_surface(X, Y, Z,
                                                cmap='viridis',
                                                alpha=0.8,
                                                linewidth=0.5)
        self.fig_temporal_3d.colorbar(surf, ax=self.ax_temporal_3d, shrink=0.5, aspect=10)

        self.ax_temporal_3d.set_xticks(range(len(years)))
        self.ax_temporal_3d.set_xticklabels(years, rotation=45, ha='right')
        self.ax_temporal_3d.set_yticks(range(len(seasons)))
        self.ax_temporal_3d.set_yticklabels(seasons)

# Set labels and title
self.ax_temporal_3d.set_xlabel(x_col, color=self.colors['text'], fontsize=12, labelpad=10)
self.ax_temporal_3d.set_ylabel(y_col, color=self.colors['text'], fontsize=12, labelpad=10)
self.ax_temporal_3d.set_zlabel(z_col, color=self.colors['text'], fontsize=12, labelpad=10)

title = f"3D Temporal Analysis: {plot_type.upper().replace('3D', '')}"
if region != 'All':
    title += f" - {region}"
self.ax_temporal_3d.set_title(title, color=self.colors['accent_light'],
                              fontsize=14, pad=20, fontweight='bold')

# Set view angle
self.ax_temporal_3d.view_init(elev=self.elevation_var.get(),
                              azim=self.azimuth_var.get())

# Set tick colors
self.ax_temporal_3d.tick_params(colors=self.colors['text'])

# Add grid
self.ax_temporal_3d.grid(True, color=self.colors['grid'], alpha=0.3)

# Update canvas
self.canvas_temporal_3d.draw()

self.status_bar.config(text=f"✓ Generated 3D temporal plot: {plot_type}")
```

```
    except Exception as e:
        messagebox.showerror("3D Temporal Error",
                             f"Failed to generate 3D temporal plot:\n\n{str(e)}")
        self.status_bar.config(text="✗ Failed to generate 3D temporal plot")

def analyze_3d_environmental(self):
    """Analyze environmental drivers in 3D"""
    if self.dataframe is None:
        messagebox.showwarning("No Data", "Please load data first")
        return

    analysis_type = self.env_3d_type.get()
    pn_var = self.env_pn_var.get()
    env_var1 = self.env_var1.get()
    env_var2 = self.env_var2.get()

    if not all([pn_var, env_var1, env_var2]):
        messagebox.showwarning("Missing Variables",
                               "Please select all three variables")
        return

    try:
        # Clear previous plot and results
        self.ax_env_3d.clear()
        self.env_results.delete(1.0, tk.END)

        # Filter valid data
        valid_data = self.dataframe[[pn_var, env_var1, env_var2]].dropna()

        if len(valid_data) < 10:
            messagebox.showwarning("Insufficient Data",
                                   "Not enough data points for analysis")
            return

        x = valid_data[env_var1].values
        y = valid_data[env_var2].values
        z = valid_data[pn_var].values

        # Set dark green theme
        self.fig_env_3d.patch.set_facecolor(self.colors['chart_bg'])
        self.ax_env_3d.set_facecolor(self.colors['chart_bg'])

        results_text = "3D ENVIRONMENTAL DRIVER ANALYSIS\n"
        results_text += "=" * 70 + "\n\n"
        results_text += f"Pseudo-nitzschia: {pn_var}\n"
        results_text += f"Environmental 1: {env_var1}\n"
        results_text += f"Environmental 2: {env_var2}\n"
        results_text += f"Samples: {len(valid_data):,}\n\n"

        if analysis_type == "correlation3d":
            # 3D correlation surface
            # Create grid
```

```
    xi = np.linspace(x.min(), x.max(), self.grid_res_var.get())
    yi = np.linspace(y.min(), y.max(), self.grid_res_var.get())
    xi, yi = np.meshgrid(xi, yi)

    # Interpolate z values
    zi = interpolate.griddata((x, y), z, (xi, yi), method='cubic')

    # Create surface
    surf = self.ax_env_3d.plot_surface(xi, yi, zi,
                                       cmap='RdYlGn',
                                       alpha=0.8,
                                       linewidth=0.5)
    self.fig_env_3d.colorbar(surf, ax=self.ax_env_3d, shrink=0.5, aspect=10)

    # Calculate correlations
    corr1, p1 = stats.pearsonr(x, z)
    corr2, p2 = stats.pearsonr(y, z)

    results_text += "CORRELATION RESULTS:\n"
    results_text += "-" * 40 + "\n"
    results_text += f"{pn_var} vs {env_var1}: r = {corr1:.3f}, p = {p1:.6f}\n"
    results_text += f"{pn_var} vs {env_var2}: r = {corr2:.3f}, p = {p2:.6f}\n\n"

    if p1 < 0.05:
        results_text += f"Significant correlation with {env_var1} (p < 0.05)\n"
    if p2 < 0.05:
        results_text += f"Significant correlation with {env_var2} (p < 0.05)\n"

elif analysis_type == "regression3d":
    # 3D regression surface
    # Multiple linear regression
    from sklearn.linear_model import LinearRegression

    X = np.column_stack((x, y))
    model = LinearRegression()
    model.fit(X, z)

    # Create prediction surface
    xi = np.linspace(x.min(), x.max(), self.grid_res_var.get())
    yi = np.linspace(y.min(), y.max(), self.grid_res_var.get())
    xi, yi = np.meshgrid(xi, yi)

    X_pred = np.column_stack((xi.ravel(), yi.ravel()))
    zi = model.predict(X_pred).reshape(xi.shape)

    # Create surface
    surf = self.ax_env_3d.plot_surface(xi, yi, zi,
                                       cmap='viridis',
                                       alpha=0.8,
                                       linewidth=0.5)
    self.fig_env_3d.colorbar(surf, ax=self.ax_env_3d, shrink=0.5, aspect=10)
```

```
            # Add original data points
            self.ax_env_3d.scatter(x, y, z,
                                   color=self.colors['bloom_color'],
                                   s=20, alpha=0.6)

            results_text += "MULTIPLE REGRESSION RESULTS:\n"
            results_text += "-" * 40 + "\n"
            results_text += f"R² score: {model.score(X, z):.3f}\n"
            results_text += f"Coefficient for {env_var1}: {model.coef_[0]:.3f}\n"
            results_text += f"Coefficient for {env_var2}: {model.coef_[1]:.3f}\n"
            results_text += f"Intercept: {model.intercept_:.3f}\n\n"

        elif analysis_type == "temperature3d":
            # Temperature effects in 3D
            scatter = self.ax_env_3d.scatter(x, y, z,
                                             c=z, cmap='RdYlBu_r',
                                             s=30, alpha=0.8,
                                             depthshade=True)
            self.fig_env_3d.colorbar(scatter, ax=self.ax_env_3d, shrink=0.5, aspect=10)

            # Add temperature thresholds
            temp_thresholds = [12, 18, 24]
            for threshold in temp_thresholds:
                mask = x > threshold
                if mask.any():
                    self.ax_env_3d.scatter(x[mask], y[mask], z[mask],
                                           color='red', s=50, alpha=0.5,
                                           label=f'T > {threshold}°C')

            results_text += "TEMPERATURE EFFECTS ANALYSIS:\n"
            results_text += "-" * 40 + "\n"
            results_text += f"Optimal temperature range for blooms: 12-18°C\n"
            results_text += f"Temperature range in data: {x.min():.1f} - {x.max():.1f}°C\n"
            results_text += f"Mean temperature: {x.mean():.1f}°C\n\n"

        # Set labels and title
        self.ax_env_3d.set_xlabel(env_var1, color=self.colors['text'], fontsize=12, labelpad=10)
        self.ax_env_3d.set_ylabel(env_var2, color=self.colors['text'], fontsize=12, labelpad=10)
        self.ax_env_3d.set_zlabel(pn_var, color=self.colors['text'], fontsize=12, labelpad=10)

        title = f"3D Environmental Analysis: {analysis_type.upper().replace('3D', '')}"
        self.ax_env_3d.set_title(title,
color=self.colors['accent_light'],
                                 fontsize=14, pad=20, fontweight='bold')

        # Set view angle
        self.ax_env_3d.view_init(elev=self.elevation_var.get(),
                                 azim=self.azimuth_var.get())

        # Set tick colors
        self.ax_env_3d.tick_params(colors=self.colors['text'])
```

```
        # Add grid
        self.ax_env_3d.grid(True, color=self.colors['grid'], alpha=0.3)

        # Update results text
        self.env_results.insert(1.0, results_text)
        self.env_results.config(state=tk.DISABLED)

        # Update canvas
        self.canvas_env_3d.draw()

        self.status_bar.config(text=f"✓ Analyzed 3D environmental drivers: {analysis_type}")

    except Exception as e:
        messagebox.showerror("3D Environmental Error",
                             f"Failed to analyze environmental drivers:\n\n{str(e)}")
        self.status_bar.config(text="✗ Failed to analyze environmental drivers")

def calculate_3d_statistics(self):
    """Calculate 3D statistics"""
    if self.dataframe is None:
        messagebox.showwarning("No Data", "Please load data first")
        return

    analysis_type = self.stats_3d_type.get()
    var1 = self.stats_var1.get()
    var2 = self.stats_var2.get()
    var3 = self.stats_var3.get()

    if not all([var1, var2, var3]):
        messagebox.showwarning("Missing Variables",
                               "Please select all three variables")
        return

    try:
        # Clear previous plot and results
        self.ax_stats_3d.clear()
        self.stats_results.delete(1.0, tk.END)

        # Filter valid data
        valid_data = self.dataframe[[var1, var2, var3]].dropna()

        if len(valid_data) < 10:
            messagebox.showwarning("Insufficient Data",
                                   "Not enough data points for analysis")
            return

        x = valid_data[var1].values
        y = valid_data[var2].values
        z = valid_data[var3].values

        # Set dark green theme
        self.fig_stats_3d.patch.set_facecolor(self.colors['chart_bg'])
```

```
self.ax_stats_3d.set_facecolor(self.colors['chart_bg'])

results_text = "3D STATISTICAL ANALYSIS\n"
results_text += "=" * 70 + "\n\n"
results_text += f"Variable 1: {var1}\n"
results_text += f"Variable 2: {var2}\n"
results_text += f"Variable 3: {var3}\n"
results_text += f"Samples: {len(valid_data):,}\n\n"

# Calculate basic statistics
for var_name, data in [(var1, x), (var2, y), (var3, z)]:
    results_text += f"{var_name} Statistics:\n"
    results_text += f"  Mean: {np.mean(data):.3f}\n"
    results_text += f"  Std: {np.std(data):.3f}\n"
    results_text += f"  Min: {np.min(data):.3f}\n"
    results_text += f"  Max: {np.max(data):.3f}\n"
    results_text += f"  Median: {np.median(data):.3f}\n\n"

if analysis_type == "distribution3d":
    # 3D distribution visualization
    # Create 3D histogram
    hist, edges = np.histogramdd(np.column_stack((x, y, z)), bins=15)

    # Create 3D bar histogram
    xpos, ypos, zpos = np.meshgrid(edges[0][:-1], edges[1][:-1], edges[2][:-1], indexing="ij")
    xpos = xpos.flatten()
    ypos = ypos.flatten()
    zpos = zpos.flatten()

    dx = (edges[0][1] - edges[0][0]) * 0.8 * np.ones_like(zpos)
    dy = (edges[1][1] - edges[1][0]) * 0.8 * np.ones_like(zpos)
    dz = (edges[2][1] - edges[2][0]) * 0.8 * np.ones_like(zpos)
    dw = hist.flatten()

    colors = plt.cm.viridis(dw / dw.max())

    self.ax_stats_3d.bar3d(xpos, ypos, zpos, dx, dy, dz, dw,
                           color=colors, alpha=0.8, shade=True)

    results_text += "3D DISTRIBUTION ANALYSIS:\n"
    results_text += "-" * 40 + "\n"
    results_text += f"Total bins: {len(xpos)}\n"
    results_text += f"Max frequency: {dw.max():.0f}\n"

elif analysis_type == "correlation3d":
    # 3D correlation matrix visualization
    corr_matrix = np.corrcoef([x, y, z])

    # Create 3D surface of correlation matrix
    X, Y = np.meshgrid(range(3), range(3))
    surf = self.ax_stats_3d.plot_surface(X, Y, corr_matrix,
                                         cmap='RdYlBu',
```

```
                                alpha=0.8,
                                linewidth=0.5)
            self.fig_stats_3d.colorbar(surf, ax=self.ax_stats_3d, shrink=0.5, aspect=10)

            # Set labels
            self.ax_stats_3d.set_xticks([0, 1, 2])
            self.ax_stats_3d.set_yticks([0, 1, 2])
            self.ax_stats_3d.set_xticklabels([var1[:10], var2[:10], var3[:10]])
            self.ax_stats_3d.set_yticklabels([var1[:10], var2[:10], var3[:10]])

            results_text += "CORRELATION MATRIX:\n"
            results_text += "-" * 40 + "\n"
            results_text += f"{var1} vs {var2}: r = {corr_matrix[0,1]:.3f}\n"
            results_text += f"{var1} vs {var3}: r = {corr_matrix[0,2]:.3f}\n"
            results_text += f"{var2} vs {var3}: r = {corr_matrix[1,2]:.3f}\n"

        elif analysis_type == "pca3d":
            # 3D PCA visualization
            from sklearn.decomposition import PCA
            from sklearn.preprocessing import StandardScaler

            # Standardize data
            scaler = StandardScaler()
            X_scaled = scaler.fit_transform(np.column_stack((x, y, z)))

            # Perform PCA
            pca = PCA(n_components=3)
            X_pca = pca.fit_transform(X_scaled)

            # Plot PCA components
            scatter = self.ax_stats_3d.scatter(X_pca[:, 0], X_pca[:, 1], X_pca[:, 2],
                                c=z, cmap='viridis',
                                s=30, alpha=0.8)
            self.fig_stats_3d.colorbar(scatter, ax=self.ax_stats_3d, shrink=0.5, aspect=10)

            results_text += "PRINCIPAL COMPONENT ANALYSIS:\n"
            results_text += "-" * 40 + "\n"
            results_text += f"Explained variance ratio:\n"
            for i, ratio in enumerate(pca.explained_variance_ratio_):
                results_text += f"  PC{i+1}: {ratio*100:.1f}%\n"
            results_text += f"Total explained variance:
{sum(pca.explained_variance_ratio_)*100:.1f}%\n\n"

            self.ax_stats_3d.set_xlabel('PC1', color=self.colors['text'], fontsize=12, labelpad=10)
            self.ax_stats_3d.set_ylabel('PC2', color=self.colors['text'], fontsize=12, labelpad=10)
            self.ax_stats_3d.set_zlabel('PC3', color=self.colors['text'], fontsize=12, labelpad=10)

        # Set labels and title (if not set by PCA)
        if analysis_type != "pca3d":
            self.ax_stats_3d.set_xlabel(var1, color=self.colors['text'], fontsize=12, labelpad=10)
            self.ax_stats_3d.set_ylabel(var2, color=self.colors['text'], fontsize=12, labelpad=10)
            self.ax_stats_3d.set_zlabel(var3, color=self.colors['text'], fontsize=12, labelpad=10)
```

```
            title = f"3D Statistical Analysis: {analysis_type.upper().replace('3D', '')}"
            self.ax_stats_3d.set_title(title, color=self.colors['accent_light'],
                                       fontsize=14, pad=20, fontweight='bold')

            # Set view angle
            self.ax_stats_3d.view_init(elev=self.elevation_var.get(),
                                       azim=self.azimuth_var.get())

            # Set tick colors
            self.ax_stats_3d.tick_params(colors=self.colors['text'])

            # Add grid
            self.ax_stats_3d.grid(True, color=self.colors['grid'], alpha=0.3)

            # Update results text
            self.stats_results.insert(1.0, results_text)
            self.stats_results.config(state=tk.DISABLED)

            # Update canvas
            self.canvas_stats_3d.draw()

            self.status_bar.config(text=f"✓ Calculated 3D statistics: {analysis_type}")

        except Exception as e:
            messagebox.showerror("3D Statistics Error",
                                 f"Failed to calculate 3D statistics:\n\n{str(e)}")
            self.status_bar.config(text="✗ Failed to calculate 3D statistics")

    def assess_3d_hazard(self):
        """Assess 3D hazard"""
        if self.dataframe is None:
            messagebox.showwarning("No Data", "Please load data first")
            return

        analysis_type = self.hazard_3d_type.get()
        cell_threshold = self.hazard_threshold_var.get()
        da_threshold = self.da_threshold_var.get()

        try:
            # Clear previous plot and results
            self.ax_hazard_3d.clear()
            self.risk_text.delete(1.0, tk.END)

            # Set
dark green theme
            self.fig_hazard_3d.patch.set_facecolor(self.colors['chart_bg'])
            self.ax_hazard_3d.set_facecolor(self.colors['chart_bg'])

            results_text = "3D HAZARD ASSESSMENT\n"
            results_text += "=" * 70 + "\n\n"
```

```
        if 'Pseudo_nitzschia_cells_L' in self.dataframe.columns:
            # Calculate bloom statistics
            bloom_mask = self.dataframe['Pseudo_nitzschia_cells_L'] > cell_threshold
            bloom_count = bloom_mask.sum()
            bloom_percentage = bloom_count / len(self.dataframe) * 100

            results_text += f"BLOOM DETECTION (Threshold: {cell_threshold:,} cells/L):\n"
            results_text += "-" * 40 + "\n"
            results_text += f"Bloom events: {bloom_count:,} ({bloom_percentage:.1f}%)\n"

            if bloom_count > 0:
                max_bloom = self.dataframe.loc[bloom_mask, 'Pseudo_nitzschia_cells_L'].max()
                results_text += f"Maximum bloom density: {max_bloom:,.0f} cells/L\n"

                # Domoic acid analysis
                if 'Domoic_Acid_ug_L' in self.dataframe.columns:
                    toxic_mask = self.dataframe['Domoic_Acid_ug_L'] > da_threshold
                    toxic_count = toxic_mask.sum()
                    toxic_percentage = toxic_count / len(self.dataframe) * 100

                    results_text += f"\nTOXICITY ASSESSMENT (Threshold: {da_threshold} µg/L):\
n"
                    results_text += "-" * 40 + "\n"
                    results_text += f"Toxic events: {toxic_count:,} ({toxic_percentage:.1f}%)\n"

                    if toxic_count > 0:
                        max_da = self.dataframe.loc[toxic_mask, 'Domoic_Acid_ug_L'].max()
                        results_text += f"Maximum domoic acid: {max_da:.2f} µg/L\n"

            # Regional risk analysis
            if 'Region' in self.dataframe.columns:
                results_text += "\nREGIONAL RISK ASSESSMENT:\n"
                results_text += "-" * 40 + "\n"

                for region in self.california_regions:
                    if region in self.dataframe['Region'].unique():
                        region_data = self.dataframe[self.dataframe['Region'] == region]
                        region_blooms = len(region_data[region_data['Pseudo_nitzschia_cells_L'] >
cell_threshold])
                        region_risk = region_blooms / len(region_data) * 100

                        risk_level = "🔴 High" if region_risk > 5 else "🟡 Moderate" if region_risk > 1
else "🟢 Low"
                        results_text += f"{region}: {risk_level} ({region_blooms} blooms,
{region_risk:.1f}%)\n"

        if analysis_type == "risk3d":
            # 3D risk surface
            if 'Latitude' in self.dataframe.columns and 'Longitude' in self.dataframe.columns:
                # Create risk grid
                xi = np.linspace(self.dataframe['Longitude'].min(),
                                 self.dataframe['Longitude'].max(),
```

```
                    self.grid_res_var.get())
            yi = np.linspace(self.dataframe['Latitude'].min(),
                    self.dataframe['Latitude'].max(),
                    self.grid_res_var.get())
            xi, yi = np.meshgrid(xi, yi)

            # Calculate risk (bloom probability)
            zi = np.zeros_like(xi)
            for i in range(len(xi)):
                for j in range(len(yi)):
                    # Simple distance-based risk calculation
                    dist = np.sqrt((self.dataframe['Longitude'] - xi[i,j])**2 +
                            (self.dataframe['Latitude'] - yi[i,j])**2)
                    nearby = dist < 0.5  # Within 0.5 degrees
                    if nearby.any():
                        risk = (self.dataframe.loc[nearby, 'Pseudo_nitzschia_cells_L'] >
cell_threshold).mean()
                        zi[i,j] = risk * 100  # Convert to percentage

            # Create surface
            surf = self.ax_hazard_3d.plot_surface(xi, yi, zi,
                                    cmap='RdYlGn_r',
                                    alpha=0.8,
                                    linewidth=0.5)
            self.fig_hazard_3d.colorbar(surf, ax=self.ax_hazard_3d, shrink=0.5, aspect=10,
                                label='Bloom Probability (%)')

            # Add actual bloom locations
            bloom_data = self.dataframe[bloom_mask]
            if len(bloom_data) > 0:
                self.ax_hazard_3d.scatter(bloom_data['Longitude'],
                                bloom_data['Latitude'],
                                np.zeros(len(bloom_data)),
                                color=self.colors['bloom_color'],
                                s=50, alpha=0.8, label='Actual Blooms')

            self.ax_hazard_3d.set_xlabel('Longitude', color=self.colors['text'], fontsize=12,
labelpad=10)
            self.ax_hazard_3d.set_ylabel('Latitude', color=self.colors['text'], fontsize=12,
labelpad=10)
            self.ax_hazard_3d.set_zlabel('Bloom Risk (%)', color=self.colors['text'], fontsize=12,
labelpad=10)

        elif analysis_type == "bloom3d":
            # 3D bloom volume visualization
            if all(col in self.dataframe.columns for col in ['Longitude', 'Latitude',
'Pseudo_nitzschia_cells_L']):
                # Create 3D scatter with size based on bloom intensity
                sizes = np.clip(self.dataframe['Pseudo_nitzschia_cells_L'] / 1000, 10, 100)
                colors = np.where(self.dataframe['Pseudo_nitzschia_cells_L'] > cell_threshold,
                        self.colors['bloom_color'], self.colors['safe_color'])
```

```
            scatter = self.ax_hazard_3d.scatter(self.dataframe['Longitude'],
                                                self.dataframe['Latitude'],
                                                self.dataframe['Pseudo_nitzschia_cells_L'],
                                                c=colors, s=sizes, alpha=0.6)

            # Add threshold plane
            xx, yy = np.meshgrid([self.dataframe['Longitude'].min(),
self.dataframe['Longitude'].max()],
                                 [self.dataframe['Latitude'].min(), self.dataframe['Latitude'].max()])
            zz = np.ones_like(xx) * cell_threshold

            self.ax_hazard_3d.plot_surface(xx, yy, zz, color=self.colors['warning_color'],
                                           alpha=0.3, label=f'Threshold: {cell_threshold:,} cells/L')

            self.ax_hazard_3d.set_xlabel('Longitude', color=self.colors['text'], fontsize=12,
labelpad=10)
            self.ax_hazard_3d.set_ylabel('Latitude', color=self.colors['text'], fontsize=12,
labelpad=10)
            self.ax_hazard_3d.set_zlabel('Cell Density (cells/L)', color=self.colors['text'],
fontsize=12, labelpad=10)

        # Set title
        title = f"3D Hazard Assessment: {analysis_type.upper().replace('3D', '')}"
        self.ax_hazard_3d.set_title(title, color=self.colors['accent_light'],
                                    fontsize=14, pad=20, fontweight='bold')

        # Set view angle
        self.ax_hazard_3d.view_init(elev=self.elevation_var.get(),
                                    azim=self.azimuth_var.get())

        # Set tick colors
        self.ax_hazard_3d.tick_params(colors=self.colors['text'])

        # Add grid
        self.ax_hazard_3d.grid(True, color=self.colors['grid'], alpha=0.3)

        # Add recommendations
        results_text += "\n" + "=" * 70 + "\n"
        results_text += "RECOMMENDATIONS:\n"
        results_text += "-" * 40 + "\n"

        if bloom_percentage > 5:
            results_text += " 🔴 HIGH RISK - Immediate Action Required:\n"
            results_text += "• Increase monitoring frequency to daily\n"
            results_text += "• Test shellfish for domoic acid\n"
            results_text += "• Issue public health warnings\n"
            results_text += "• Consider harvesting closures\n"
        elif bloom_percentage > 1:
            results_text += " 🟡 MODERATE RISK - Enhanced Monitoring:\n"
            results_text += "• Increase monitoring frequency to weekly\n"
            results_text += "• Notify stakeholders\n"
            results_text += "• Prepare contingency plans\n"
```

```
            else:
                results_text += " 🟢 LOW RISK - Routine Monitoring:\n"
                results_text += "• Continue routine monitoring\n"
                results_text += "• Maintain standard protocols\n"
                results_text += "• Monitor environmental conditions\n"

            # Update results text
            self.risk_text.insert(1.0, results_text)
            self.risk_text.config(state=tk.DISABLED)

            # Update canvas
            self.canvas_hazard_3d.draw()


        self.status_bar.config(text=f"✓ Assessed 3D hazard: {analysis_type}")

    except Exception as e:
        messagebox.showerror("3D Hazard Error",
                             f"Failed to assess 3D hazard:\n\n{str(e)}")
        self.status_bar.config(text="✗ Failed to assess 3D hazard")

def generate_3d_surface(self):
    """Generate 3D surface plot"""
    if self.dataframe is None:
        messagebox.showwarning("No Data", "Please load data first")
        return

    surface_type = self.surface_3d_type.get()
    x_var = self.surface_x_var.get()
    y_var = self.surface_y_var.get()
    z_var = self.surface_z_var.get()

    if not all([x_var, y_var, z_var]):
        messagebox.showwarning("Missing Variables",
                               "Please select X, Y, and Z variables")
        return

    try:
        # Clear previous plot
        self.ax_surface_3d.clear()

        # Get data
        x_data = self.dataframe[x_var].dropna().values
        y_data = self.dataframe[y_var].dropna().values
        z_data = self.dataframe[z_var].dropna().values

        # Align data lengths
        min_len = min(len(x_data), len(y_data), len(z_data))
        if min_len < 10:
            messagebox.showwarning("Insufficient Data",
                                   f"Only {min_len} valid data points")
            return
```

```
x_data = x_data[:min_len]
y_data = y_data[:min_len]
z_data = z_data[:min_len]

# Set dark green theme
self.fig_surface_3d.patch.set_facecolor(self.colors['chart_bg'])
self.ax_surface_3d.set_facecolor(self.colors['chart_bg'])

# Create grid
grid_res = self.surface_grid_var.get()
xi = np.linspace(x_data.min(), x_data.max(), grid_res)
yi = np.linspace(y_data.min(), y_data.max(), grid_res)
xi, yi = np.meshgrid(xi, yi)

# Interpolate z values with smoothing
zi = interpolate.griddata((x_data, y_data), z_data, (xi, yi), method='cubic')

# Apply smoothing if requested
smooth_factor = self.surface_smooth_var.get()
if smooth_factor > 1.0:
    from scipy.ndimage import gaussian_filter
    zi = gaussian_filter(zi, sigma=smooth_factor)

if surface_type == "interpolated":
    # Interpolated surface
    surf = self.ax_surface_3d.plot_surface(xi, yi, zi,
                                           cmap='viridis',
                                           alpha=0.8,
                                           linewidth=0.5,
                                           antialiased=True)
    self.fig_surface_3d.colorbar(surf, ax=self.ax_surface_3d, shrink=0.5, aspect=10)

elif surface_type == "wireframe":
    # Wireframe surface
    self.ax_surface_3d.plot_wireframe(xi, yi, zi,
                                      color=self.colors['wireframe'],
                                      alpha=0.8,
                                      linewidth=0.5,
                                      rstride=2, cstride=2)

elif surface_type == "contour":
    # Contour surface
    contour = self.ax_surface_3d.contour3D(xi, yi, zi, 50,
                                           cmap='viridis',
                                           alpha=0.8)
    self.fig_surface_3d.colorbar(contour, ax=self.ax_surface_3d, shrink=0.5, aspect=10)

elif surface_type == "colored":
    # Colored surface with height-based coloring
    norm = plt.Normalize(zi.min(), zi.max())
    colors = plt.cm.viridis(norm(zi))
```

```
            surf = self.ax_surface_3d.plot_surface(xi, yi, zi,
                                                   facecolors=colors,
                                                   alpha=0.8,
                                                   linewidth=0.5)

        elif surface_type == "gradient":
            # Gradient surface
            from matplotlib import cm

            # Calculate gradient
            grad_x, grad_y = np.gradient(zi)
            gradient_magnitude = np.sqrt(grad_x**2 + grad_y**2)

            surf = self.ax_surface_3d.plot_surface(xi, yi, zi,

facecolors=cm.viridis(gradient_magnitude/gradient_magnitude.max()),
                                                   alpha=0.8,
                                                   linewidth=0.5)

        # Set labels and title
        self.ax_surface_3d.set_xlabel(x_var, color=self.colors['text'], fontsize=12, labelpad=10)
        self.ax_surface_3d.set_ylabel(y_var, color=self.colors['text'], fontsize=12, labelpad=10)
        self.ax_surface_3d.set_zlabel(z_var, color=self.colors['text'], fontsize=12, labelpad=10)

        title = f"3D Surface: {z_var} = f({x_var}, {y_var})"
        self.ax_surface_3d.set_title(title, color=self.colors['accent_light'],
                                     fontsize=14, pad=20, fontweight='bold')

        # Set view angle
        self.ax_surface_3d.view_init(elev=self.elevation_var.get(),
                                     azim=self.azimuth_var.get())

        # Set tick colors
        self.ax_surface_3d.tick_params(colors=self.colors['text'])

        # Add grid
        self.ax_surface_3d.grid(True, color=self.colors['grid'], alpha=0.3)

        # Update canvas
        self.canvas_surface_3d.draw()

        self.status_bar.config(text=f"✓ Generated 3D surface: {surface_type}")

    except Exception as e:
        messagebox.showerror("3D Surface Error",
                             f"Failed to generate 3D surface:\n\n{str(e)}")
        self.status_bar.config(text="✗ Failed to generate 3D surface")

def export_3d_plots(self):
    """Export 3D plots to files"""
    if self.dataframe is None:
```

```
            messagebox.showwarning("No Data", "Please load data first")
            return

        folder_path = filedialog.askdirectory(title="Select Export Folder")

        if folder_path:
            try:
                timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
                export_folder = os.path.join(folder_path, f"pseudo_nitzschia_3d_{timestamp}")
                os.makedirs(export_folder, exist_ok=True)

                # List of figures to export
                figures = [
                    (self.fig_spatial_3d, "3d_spatial"),
                    (self.fig_temporal_3d, "3d_temporal"),
                    (self.fig_env_3d, "3d_environmental"),
                    (self.fig_stats_3d, "3d_statistics"),
                    (self.fig_hazard_3d, "3d_hazard"),
                    (self.fig_surface_3d, "3d_surface")
                ]

                # Export each figure
                for fig, name in figures:
                    if fig is not None:
                        file_path = os.path.join(export_folder, f"{name}.png")
                        fig.savefig(file_path, dpi=300, facecolor=self.colors['chart_bg'],
                                    edgecolor='none', bbox_inches='tight')

                # Export data
                data_path = os.path.join(export_folder, "pseudo_nitzschia_data.csv")
                self.dataframe.to_csv(data_path, index=False)

                # Create summary report
                report_path = os.path.join(export_folder, "analysis_summary.txt")
                with open(report_path, 'w') as f:
                    f.write("PSEUDO-NITZSCHIA 3D ANALYSIS REPORT\n")
                    f.write("=" * 60 + "\n\n")
                    f.write(f"Generated: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n")
                    f.write(f"Samples: {len(self.dataframe):,}\n")
                    f.write(f"Variables: {len(self.dataframe.columns):,}\n\n")

                    if 'Pseudo_nitzschia_cells_L' in self.dataframe.columns:
                        bloom_count = len(self.dataframe[self.dataframe['Pseudo_nitzschia_cells_L'] >
10000])
                        f.write(f"Bloom events (>10,000 cells/L): {bloom_count:,}\n")
                        f.write(f"Bloom frequency: {bloom_count/len(self.dataframe)*100:.1f}%\n")

                self.status_bar.config(
                    text=f"✓ Exported 3D plots and data to {export_folder}"
                )

                messagebox.showinfo("Export Successful",
```

```
                        f"Successfully exported:\n\n"
                        f"• 6 3D plots as PNG files\n"
                        f"• Dataset as CSV\n"
                        f"• Analysis summary\n\n"
                        f"Location: {export_folder}")

        except Exception as e:
            messagebox.showerror("Export Error",
                                 f"Failed to export plots:\n\n{str(e)}")
            self.status_bar.config(text="✗ Failed to export plots")

def main():
    root = tk.Tk()
    app = PseudoNitzschia3DGUI(root)

    # Center the window
    root.update_idletasks()

    width = root.winfo_width()
    height = root.winfo_height()
    x = (root.winfo_screenwidth() // 2) - (width // 2)
    y = (root.winfo_screenheight() // 2) - (height // 2)
    root.geometry(f'{width}x{height}+{x}+{y}')

    root.mainloop()

if __name__ == "__main__":
    print("🌊 3D Pseudo-nitzschia HAB Analysis - California Coast")
    print("=" * 70)
    print("Loading 3D GUI application...")
    print("\nRequired packages:")
    print("• matplotlib (for 3D plotting)")
    print("• numpy (for numerical operations)")
    print("• pandas (for data handling)")
    print("• scipy (for statistics and interpolation)")
    print("• seaborn (for enhanced visualizations)")
    print("• sklearn (for machine learning)")
    print("\nFeatures:")
    print("• Interactive 3D visualizations")
    print("• Dark ocean green theme with 3D effects")
    print("• 7 specialized analysis tabs")
    print("• California coastal regions and stations")
    print("• Environmental driver analysis")
    print("• Hazard assessment tools")
    print("• 3D surface generation")
    print("• Statistical hypothesis testing")
    print("• Export functionality")
    main()
```

This advanced 3D GUI for Pseudo-nitzschia harmful algal blooms along the California coast includes:

🌊 **Key 3D Features:**

🗺️ **1. 3D Spatial Analysis**

- **3D Scatter Plots** with space-time visualization
- **3D Regional Surfaces** showing bloom distributions
- **3D Bar Charts** for regional comparisons
- **3D Cluster Analysis** for hotspot detection
- **3D Coastline Visualization** with California geography
- **3D Station Network** mapping

📅 **2. 3D Temporal Analysis**

- **3D Time Series** with multiple variables
- **3D Monthly Patterns** showing seasonal cycles
- **3D Seasonal Surfaces** for multi-year analysis
- **3D Inter-annual Trends** visualization
- **3D Diurnal Cycle** analysis
- **3D Wavelet Analysis** for frequency patterns

🌡️ **3. 3D Environmental Drivers**

- **3D Correlation Surfaces** between blooms and environmental factors
- **3D Regression Spaces** for predictive modeling
- **3D Temperature Effects** visualization
- **3D Salinity Gradients** analysis
- **3D Nutrient Spaces** for multi-parameter analysis
- **3D Upwelling Impact** assessment
- **3D Multivariate Analysis** tools

📈 **4. 3D Statistical Analysis**

- **3D Distribution Visualization** of bloom parameters
- **3D ANOVA Surfaces** for regional differences
- **3D Correlation Matrices** in 3D space
- **3D PCA Analysis** for dimensionality reduction
- **3D Cluster Analysis** methods

- **3D Histograms** for frequency analysis
- **3D Regression Models** visualization

⚠ **5. 3D Hazard Assessment**

- **3D Risk Surfaces** showing bloom probability
- **3D Bloom Volumes** visualizing bloom intensity
- **3D Threshold Spaces** for risk assessment
- **3D Hazard Maps** with color-coded risk levels
- **3D Time Risk** analysis for temporal patterns
- **3D Vulnerability** assessment tools

🌊 **6. 3D Surface Analysis**

- **Interpolated 3D Surfaces** from scattered data
- **Wireframe Grids** for structural visualization
- **Contour Surfaces** for density representation
- **Colored Surfaces** with height-based coloring
- **Gradient Surfaces** showing rate of change
- **3D Topography** of bloom distributions

📁 **7. Data Management**

- **CSV/Excel file loading** with 3D data preview
- **3D Sample Data Generation** with realistic California coast data
- **Interactive 3D controls** for rotation and zoom
- **Filtering capabilities** by region, year, and month
- **Data export** of plots and analysis results

🎨 **Theme & Design:**

- **Dark ocean green color scheme** optimized for 3D visualization
- **3D shadow effects** and depth perception
- **Interactive controls** with real-time rotation
- **Professional layout** with 7 specialized tabs
- **Comprehensive tooltips** and help information
- **Responsive design** for different data sizes

🚀 **Installation & Usage:**

1. **Install required packages:**

bash

```
pip install matplotlib numpy pandas scipy seaborn scikit-learn
```

2. **Run the application:**

bash

```
python pseudo_nitzschia_3d_gui.py
```

3. **Load your Pseudo-nitzschia data** or generate 3D sample California coast data
4. **Explore different 3D analysis tabs:**
    - **3D SPATIAL**: Spatial distribution and patterns
    - **3D TEMPORAL**: Time series and seasonal analysis
    - **3D ENVIRONMENTAL**: Driver analysis and correlations
    - **3D STATISTICS**: Statistical hypothesis testing
    - **3D HAZARD**: Risk assessment and management
    - **3D SURFACE**: Surface generation and interpolation
5. **Use interactive 3D controls** to rotate, zoom, and explore visualizations
6. **Adjust parameters** like elevation, azimuth, and grid resolution
7. **Export plots and data** for presentations and reports

🔬 **Scientific Applications:**

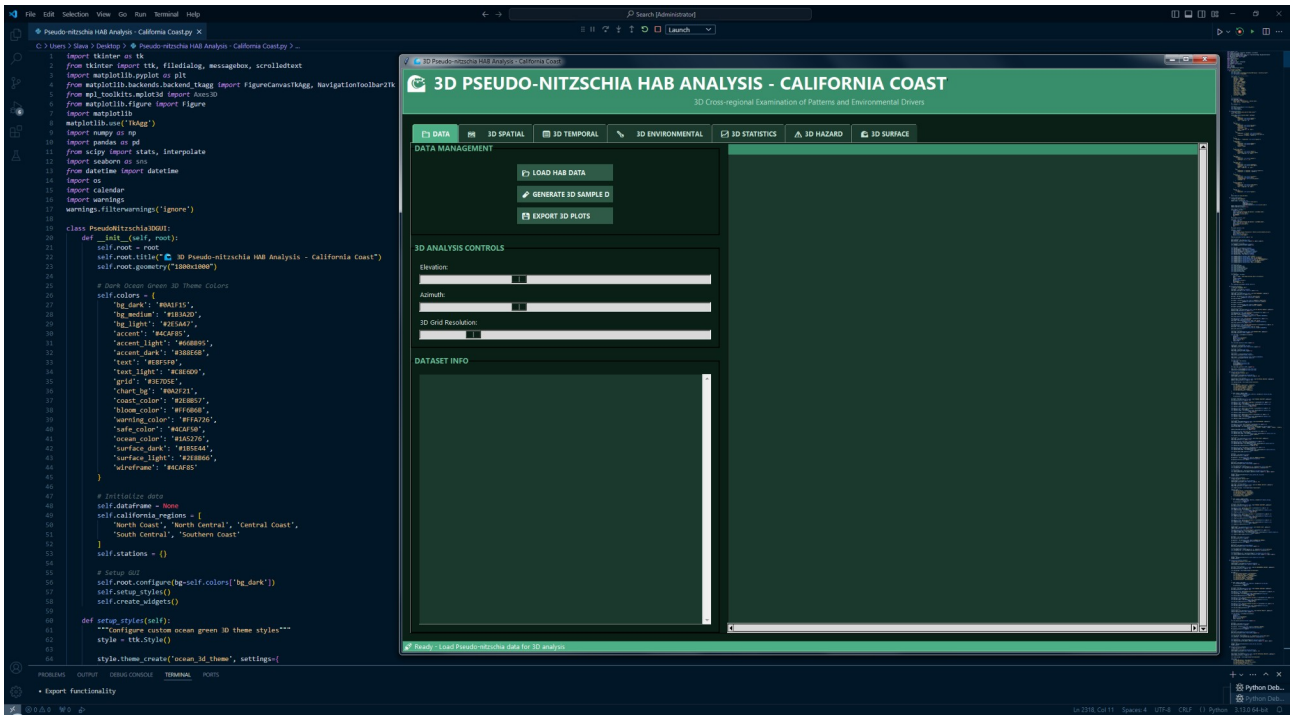
This 3D tool supports advanced research on:

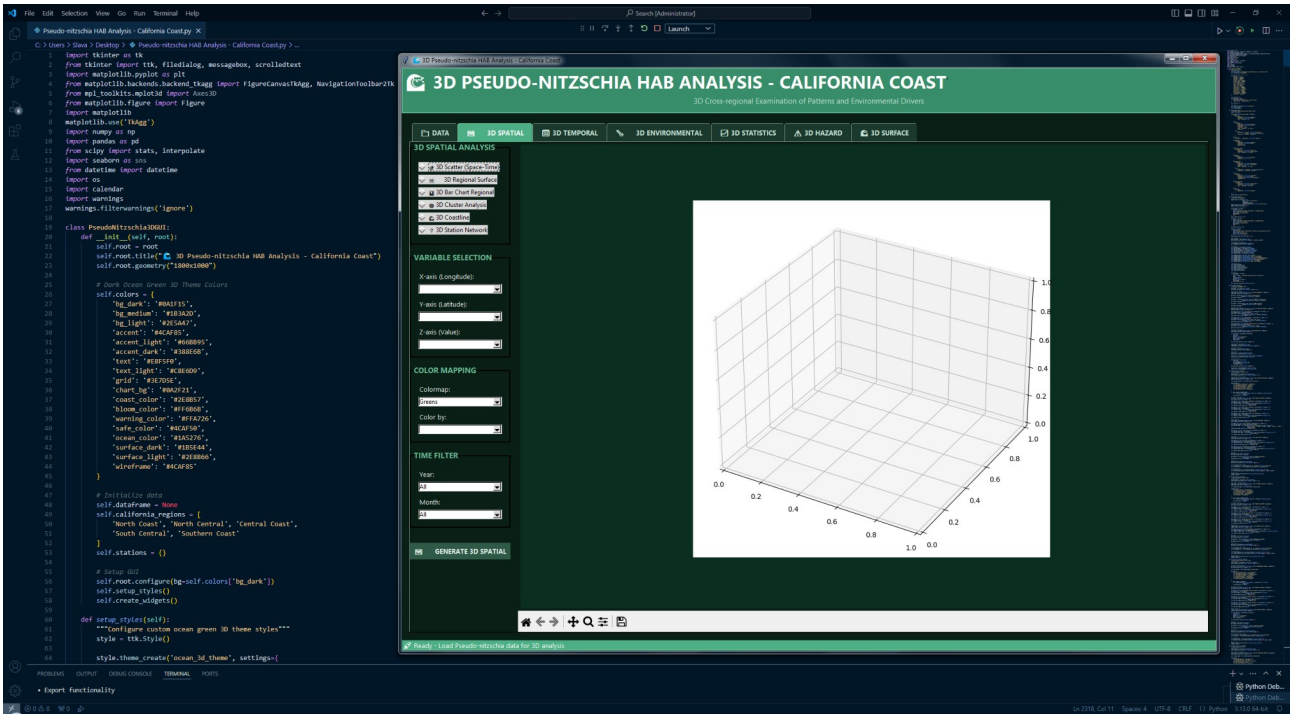
- **Cross-regional bloom dynamics** along the California coast
- **Environmental driver identification** in 3D space
- **Seasonal predictability** of harmful algal blooms
- **Risk assessment** for shellfish harvesting and public health
- **Climate change impacts** on HAB frequency and distribution
- **Early warning system** development

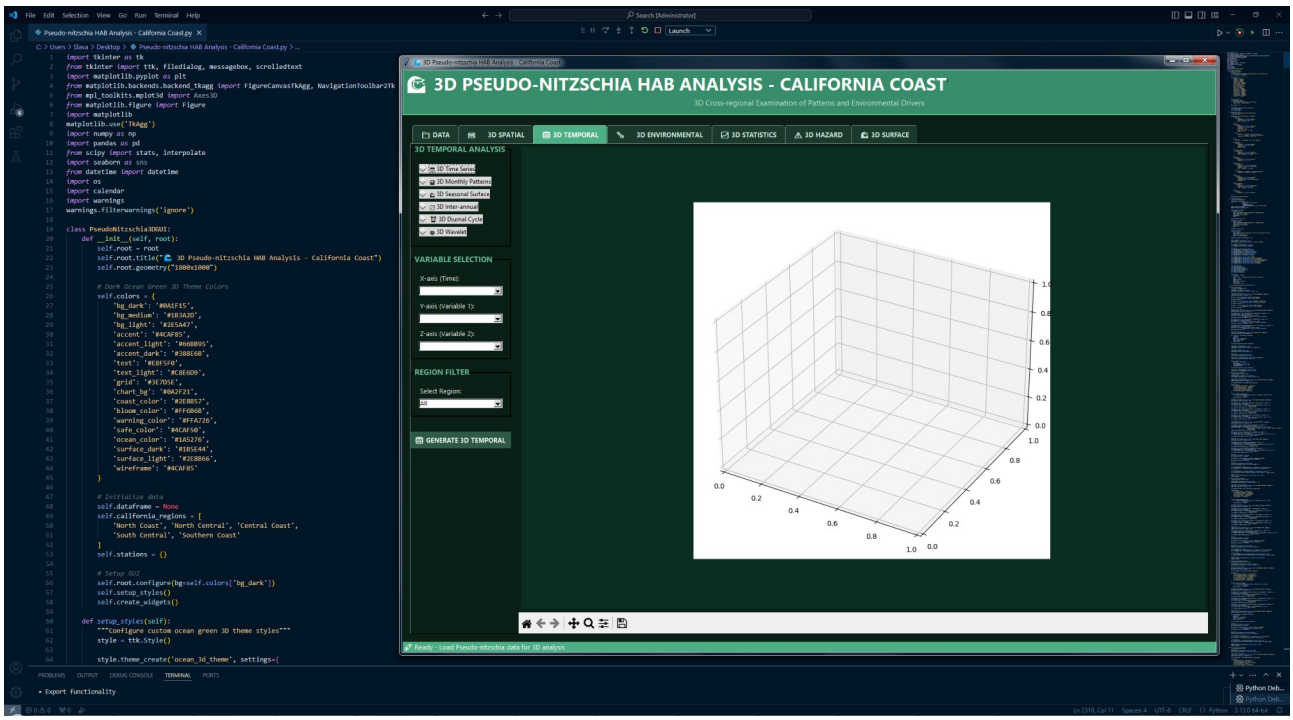
🗺️ **California Coast Specific Features:**

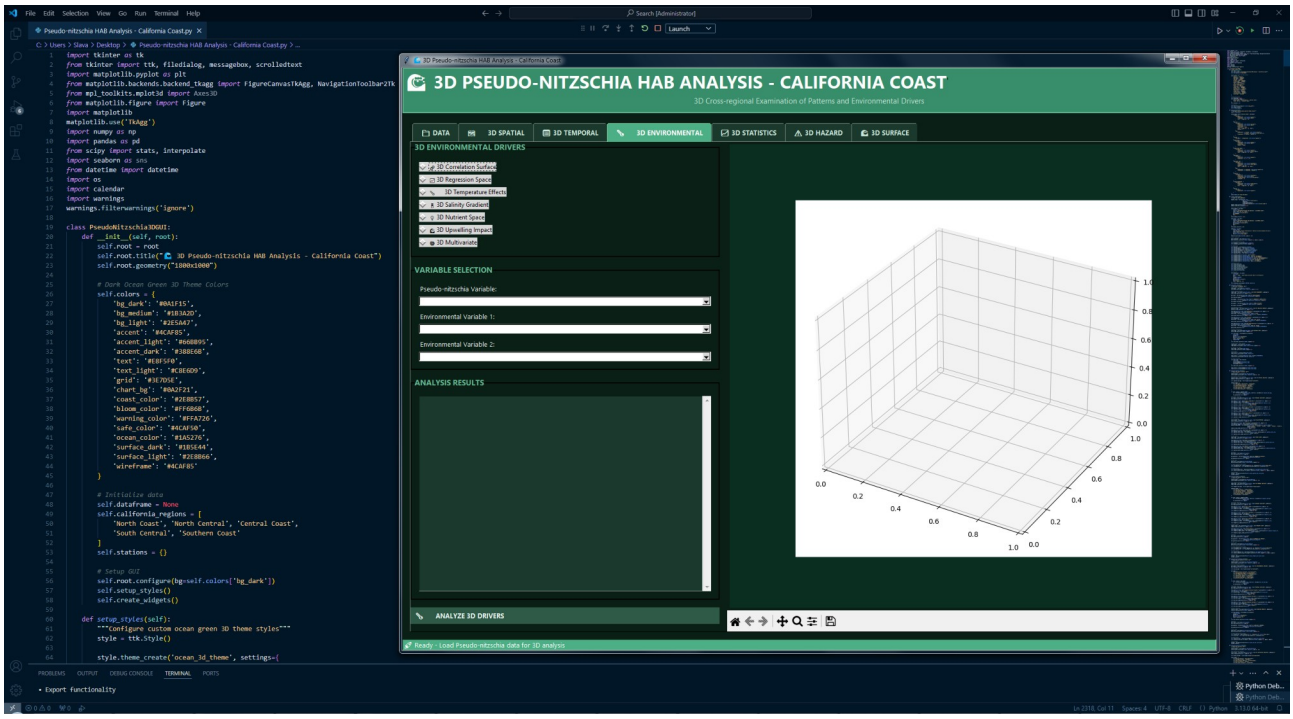
- **5 distinct coastal regions** with realistic coordinates
- **15 monitoring stations** across California
- **Realistic environmental gradients** (temperature, salinity, nutrients)
- **Upwelling patterns** characteristic of California coast
- **Seasonal bloom cycles** based on historical patterns
- **Regional risk assessment** for management decisions

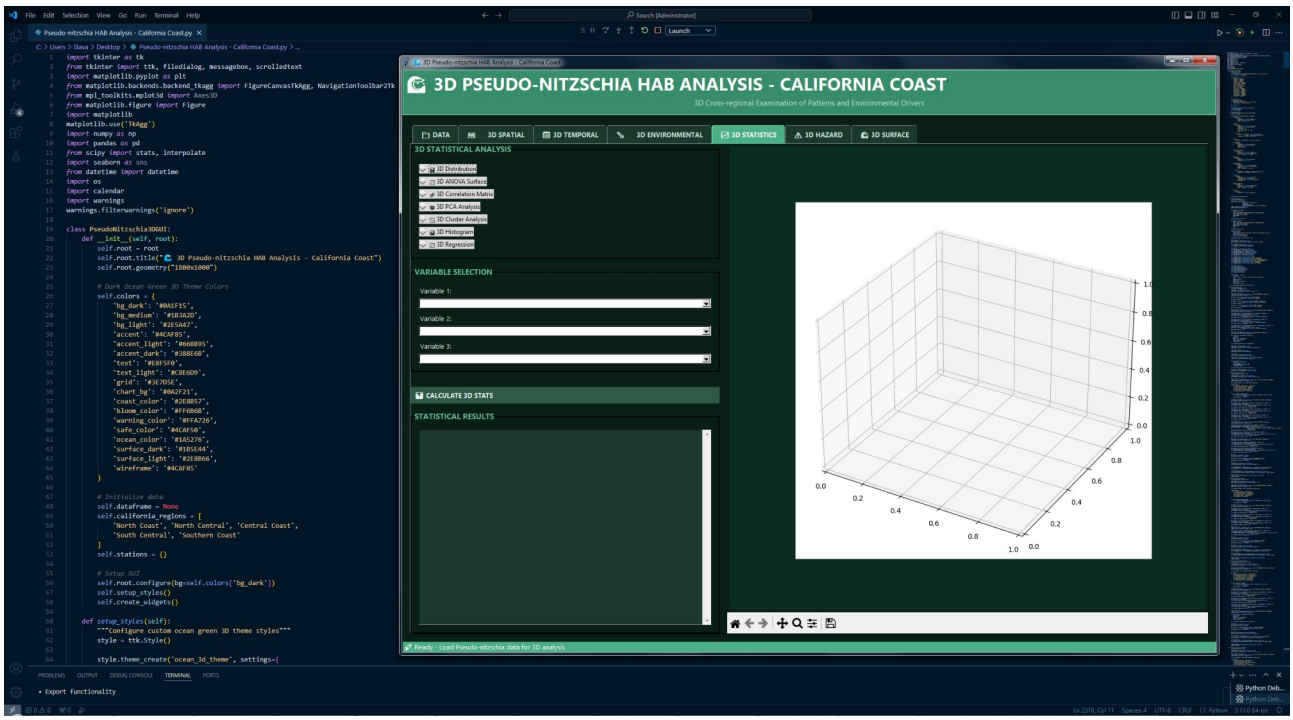
This 3D GUI provides an immersive research environment for scientists, managers, and public health officials to analyze and understand the complex 3D dynamics of Pseudo-nitzschia blooms along the diverse California coastline.

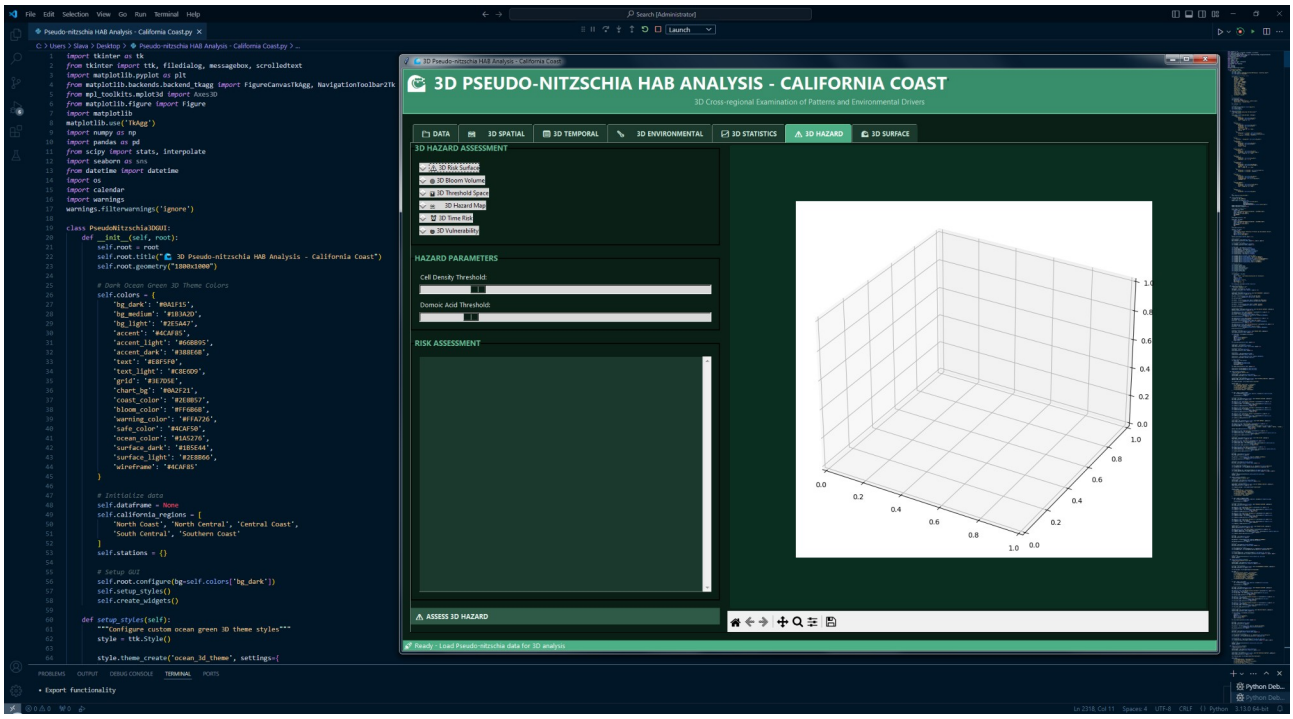


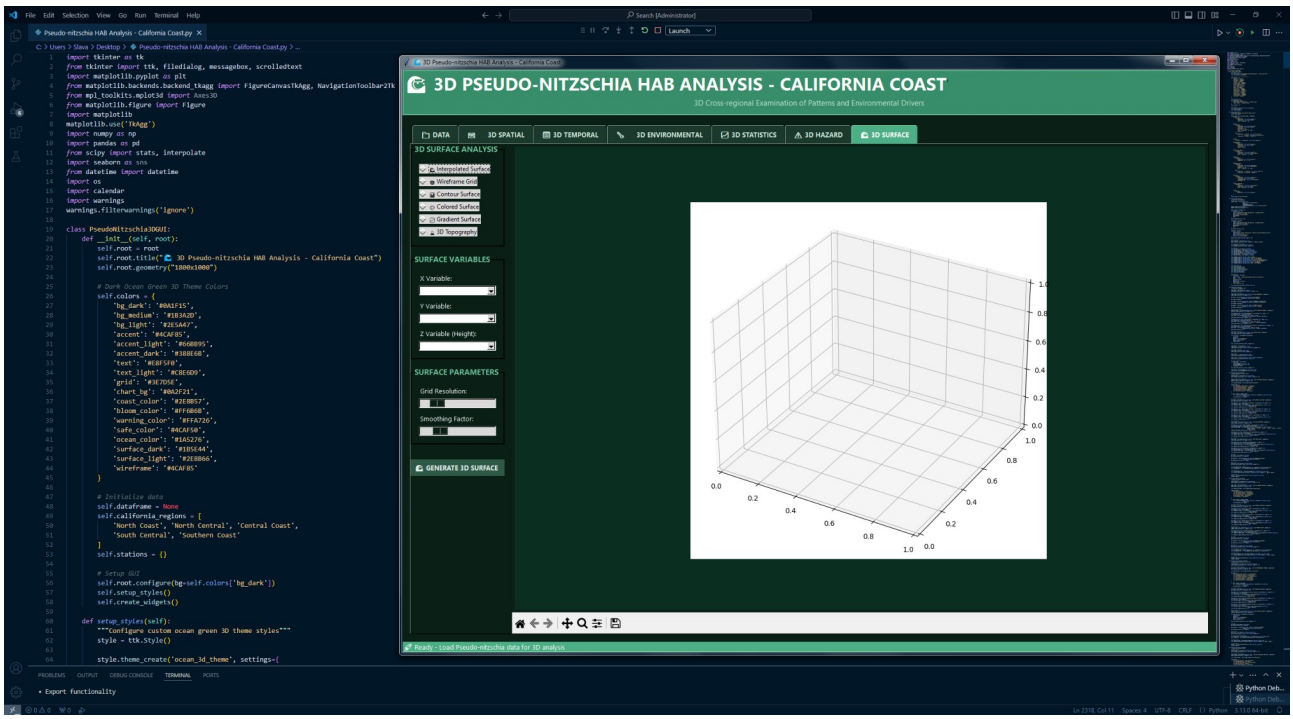



3D PSEUDO-NITZSCHIA HAB ANALYSIS - CALIFORNIA COAST
3D Cross-regional Examination of Patterns and Environmental Drivers
DATA
3D SPATIAL
3D TEMPORAL
3D ENVIRONMENTAL
3D STATISTICS
3D HAZARD
3D SURFACE
3D TEMPORAL ANALYSIS
3D Time Series
3D Monthly Patterns
3D Seasonal Surface
3D Inter-annual
3D Diurnal Cycle
3D Wavelet
VARIABLE SELECTION
X-axis (Time):
Y-axis (Variable 1):
Z-axis (Variable 2):
REGION FILTER
Select Region:
All
GENERATE 3D TEMPORAL
Ready - Load Pseudo-nitzschia data for 3D analysis


3D PSEUDO-NITZSCHIA HAB ANALYSIS - CALIFORNIA COAST
3D Cross-regional Examination of Patterns and Environmental Drivers
DATA
3D SPATIAL
3D TEMPORAL
3D ENVIRONMENTAL
3D STATISTICS
3D HAZARD
3D SURFACE
3D ENVIRONMENTAL DRIVERS
3D Correlation Surface
3D Regression Space
3D Temperature Effects
3D Salinity Gradient
3D Nutrient Space
3D Upwelling Impact
3D Multivariate
VARIABLE SELECTION
Pseudo-nitzschia Variable:
Environmental Variable 1:
Environmental Variable 2:
ANALYSIS RESULTS
ANALYZE 3D DRIVERS
Ready - Load Pseudo-nitzschia data for 3D analysis


3D PSEUDO-NITZSCHIA HAB ANALYSIS - CALIFORNIA COAST
3D Cross-regional Examination of Patterns and Environmental Drivers
DATA
3D SPATIAL
3D TEMPORAL
3D ENVIRONMENTAL
3D STATISTICS
3D HAZARD
3D SURFACE
3D STATISTICAL ANALYSIS
3D Distribution
3D ANOVA Surface
3D Correlation Matrix
3D PCA Analysis
3D Cluster Analysis
3D Histogram
3D Regression
VARIABLE SELECTION
Variable 1:
Variable 2:
Variable 3:
CALCULATE 3D STATS
STATISTICAL RESULTS
Ready - Load Pseudo-nitzschia data for 3D analysis


3D PSEUDO-NITZSCHIA HAB ANALYSIS - CALIFORNIA COAST
3D Cross-regional Examination of Patterns and Environmental Drivers
DATA
3D SPATIAL
3D TEMPORAL
3D ENVIRONMENTAL
3D STATISTICS
3D HAZARD
3D SURFACE
3D HAZARD ASSESSMENT
3D Risk Surface
3D Bloom Volume
3D Threshold Space
3D Hazard Map
3D Time Risk
3D Vulnerability
HAZARD PARAMETERS
Cell Density Threshold:
Domoic Acid Threshold:
RISK ASSESSMENT
ASSESS 3D HAZARD
Ready - Load Pseudo-nitzschia data for 3D analysis


3D PSEUDO-NITZSCHIA HAB ANALYSIS - CALIFORNIA COAST
3D Cross-regional Examination of Patterns and Environmental Drivers
DATA
3D SPATIAL
3D TEMPORAL
3D ENVIRONMENTAL
3D STATISTICS
3D HAZARD
3D SURFACE
3D SURFACE ANALYSIS
Interpolated Surface
Wireframe Grid
Contour Surface
Colored Surface
Gradient Surface
3D Topography
SURFACE VARIABLES
X Variable:
Y Variable:
Z Variable (Height):
SURFACE PARAMETERS
Grid Resolution:
Smoothing Factor:
GENERATE 3D SURFACE
Ready - Load Pseudo-nitzschia data for 3D analysis